

Tahoe

User Guide

Input Version 3.4.1

last updated September 27, 2026



Abstract

Tahoe is a general purpose simulation code incorporating finite element, meshfree methods, and atomistic methods. The code is capable of execution on parallel platforms and includes interfaces to third-party libraries that can solve linear systems of equations in parallel. This document is intended as a guide for users. It does not contain the detailed descriptions of the code required by developers. This guide is a work in progress. Please send questions and bug reports to tahoe-help@sandia.gov.

Contents

1	Introduction	11
1.1	Conventions	11
1.2	Overview of Tahoe components	11
1.2.1	Tahoe manager	12
1.2.2	Time manager	12
1.2.3	Controller	12
1.2.4	Node manager	13
1.2.5	Element groups	13
1.2.6	I/O manager	13
1.2.7	Solver	13
1.3	Supported platforms	13
2	Execution procedures	15
2.1	Building Tahoe with CMake	15
2.2	Modes of execution	15
2.3	Serial analysis	16
2.4	Parallel analysis	16
2.5	Domain decomposition	18
2.6	Joining results files	19
3	Regression tests and benchmarks	20
3.1	Benchmark levels	20
3.2	Running a level	20
3.3	Meshfree Kirchhoff–Love shell	21
3.4	Repeatability of tests across platforms	22
3.5	Repeatability of tests during parallel execution	22
4	Input file version 3.4.1	23
4.1	Global analysis parameters	24
4.2	Time parameters	25
4.2.1	Time sequence parameters	25
4.2.2	Schedule functions	26
4.3	Nodal parameters	26
4.4	Element parameters	29
4.5	Solver parameters	29
4.6	Batch file format	29
5	Analysis types	30
5.1	Analysis types	30
5.1.1	Elastostatic analysis	30
5.1.2	Implicit elastodynamic analysis	31

5.1.3	Explicit elastodynamic analysis	31
5.1.4	Steady-state and transient diffusion	31
5.2	Solution techniques	31
5.2.1	Linear	31
5.2.2	Nonlinear	32
5.2.2.1	Standard Newton	33
5.2.2.2	Initial tangent	33
5.2.2.3	Modified Newton	33
5.2.2.4	Dynamic relaxation	33
5.2.2.5	Newton with line search	33
5.2.2.6	Preconditioned nonlinear conjugate gradient	33
5.2.2.7	Interactive Newton	34
6	Initial conditions and boundary conditions	35
6.1	Nodal initial conditions	35
6.2	Nodal kinematic boundary conditions	35
6.3	Kinematic boundary condition controllers	36
6.3.1	K-field	37
6.3.2	Interface K-field	37
6.3.3	Mapped displacements	37
6.3.4	Tied displacements	37
6.3.5	Periodic displacements	37
6.3.6	Scaled velocity	37
6.3.7	Torsion	37
6.3.8	Conveyor	37
6.3.9	Direct nodal collocation	37
6.4	Applied nodal forces	37
6.5	Force controllers	37
6.5.1	Rigid barrier (penalty method)	38
6.5.2	Rigid sphere (penalty method)	38
6.5.3	Rigid sphere (augmented Lagrangian method)	38
6.5.4	Rigid sphere for meshfree domains (penalty method)	38
6.5.5	Rigid barrier (augmented Lagrangian method)	38
6.5.6	Rigid cylinder (penalty and augmented Lagrangian methods)	38
6.5.7	Kinematic constraints on meshfree nodes (augmented Lagrangian method)	38
6.5.8	Pressure boundary condition	38
6.5.9	Inclined kinematic condition (penalty method)	38
6.5.10	Prescribed physical displacement on meshfree fields (penalty method)	38
6.6	Essential boundary conditions for meshfree fields	39
6.6.1	Direct nodal collocation	39
6.6.2	Penalty enforcement of physical displacements	39
6.7	Body forces	40

6.8	Natural boundary conditions	40
7	Element types	42
7.1	Integration domains	42
7.2	Meshfree methods	42
7.2.1	Reproducing Kernel Particle Method (RKPM)	45
7.3	Meshfree Kirchhoff–Love shell	47
7.3.1	Formulation	47
7.3.2	Input parameters	48
7.3.3	Output variables	50
7.3.4	Example	50
7.3.5	Verification	50
7.4	Solid deformation	51
7.4.1	Element 29: finite strain element with enhanced modes	51
7.4.2	Element 36: finite strain mixed element	51
7.4.3	Output variables	52
7.4.4	Constitutive models	52
7.5	Diffusion	52
7.5.1	Output variables	52
7.5.2	Constitutive models	52
7.6	Cohesive surface elements	52
7.6.1	Output variables	53
7.6.1.1	Nodal output variables	53
7.6.1.2	Element output variables	54
7.6.2	Constitutive models	54
7.7	Cohesive surface elements for meshfree domains	55
7.7.1	Constitutive models	55
7.7.2	Parameters for adaptive insertion	55
7.8	Contact	55
7.8.1	Penalty formulation	55
7.8.2	Augmented Lagrangian formulation	55
7.9	Pair-potential	56
7.10	Two- and three-body potential	56
7.11	Surface-surface interactions	56
7.11.1	Element 37: two-point surface interactions	56
8	Constitutive models for solid mechanics	61
8.0.1	Common input parameters	62
8.0.2	Common input parameters for 2D analysis	63
8.1	Material 1: small deformation Kirchhoff–St. Venant	63
8.1.1	3D input parameters	64
8.1.2	2D input parameters	65

8.1.3	Output values	65
8.2	Material 2: finite deformation Kirchhoff-St. Venant	65
8.2.1	3D input parameters	66
8.2.2	2D input parameters	66
8.2.3	Output values	66
8.3	Material 3: small deformation cubic	66
8.3.1	3D input parameters	67
8.3.2	2D input parameters	68
8.3.3	Output values	68
8.4	Material 4: finite deformation cubic	68
8.4.1	3D input parameters	69
8.4.2	2D input parameters	69
8.4.3	Output values	70
8.5	Material 5: uncoupled volumetric/deviatoric	70
8.5.1	3D input parameters	70
8.5.2	2D input parameters	71
8.5.3	Output values	71
8.6	Material 6: quadratic-logarithmic (Simo)	71
8.6.1	3D input parameters	72
8.6.2	2D input parameters	72
8.6.3	Output values	73
8.7	Material 7: quadratic-logarithmic (Ogden)	73
8.7.1	3D input parameters	73
8.7.2	2D input parameters	74
8.7.3	Output values	74
8.8	Material 8: small deformation J_2 plasticity	74
8.8.1	3D input parameters	75
8.8.2	2D input parameters	76
8.8.3	Output values	76
8.9	Material 9: finite deformation J_2 plasticity	77
8.9.1	3D input parameters	78
8.9.2	2D input parameters	79
8.9.3	Output values	79
8.9.4	Hardening functions	79
8.9.4.1	linear	80
8.9.4.2	linear-exponential saturation	80
8.9.4.3	general cubic spline	81
8.10	Material 10: finite deformation J_2 plasticity in principal stretches	82
8.10.1	3D input parameters	83
8.10.2	2D input parameters	83
8.10.3	Output values	84
8.11	Material 11: small deformation Drucker-Prager plasticity	84

8.11.1	3D input parameters	85
8.11.2	2D input parameters	86
8.11.3	Output values	86
8.12	Material 12: 2D hexagonal Lennard-Jones lattice	86
8.12.1	3D input parameters	89
8.12.2	2D input parameters	89
8.12.3	Output values	89
8.13	Material 13: plane strain FCC Lennard-Jones lattice	90
8.13.1	3D input parameters	90
8.13.2	2D input parameters	90
8.13.3	Output values	90
8.14	Material 14: FCC lattice with EAM potentials	90
8.14.1	3D input parameters	91
8.14.2	2D input parameters	92
8.14.3	Output values	92
8.15	Material 15: Stillinger-Weber diamond cubic	92
8.15.1	3D input parameters	94
8.15.2	2D input parameters	94
8.15.3	Output values	94
8.16	Material 16: Virtual Internal Bond model	94
8.16.1	3D input parameters	98
8.16.2	2D input parameters	98
8.16.3	Output values	98
8.17	Material 17: isotropic Virtual Internal Bond model (Simo)	99
8.17.1	3D input parameters	99
8.17.2	2D input parameters	99
8.17.3	Output values	99
8.18	Material 18: isotropic Virtual Internal Bond model (Ogden)	99
8.18.1	3D input parameters	99
8.18.2	2D input parameters	99
8.18.3	Output values	99
8.19	Material 19: isotropic Virtual Internal Bond model with J_2 plasticity	100
8.19.1	Output values	100
8.80	Material 80: ABAQUS/Standard UMAT BCJ	101
8.80.1	3D input parameters	101
8.80.2	2D input parameters	102
8.80.3	Output values	102
9	Constitutive models for cohesive surface elements	103
9.0.1	Background	103
9.1	Surface Material 0: Xu-Needleman (elastic)	104
9.1.1	Input parameters	104

9.1.2	Output values	105
9.2	Surface Material 1: Tvergaard-Hutchinson (elastic)	105
9.2.1	Input parameters	105
9.2.2	Output values	106
9.3	Surface Material 2: Linear Damage Model	106
9.3.1	Input parameters	106
9.3.2	Output values	106
9.4	Surface Material 3: Tvergaard-Huthcinson with viscous dissipation	106
9.4.1	Input parameters	107
9.4.2	Output values	107
9.5	Surface Material 4: Tijssens' crazing model (elastic-plastic)	107
9.5.1	Input Parameters	109
9.5.2	Output Values	109
9.6	Surface Material 5: trilinear rate dependent	109
9.6.1	Input parameters	110
9.6.2	Output values	110
9.7	Surface Material 6: Tied Potential	110
9.7.1	Input parameters	111
9.7.2	Output values	112
9.8	Material 7: Yoon, Allen, and Searcy (linear viscoelastic)	112
9.8.1	Input parameters	114
9.8.2	Output values	114
10	Geometry and results files	115
10.1	Geometry file formats	116
10.1.1	TahoeI format	116
10.1.1.1	Coordinate list	116
10.1.1.2	Element block	117
10.1.1.3	Node set	117
10.1.1.4	Side set	117
10.1.2	TahoeII format	117
10.1.2.1	Version	118
10.1.2.2	Title	118
10.1.2.3	Dimensions	118
10.1.2.4	Node sets	119
10.1.2.5	Side sets	120
10.1.2.6	Element sets	121
10.1.2.7	Nodal coordinates	122
10.1.3	EnSight Gold format	123
10.1.4	ExodusII/Genesis format	123
10.1.5	ABAQUS Results Files format	123
10.1.6	MSC PATRAN Neutral format	123

10.1.7	Tahoe Results format	124
10.2	Results file formats	124
10.2.1	TecPlot	125
10.2.2	EnSight	125
10.2.3	ExodusII	126
10.2.4	ABAQUS Results	126
10.2.5	PATRAN Results	126
10.2.6	Fracture surface area	126
10.2.7	Node history output	126
11	History of updates to the Tahoe User Guide	127
11.1	History	127
11.2	Changes since Version 3.4	127
11.2.1	update 3.4.1	127
11.3	Modernisation track (post-3.4.1, GitHub)	127
11.3.0.1	Build system.	127
11.3.0.2	Activated Drucker–Prager plasticity (Material 11).	127
11.3.0.3	Implicit Coulomb friction in <code>PenaltyContact3DT</code> .	127
11.3.0.4	BonetTet element with lagged-J-bar and FD tangent.	128
11.3.0.5	New benchmarks.	128
11.3.0.6	Documentation infrastructure.	128
11.3.0.7	Meshfree Kirchhoff–Love shell.	128
11.3.0.8	Essential boundary conditions for meshfree fields.	129
11.3.0.9	Meshfree shell regression tests.	129
12	References	130

List of Figures

1.1	Tahoe components	12
4.1	Layout of a Tahoe parameters file.	23
7.1	Node, face, and node-on-face numbering for line integration domains.	42
7.2	Node, face, and node-on-face numbering for quadrilateral integration domains.	43
7.3	Node, face, and node-on-face numbering for triangular integration domains.	43
7.4	Node, face, and node-on-face numbering for hexahedral integration domains.	43
7.5	Node, face, and node-on-face numbering for tetrahedral integration domains.	44
7.6	Node, face, and node-on-face numbering for pentahedral integration domains.	44
7.7	Local coordinate frame	53
8.1	The Cauchy-Born rule	87
8.2	Deformation of a unit cell under the Cauchy-Born rule	88
8.3	Cubic unit cell of the diamond cubic lattice	92
8.4	Convergence to isotropic response for the plane stress, VIB model	95
8.5	Integration point arrangements in three dimensions	96
8.6	Orientational dependence of C_{1111}	97

List of Tables

3.1	Regression tests for the meshfree Kirchhoff–Love shell.	21
5.1	This is the list of types.	30
5.2	Matrix types for the global system of equations.	31
5.3	Nonlinear solution techniques.	32
6.1	Identification of nodes and boundaries.	35
6.2	Codes for nodal kinematic boundary conditions.	36
6.3	Kinematic boundary condition controllers.	36
6.4	Force boundary condition controllers.	38
6.5	Coordinate systems for natural boundary conditions.	41
7.3	Parameters of the <code>meshfree_kl_shell</code> element.	49
7.5	Verification of the meshfree Kirchhoff–Love shell against the reference paper [56].	51
7.1	Element types.	57
7.2	Integration domain geometries and Gauss rules.	58
7.4	Nodal output values for the meshfree Kirchhoff–Love shell.	58
7.6	Nodal output values for solid deformation.	58
7.7	Element output values for solid deformation.	59
7.8	Nodal output variables for isotropic cohesive surface elements.	59
7.9	Nodal output variables for anisotropic cohesive surface elements.	59
7.10	Element output variables for cohesive surface elements.	60
8.1	Materials for analysis of displacements and stress.	61
8.1	Materials for analysis of displacements and stress.	62
8.2	Constraint options for two-dimensional analysis.	63

8.3	Output values for a small strain J_2 plasticity model.	76
8.4	Output values for a small strain J_2 plasticity model.	80
8.5	Isotropic hardening functions.	80
8.6	Output values for a finite strain J_2 plasticity model.	84
8.7	Output values for the small strain Drucker-Prager plasticity model.	87
8.8	FCC lattice orientation options.	90
8.9	Embedded atom method potentials.	91
8.10	Diamond cubic lattice orientation options.	94
9.1	Constitutive models for cohesive surface elements	103
10.1	Geometry input file formats.	115
10.2	Result output file formats.	115
10.3	Keywords for the TahoeII file format.	118
10.4	File extensions for results files.	125

1 Introduction

Tahoe is a general purpose simulation code. It originally included only finite element methods, but has been extended to include meshfree methods, atomistics, and cohesive surface approaches to modeling fracture. Aside from interfaces for adding element formulations and constitutive models, Tahoe also supports specialized boundary conditions such as K -fields and rigid boundaries of various geometries. The code has also been extended parallel execution.

The purpose of this guide is to explain how to run Tahoe in its various modes. Tahoe execution is driven by a parameters file. The layout of this file is described in Section 4. The parameters in the file are scanned sequentially. The interceding chapters provide some additional information about the various formulations available for execution. This guide is not intended as a guide of developers. Although some references to specific classes do appear throughout this guide, the structure of the code itself is not explained in any detail. Additional information about the code itself can be obtained from the Tahoe Development Server at

[<http://tahoe.ca.sandia.gov>](http://tahoe.ca.sandia.gov)

The original Sandia development server is no longer maintained. Active development now takes place on GitHub at

<https://github.com/samanseifi/tahoe>

The repository is cloned and built directly with `git` and CMake — there is no longer a separate `tahoe-manager` or SourceForge/CVS workflow. See Section 2 for build and execution procedures. This guide contains hypertext links. These can be used to navigate the guide if your document viewer support links.

1.1 Conventions

[To do: describe conventions used for this guide.]

1.2 Overview of Tahoe components

Although this guide will not discuss the implementation of Tahoe in any detail, some discussion of its internal organization is included to provide a rationale for the organization of input parameters. Internally, Tahoe is organized into the components shown in Figure 1.1. Data and parameters are “owned” by specific components of the code. A brief description of the components follows. For further information on a given capability, identify the component whose scope covers the capability and seek more information in the later sections of this guide.

Tahoe is a hierarchical collection of components, or managers. Each manager is responsible for specific data and functionality. The configuration of each manager is set in the parameters input file. A brief description of each follows.

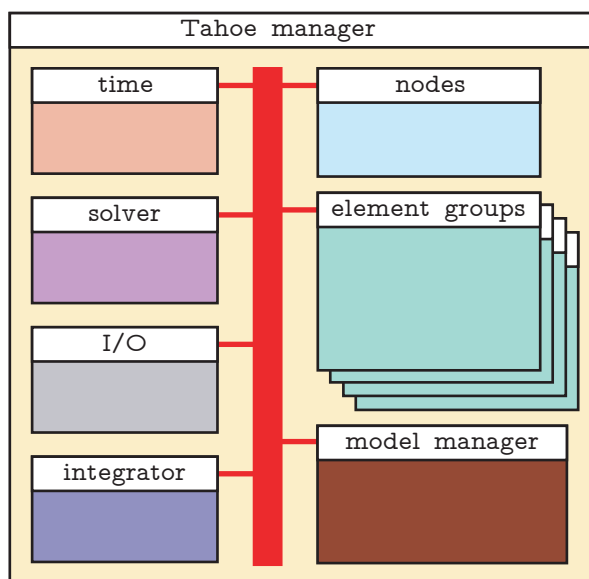


Figure 1.1: Division of functionality and data within Tahoe.

1.2.1 Tahoe manager

The **Tahoe manager** sits at the top of the manager hierarchy. It is responsible for first configuring other managers during the initialization phase of a simulation, checking for consistency and compatibility of input parameters, and second serving as the back bone for communication between the managers during the solution phase. Additionally, the **Tahoe manager** is the “last hope” for trapping and correcting exceptional conditions that occur during the solution phase.

1.2.2 Time manager

The **time manager** is responsible for all parameters related to the solution progress variable, or time. This includes information about the initial time, final time, and time step. Boundary conditions may be specified as functions of time. These schedules are specified to the **time manager**. With solution procedures for which it is supported, the **time manager** handles automatic time step control.

1.2.3 Controller

Tahoe supports a number of time integration schemes that are handled by the **controller**. There are no input parameters for directly affecting the configuration of the **controller**. The time integration scheme is determined by parameters specified to the **Tahoe manager** and the **time manager**.

1.2.4 Node manager

The **node manager** is responsible for handling the fields of unknowns, i.e. $\mathbf{u}(\mathbf{X}, t)$. Depending on the type of analysis, this data includes the field variables and their derivatives at the current time step t_n , the last time t_{n-1} , and at other times t_{n-N} for recovery from exceptional conditions that occur during the solution phase. The **node manager** handles essential boundary conditions and specification of the externally applied conjugate forces.

1.2.5 Element groups

An **element group** in **Tahoe** generally refers to the implementation of any formulation that translates nodal field variables to their conjugate forces. For most simulations, a majority of the parameters provided to **Tahoe** are concerned with the **element groups**. For finite element types, these parameters would include specifications of the element geometry and integration scheme for the element-level variational equations. The specification of constitutive parameters is embedded in the element data although the types of constitutive models will depend on the particular element. The elements supported by **Tahoe** are listed in Table 7.1.

1.2.6 I/O manager

The **I/O manager** is responsible for supporting the various input formats for geometry information and the various output formats for results data. There are no input parameters for directly affecting the **I/O manager**. I/O file formats are specified in the parameters for the **Tahoe manager**. A description of supported file formats is given in Section 10.

1.2.7 Solver

The **solver** is responsible bringing the system to equilibrium

$$\mathbf{F}^{\text{int}}(\mathbf{u}) = \mathbf{F}^{\text{ext}}(t).$$

Depending on the type of analysis, the **solver** may require specifications of the nonlinear solution algorithm and convergence tolerances. Parameters for solvers for linear system of equations is embedded in the **solver** data, including installed third-party libraries.

1.3 Supported platforms

The modernised **Tahoe** build targets Linux distributions with a C++17 compiler. The reference platform for development and continuous verification is:

- Ubuntu 22.04 (jammy) on x86_64, also tested under WSL2 (Windows Subsystem for Linux).
- GCC 11 or newer; Clang 13+ also works.

- CMake 3.16 or newer.
- Open MPI 4.x for parallel builds (optional — a serial build is the default).

All third-party C dependencies (**expat**, **SPOOLES**, **f2c**) are bundled under **third_party/** and built from source by the top-level CMake project; no system packages are required for those. The default linear solver, MUMPS on OpenBLAS, uses the system packages **libmumps-dev** and **libopenblas-dev**; without them the build falls back to the bundled SPOOLES. METIS and SuperLU support are optional. All of these are toggled through CMake cache variables.

2 Execution procedures

2.1 Building Tahoe with CMake

The modernised Tahoe build is driven by a single top-level CMake project. All bundled third-party C libraries (`expat`, `SPOOLES`, `f2c`) are compiled from source under `third_party/`; the legacy `tahoe-manager` Perl driver and the CVS/SourceForge workflow are no longer used.

The supported platforms are listed in Section 1.3. From the repository root a standard out-of-tree build is performed with:

```
$ cmake -S . -B build
$ cmake --build build --parallel $(nproc)
```

The serial executable is produced as `build/bin/tahoe`. Several auxiliary post-processing tools (`MakeCSE`, `translate`, `wrap`, `compare`, `extract_*`) are also built in the same tree under `build/bin/`.

Optional features are toggled through the usual CMake cache mechanism, for example:

```
$ cmake -S . -B build -DTAHOE_MPI=ON -DTAHOE_METIS=ON
```

A parallel build requires an MPI implementation (Open MPI 4.x is the reference); METIS and SuperLU support is optional.

The default linear solver (Table 5.2) needs the system MUMPS library and an optimized BLAS, on Debian and Ubuntu `sudo apt-get install libmumps-dev libopenblas-dev`. Both are optional: when one is missing, CMake prints a warning and the build falls back to SPOOLES, or to the BLAS MUMPS was built against. The BLAS is linked directly into `tahoe`, so MUMPS uses it regardless of the system's default `libblas.so.3`; another vendor can be chosen with `-DTAHOE_BLAS_VENDOR`. The configure step ends with a line naming the default solver and the BLAS it found.

2.2 Modes of execution

Tahoe runs in several modes that support execution in parallel. These modes are listed below. Only the Parallel Analysis mode actually runs in parallel; the other modes run in serial.

- (2.3) Serial analysis
- (2.4) Parallel analysis
- (2.5) Domain decomposition
- (2.6) Joining results data

The procedure for running in serial and parallel has been made as similar as possible. The only differences arise from machine-dependent limitations. The parameters file and geometry file that are needed to define an analysis are identical for both modes of analysis. Some additional files are needed to run parallel analysis, but **Tahoe** generates these for the user. The user must only be aware of which files are needed on the machine for each mode of operation at the time of execution. The following sections explain how to execute **Tahoe** with the following command line options:

```
tahoe
    -f [job name].xml
    -decomp
    -join
    -split_io
```

Throughout the rest of this section the parameters file is shown with the historical `.in` extension, but the file extension is not interpreted by **Tahoe**— current benchmarks and applications use `.xml` parameters files (see Section 4) and `.geom` geometry files in Tahoe ASCII format.

2.3 Serial analysis

For running serially, the user must have:

<code>[job name].in</code>	parameters file
<code>[geometry name].exo</code>	geometry file (ExodusII [39])

The file extensions are not interpreted by **Tahoe**, so these may be selected arbitrarily. The resulting output files will be:

<code>[job name].out</code>	“echo” file of parameters file
<code>[job name].io[j].exo</code>	results data (ExodusII [39])

The results data have labels j , where $j = 0, \dots, (\text{number of output groups} - 1)$. The number of output groups is determined by the type of analysis. For the simplest case, each element group will produce one output group, and the sequence of output groups will be the same as the sequence the element groups are declared in the parameters file. Each results file has only one element block corresponding to the connectivities for the element group. If multiple element blocks are used to define the connectivities for a given element group, these will be combined into a single element block for output. Serial analysis can also be done by running a parallel executable with the number of processors specified as one.

2.4 Parallel analysis

The procedure for launching parallel executables varies depending on the user environment. In the examples that follow, we assume parallel executables are launched using the `mpirun` command:

```
% mpirun -np [number of processors] tahoe
```

Users should substitute the appropriate command, such as `dmpirun` or `yod`, depending on the environment.

The procedure for running **Tahoe** in parallel varies depending on whether the machine has “limited” memory with respect to the problem size. A machine is considered to have “limited” memory with respect to a given problem if the entire model from the geometry definition cannot be stored in memory at the same time. On machines that are not memory limited, execution of **Tahoe** is *identical* whether running on serial or in parallel, aside from any special procedure needed to launch a parallel job. For this case, no additional steps are required for the user in order to pre- and post-process the model definition. With limited memory machines, a non-limited machine is needed in order to decompose the model geometry and join the results data after execution. Domain decomposition and result file joining is discussed in the next two sections.

The identical input files (parameters and geometry) are needed to run in parallel as are needed to run in serial.

```
[job name].in    parameters file
[geometry name].exo geometry file (ExodusII [39])
```

The first thing that the program does is look to see if the required decomposition files already exist. **Tahoe** creates a domain decomposition based on the connectivities seen at run time. If the geometry file contains blocks of elements that are not referenced in the parameters file, these blocks are not considered during decomposition. Additionally, meshfree methods generate connectivities at run time the depend on parameters defining the shape functions. **Tahoe** does not keep track of which values in the parameters file affect the domain decomposition. If any of these parameters are changed, the user must delete any of the corresponding decomposition files. The decomposition files are:

```
[geometry name].io.exo  global geometry file
[geometry name].n[M].io.map  output map
[geometry name].io.ID    block ID's per output group
[geometry name].n[M].part[i]  decomposition data file
[geometry name].n[M].p[i].exo  partial geometry file
```

In the file names above, M is the total number of partitions, while $i = 0, \dots, (M - 1)$. If any decomposition files are not found, **Tahoe** will attempt to create all of them. If **Tahoe** is launched on a non-limited memory machine, mesh decomposition is done serially on the processor with rank 0 before starting the analysis phase of execution in parallel. If the decomposition files already exist, the decomposition phase is not repeated. **Tahoe** produces the following files during execution:

```
[job name].p[i].out  “echo” file of parameters file
[job name].p[i].log  message passing log file
      console[i]    redirected console output
[job name].io[j].exo  results data (ExodusII [39])
```

where $j = 0, \dots, (\text{number of output groups} - 1)$. The output from the standard output streams is not redirected to a file for the processor with rank 0, that is, no `console0` file is created. Any prompts for user input still appear on the screen. When running a job in the background the user may opt to redirect the screen output to a file, called `console0` for consistency. The following input makes use of the `-f` command line option to specify the name of the parameters file:

```
% mpirun -np [number of processors] tahoe -f [job name].in > console0 &
```

On non-limited memory machines, the output from the distributed output groups is collected on processors as designated in the output map file. Since the output is “joined” at run time, no post-processing of the results data is required. Running **Tahoe** in this mode does require more memory. **Tahoe** will leave the output in its decomposed form if launched with the `-split_io` command line option:

```
% mpirun -np [number of processors] tahoe -split_io
```

When running on a limited memory machine, or when specifying the `-split_io` option, **Tahoe** will generate distributed results files:

```
[job name].p[i].io[j].exo distributed results file
```

where $i = 0, \dots, (\text{number of partitions} - 1)$ and $j = 0, \dots, (\text{number of output groups} - 1)$. The distributed results files must then be joined in a separate post-processing step to produce the results files that would have been written by a serial execution of the same calculation.

2.5 Domain decomposition

Serial mesh decomposition of a large mesh often requires more memory than some architectures have for each individual compute node. In these cases the user must perform the decomposition and results file joining on a pre/post processor machine that is not memory limited. If **Tahoe** is launched with the `-decomp` command line option, it will generate the decomposition files without subsequently performing any analysis:

```
% tahoe -decomp
```

As mentioned about, **Tahoe** creates a domain decomposition based on the connectivities seen *at run time*. This is a combination of information in the parameters and geometry files. If the geometry file contains blocks of elements that are not referenced in the parameters file, these blocks are not considered during decomposition. Also, **Tahoe** does not record the values in the parameters files used to generate a given decomposition. If the parameters file changes in any way that would affect the set of “active” elements, or the connectivities active at run time, the user must remove any existing decomposition files. Otherwise, **Tahoe** will use any existing decomposition files.

The decomposition process is not parallelized. In order to avoid allocating processors that will remain idle, **Tahoe** will not perform decomposition on a parallel platform if more

than one processors is allocated for execution. Therefore, decomposition must be initiated with:

```
% mpirun -np 1 tahoe -decomp
```

After prompting the user for the name of the job file, Tahoe will prompt the user for the number of partitions to create.

2.6 Joining results files

To join distributed output files, the parameters, geometry, decomposition files, and distributed results files are needed. As with the decomposition process, the joining process has not been parallelized. On a multi-processor platform, it must be launched with a single process:

```
% mpirun -np 1 tahoe -join
```

After prompting the user for the name of the job file, Tahoe will prompt the user for the number of partitions in the distributed results data. The number given to Tahoe must be the *total* number of partitions generated from the original model geometry. Under certain circumstances, not all partitions will contribute distributed output to all output groups.

3 Regression tests and benchmarks

The repository ships a layered benchmark suite under `benchmark_XML/` that doubles as a regression catalogue and a collection of fully-worked input examples. Each level has a `run.batch` that drives the bundled `tahoe` binary through every input file in that level, and a companion `compare` tool diffs the produced results against committed reference data. The complete entry point is `benchmark_XML/ReadMe`, and each level has its own `README.md` describing the cases in that tier.

3.1 Benchmark levels

Level 0 — Core capabilities. Smoke tests for the springs, solid mechanics, diffusion, particle, meshfree, and time-integrator paths. All code changes are expected to keep level.0 green before being merged.

Level 1 — Constitutive verification. Bulk material models, cohesive surface relations, and atomistic potentials exercised under simple loadings whose closed-form or high-quality reference answers are known.

Level 2 — Extended capabilities. Larger models and longer run times than level.1. Still serial and self-contained (no optional modules required): K -fields, contact, conveyor, thermostats, tied interfaces, torsion, controllers.

Level 3 — Parallel. MPI distributed-memory tests. Requires building Tahoe with `-DTAHOE_MPI=ON` and launching through `mpirun -np 4`.

Level 4 — Solver performance. Multi-physics performance cases used to compare sparse direct solvers (SPOOLES vs MUMPS vs SuperLU) on non-trivial DOF counts. The representative case is a 2D dielectric elastomer with a liquid inclusion (`dielectric_elastomer_Q1P0Elastocapillary`, ~18k DOF).

Level 5 — Explicit-solver and modern element benchmarks. Benchmarks added alongside the modernised explicit solver and the new finite-deformation elements. Includes the original legacy-vs-vectorized speed-up matrix (`explicit_benchmark/`), the classic Taylor bar (`taylor_bar/`), the Hertz contact case (`hertz/`) covering both explicit and implicit pathways and a mixed Tet4/Hex8 indenter variant, the implicit-friction case (`implicit_friction/`, see 11.3), the J_2 -plasticity Brinell indentation (`brinell/`, see 11.3), the uniaxial tension verification of cubic-spline isotropic hardening (`tensile/`, see 11.3), and the classic-Tahoe BonetTet implicit Tet4 path (`tet_classic/`, see 11.3).

3.2 Running a level

From the repository root:

```
$ cd benchmark_XML/level.0
$ printf "run.batch\nquit\n" | ../../build/bin/tahoe
$ printf "run.batch\nquit\n" | ../../build/bin/compare
```

Output files (`*.run`, `*.geo`, ...) are written alongside the input XML files and are excluded from version control via the project `.gitignore`. A driver script `run_benchmarks.sh` at the repository root sweeps any subset of levels — e.g. `./run_benchmarks.sh level.3` for the MPI tier alone; it reports passing, failing and crashing tests. The second command above compares the nodal output with the references to a relative tolerance of 10^{-8} and writes `compare.summary`. Unit tests of individual classes are built with Google Test and run with `ctest`. For level.3, the system `/usr/bin/mpirun` (Open MPI) must be used; a Conda-shipped `mpirun` is typically MPICH and ABI-incompatible with the OpenMPI that Tahoe links against.

3.3 Meshfree Kirchhoff–Love shell

The meshfree Kirchhoff–Love shell of Section 7.3 is tested by the input files of Table 3.1. The decks at level 2 are shortened, mass-scaled versions of the elasto-plastic examples of [56]; they test the plastic path of the element but are not validations. The full validation decks are in `docs/meshfree_kl_shell`.

Table 3.1: Regression tests for the meshfree Kirchhoff–Love shell.

test	description	checks
level.0/meshfree_kl_shell		
<code>scordelis_lo.xml</code>	Scordelis–Lo roof, 315 nodes	deflection 0.259 (reference 0.3006)
<code>hemisphere.xml</code>	pinched hemisphere	bending-dominated response
<code>pinched_cyl.xml</code>	linear pinched cylinder	membrane and bending
<code>collocation_bc.xml</code>	collocation of prescribed displacements	physical displacement reaches its target
<code>penalty_displacement_bc.xml</code>	penalty controller, explicit dynamics	physical displacement and reaction
<code>penalty_displacement_</code> <code>static.xml</code>	penalty controller, static	convergence in one Newton iteration
level.2/meshfree_kl_shell		
<code>necking.xml</code>	necking of a cylindrical shell, 504 nodes	saturation hardening and thickness update
<code>pinch_plastic.xml</code>	elasto-plastic pinched cylinder, 600 nodes	yield and plastic folding

3.4 Repeatability of tests across platforms

The bundled `compare` tool (built under `build/bin/compare`) performs the file-by-file diff against the reference results committed under each benchmark directory. Cross-platform discrepancies should be reported as issues on the project GitHub tracker; numerical differences within the `compare` tolerance are expected, while structural differences (missing fields, mismatched mesh ordering) indicate a regression.

3.5 Repeatability of tests during parallel execution

The level.3 tier is the authoritative parallel regression check. A test that passes serially but fails in parallel typically indicates a problem in the partitioning of state variables across the partition boundary, in the assembly of the global RHS, or in the solver-specific communication path.

4 Input file version 3.4.1

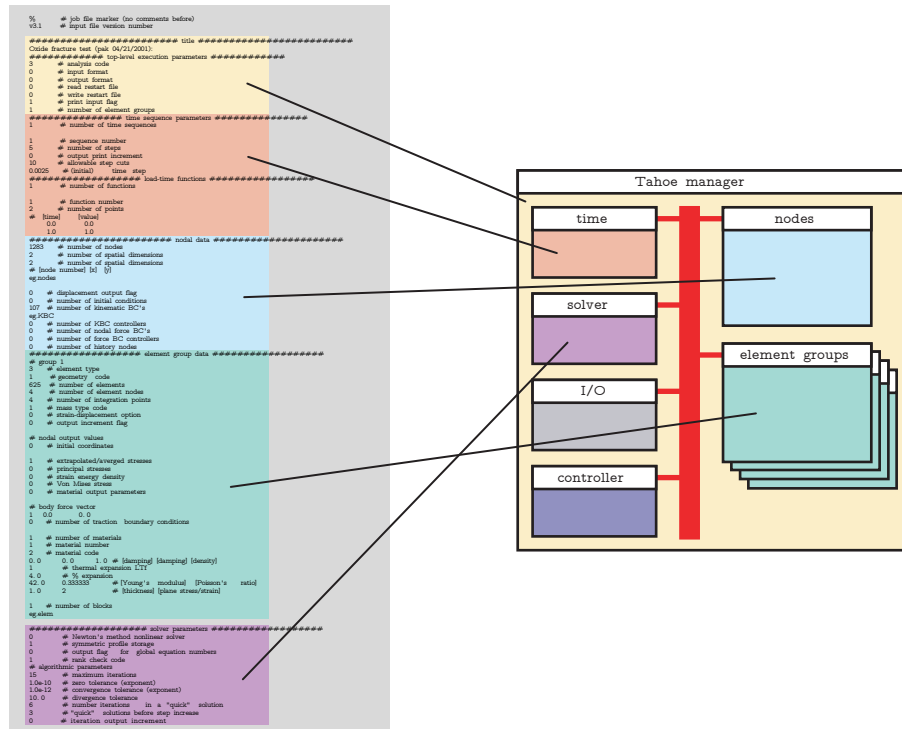


Figure 4.1: Layout of a Tahoe parameters file.

Simulation parameters for Tahoe are specified in white space delineated, plain text files. Input values are separated by white space, but the arrangement of white space is ignored. White space includes blank spaces, tabs, new line characters, and carriage returns. Tahoe uses calls to the platform-specific, standard C/C++ libraries to handle input from text files. Plain text files contain platform-specific line ending characters that may cause compatibility problems when files are transferred between platforms unless some file translation is applied. The # symbol denotes that the remainder of a given line of text should be ignored, or treated as a comment. Values are read from the parameters file sequentially. Therefore, all parameters must be specified, and there are no default values. The only exceptions to these conventions may come in specifications to the third-party libraries used by Tahoe for certain types of analysis.

As discussed in Section 1.2, Tahoe is internally divided into components. Data and parameters are owned by each component. Figure 4.1 shows how the parameters file may be divided into sections of parameters for the different components.

- (1) “Global” analysis parameters
- (2) Time parameters
- (3) Nodal parameters

(4) Element group(s) parameters

(5) Solver parameters

Details of the parameters in each block follow below.

4.1 Global analysis parameters

The format of the global analysis parameters is shown below:

```
%      # job file marker
3.4.1  # input file version marker
# title
[one line of at most 254 characters]
#### analysis parameters
[analysis code]
[input format]
[output format]
[read restart flag]
[write restart increment]
[echo input flag]
```

The first character in a parameters file must be `%`. The input file version is used by **Tahoe** to determine the compatibility of the input file with the version of the executable. **Tahoe** issues a warning if the input file version is older than the current. Developers are encouraged to make changes to the input file format backward compatible; however, backward compatibility cannot be maintained in all cases. Changes to the input file format since version 3.1 are logged in Section 11.2. Values of the *analysis code* are listed in Table 5.1. Table 10.1 lists the values for the *input format*. For input file formats using external geometry files, the code is followed by the relative path to the geometry database, as in:

```
# input format
5      # ExodusII format
[path to geometry database]
```

Table 10.2 lists the values for the *output format*. The *read restart flag* is boolean of either 0 or 1. A 1 indicates that **Tahoe** should initialize the system using information in a restart file. In this case, the flag is followed by the relative path to the restart file, as in:

```
# read restart flag
1      [path to restart file]
```

A *write restart increment* of $N > 0$ indicates that **Tahoe** should write a restart file every N steps. The *echo input flag* is a boolean flag of either 0 or 1. A value of 1 causes verbose echoing of input parameters and geometry information.

4.2 Time parameters

The time parameters are grouped into three sections:

```
#### time parameters
# list of time sequences
[n_seq = number of time sequences]
  1      [parameters for sequence 1]
  :
[n_seq]  [parameters for sequence n_seq]
# list of schedule functions
[n_scf = number of schedule functions]
  1      [parameters for schedule function 1]
  :
[n_scf]  [parameters for schedule function n_scf]
# time integration parameters
[parameter list]
```

The numbered sets of time sequence parameters and numbered schedule functions may appear in any order in their respective lists. The format of time sequence parameters is described in Section 4.2.1, while the format of schedule functions is described in Section 4.2.2. Most of the time integration schemes do not have any input parameters. Only the time integration scheme used for implicit elastodynamics (see Table 5.1) requires additional parameters. The HHT- α [23] integrator takes the single parameter $0 \leq \alpha \leq 1$, where $\alpha > 0$ introduces numerical dissipation. Dissipation is not supported for nonlinear analysis.

4.2.1 Time sequence parameters

A time sequence is specified by the following list of parameters:

```
# time sequence parameters
[number of steps]
[output print increment]
[maximum number of step cuts]
[initial step size]
```

If the *number of steps* is specified as zero, **Tahoe** will only complete the initialization phase of a simulation, reading all values from the parameters file. An *output print increment* specified as $N > 0$ causes results data to be written every N steps of the initial step size. An *output print increment* specified as $N < 0$ produces output every N steps of the current step size. Some analysis times support automatic time step control. The *maximum number of step*

cuts controls how many times the initial time step can be reduced by a factor of two before the simulation is aborted.

4.2.2 Schedule functions

Schedule functions are used to specify the time-varying behavior of boundary conditions or other analysis-specific parameters such as the thermal expansion in elastostatic and elastodynamic analysis. A function $f(t)$ is specified with a list of ordered pairs $\{t_i, f(t_i)\}$, $i = 1, \dots, n_{pts}$. The function is linearly interpolated between the data provided. The function is extended at zero slope beyond the range of time provided, that is

$$f(t) = \begin{cases} f(t_1) & \text{if } t < t_1, \\ f(t_i) + (t - t_i) \frac{f(t_{i+1}) - f(t_i)}{t_{i+1} - t_i} & \text{if } t_i < t < t_{i+1} \text{ for } i = 1, \dots, n_{pts} - 1, \\ f(t_{n_{pts}}) & \text{if } t > t_{n_{pts}}. \end{cases}$$

[To do: add figure] An example of the input format for schedule function is shown below:

$$\begin{array}{ll} [n_{pts} = \text{number of points on the schedule}] & \\ [t_1] & [f(t_1)] \\ \vdots & \\ [t_{n_{pts}}] & [f(t_{n_{pts}})] \end{array}$$

The ordered pairs on the schedule function must be specified with time in sequentially increasing order.

4.3 Nodal parameters

The nodal parameters of the input file contains the following sections:

```
#### nodal data
[coordinate data]
[list of initial conditions]
[list of kinematic boundary conditions]
[list of kinematic boundary condition controllers]
[list of force boundary conditions]
[list of force boundary condition controllers]
[list of history node specifications]
```

The *coordinate data* only appears in the nodal section if geometry format TahoeI is specified in the global parameters section, described in Section 4.1. For all other geometry file formats listed in Table 10.1, the nodal coordinate data resides in an external file, and this section

of the nodal parameters is omitted. The format for all of the lists specified in this section is similar. The list is initiated with its length and followed by the entries in the list if the length is nonzero. The *list of initial conditions* is either empty

0 # initial conditions

or contains a list specified as

```
# initial conditions
[nic = number of initial conditions]
[parameters for initial condition 1]
:
[parameters for initial condition nic]
```

The format of initial condition specifications is described in Section 6.1. The *list of kinematic boundary conditions*, or essential boundary conditions, is either empty

0 # kinematic boundary conditions

or contains a list specified as

```
# kinematic boundary conditions
[nkbc = number of kinematic boundary conditions]
[parameters for kinematic boundary condition 1]
:
[parameters for kinematic boundary condition nkbc]
```

The format of kinematic boundary condition specifications is described in Section 6.2. Kinematic boundary conditions allow specification of the primal variables along the cartesian axes as a function of time. Kinematic boundary condition controllers allow specification the field at groups of nodes to follow specific functional forms, such as the displacements surrounding a crack tip based on the asymptotic K -field solution from isotropic linear elasticity. The *list of kinematic boundary condition controllers* is either empty

0 # kinematic boundary condition controllers

or contains a list specified as

```
# kinematic boundary condition controllers
[nkctl = number of kinematic boundary conditions]
[parameters for kinematic boundary condition controller 1]
:
[parameters for kinematic boundary condition controller nkctl]
```

The format of kinematic boundary condition controllers specifications is described in Section 6.3. The *list of force boundary conditions* is either empty

```
0 # nodal force boundary conditions
```

or contains a list specified as

```
# nodal force boundary conditions
[nfbc = number of nodal force boundary conditions]
[parameters for force boundary condition 1]
⋮
[parameters for force boundary condition nfbc]
```

The format of nodal force boundary condition specifications is described in Section 6.4. The *list of force boundary condition controllers* is either empty

```
0 # force boundary condition controllers
```

or contains a list specified as

```
# force boundary condition controllers
[nctl = number of force boundary condition controllers]
[parameters for force controller 1]
⋮
[parameters for force controller nctl]
```

The format of nodal force boundary condition controllers specifications is described in Section 6.5. Force boundary condition controllers implement a number of “rigid” objects that generate forces on nodes through contact. For example, the rigid sphere controllers can be used to represent an indenter with a motion specified as a function of time.

Nodes, or sets of nodes, can be specified to produce more detailed output. The format of history node output is described in Section 10.2.7. The *list of history node specifications* is either empty

```
0 # number of history nodes
```

or contains a list specified as

```
# history nodes
[nhst = number of history node ID's]
[node ID1] ... [node IDnhst]
```

The *node ID* differs depending on geometry file format (see Table 10.1). For TahoeI format, the *node ID* is simply the node number. Some of the geometry database formats supported by Tahoe allow nodes to be grouped into sets identified by an *ID*. With these database formats, the *node ID* corresponds to the node set *ID*.

4.4 Element parameters

4.5 Solver parameters

4.6 Batch file format

5 Analysis types

Table 5.1 shows a list of analysis available in Tahoe.

Table 5.1: This is the list of types.

analysis type	linear	nonlinear
elastostatic	1	3
implicit elastodynamic	2	4
explicit elastodynamic	6	7
steady-state diffusion	19	N/A
transient diffusion	20	N/A

5.1 Analysis types

Linear analysis implies that the governing equations are linear in the unknown variables over a given time increment. Nonlinear analysis implies that some iterative procedure is required to determine the solution in the unknown variables. Most cases require the nonlinear solution procedures. Specifying a nonlinear analysis type for a linear problem will produce a solution after a single iteration of the solution procedure.

Whether linear or nonlinear, the solution procedure ultimately results in a linear system of equations. A number of different storage schemes and solvers are available for the global matrix associated with this linear system. They are listed in Table 5.2 and selected with one element inside the solver block of the input file, for example `<nonlinear_solver ...><SPOOLES_matrix/></nonlinear_solver>`. Which of them are available depends on the build options (Section 2.1).

When the solver block names no matrix, Tahoe uses the default linear solver: the multifrontal direct solver **MUMPS** [4] with the PORD fill-reducing ordering [40], running on the optimized BLAS linked at build time (**OpenBLAS** [58] by default). Under **mpirun** the distributed variant is used. If Tahoe was built without **MUMPS**, the default is **SPOOLES**. Explicit dynamic analyses, whose system matrix is diagonal, always use the diagonal matrix, whatever the input names. The choice, the ordering and the BLAS in use are written to the output file. On a 29,022-equation elasto-plastic contact problem the default solver factorizes about 15 times faster than **SPOOLES** with identical Newton iterations.

5.1.1 Elastostatic analysis

This analysis can be used with any of the sparse or dense matrix types.

Table 5.2: Matrix types for the global system of equations.

input element	build option	notes
MUMPS_matrix	TAHOE_MUMPS (on)	default; serial MUMPS [4]. Parameters <code>message_level</code> , <code>always_symmetric</code> , <code>ordering</code> (PORD default, AMD, AMF, QAMD, METIS, SCOTCH, automatic)
MUMPS_MPI_matrix	TAHOE_MUMPS, TAHOE_MPI	distributed MUMPS; same parameters
SPOOLES_matrix	TAHOE_SPOOLES (on)	bundled SPOOLES [7]; serial or, under <code>mpirun</code> , distributed
SPOOLES_MT_matrix	TAHOE_SPOOLES_ MT	multithreaded SPOOLES
SuperLU_matrix	TAHOE_SUPERLU	bundled SuperLU 3.0
profile_matrix	always	skyline storage, serial
full_matrix	always	dense storage, small problems and tangent checks
diagonal_matrix	always	explicit dynamics (lumped mass)

5.1.2 Implicit elastodynamic analysis

This analysis uses HHT- α time integration developed by Hilber *et al.* [23] and can be used with any of the sparse or dense matrix types.

5.1.3 Explicit elastodynamic analysis

This analysis uses the explicit, central-difference time integration scheme from the classic Newmark family of methods. In order to make this method “matrix-free”, the diagonal matrix storage should be used.

5.1.4 Steady-state and transient diffusion

Transient head conduction uses a trapezoidal time integration scheme, and can be used with any of the sparse or dense matrix types.

5.2 Solution techniques

5.2.1 Linear

The linear solution technique is used with all of the linear analyses and with explicit dynamic analysis. Explicit dynamic analysis may involve large, nonlinear kinematics of deformation,

nonlinear constitutive response, and contact; however, as a result of the time discretization, the incremental accelerations are linearly related to the driving forces over a single time increment.

5.2.2 Nonlinear

Nonlinear solution techniques are used with all nonlinear analyses except with nonlinear explicit dynamics analysis. Table 5.3 shows a list of nonlinear solution techniques supported. As mentioned above, the nonlinear solution procedures iteratively search for the solution until some convergence tolerance is satisfied. In general, a solution procedure involves determining two aspects of the search for the solution. The procedure must determine the “length” and “direction” of the update to the current best guess of the solution. The methods listed in Table 5.3 differ in how they determine these two parameters. The various nonlinear solution methods use two criteria for assessing convergence based on the norm of the residual forces. The first is an “absolute” tolerance while the second is a “relative” tolerance. The “absolute” tolerance is an indication of “small” in terms of the numerical precision displayed for a given calculation. Depending on the platform, the various formulations used throughout a calculation, and details of the implementation, the level of numerical “noise” in a calculation may limit how close to absolute zero the solution procedure is able to drive the norm of the residual. The second tolerance examines the “relative” error. This compares the norm of the residual at the end of an iteration with the norm of the residual at the first iteration. If one examines the console output during a simulation, the absolute error is written to the output after incremental boundary conditions are applied followed by the relative error after each iteration. The solution procedure stops if either condition is satisfied.

Table 5.3: Nonlinear solution techniques.

solution method	code
standard Newton	0
initial tangent	1
modified Newton	2
dynamic relaxation [53]	3
Newton with line search	4
preconditioned nonlinear conjugate gradient	5
interactive Newton	6

All of the nonlinear solution techniques automatically adjust the time, or load, increment depending on the behavior of the iterative solution procedure. The solvers have user-adjustable parameters for determining whether the increment should increase or decrease. A cut in the increment is triggered if the number of iterations exceeds to the user-defined value or if the “relative” error exceeds a user-defined value. Additionally, the increment is

decreased if “overload” conditions are detected in calculations anywhere in the code. Once the increment has been cut, **Tahoe** begins to monitor two parameters in combination to determine if the increment should be increased. The user can prescribe the number of iterations in a “quick” solution and the number of sequential “quick” solutions that must pass before the increment is increased. An increment adjustment changes the step size by a factor of two.

5.2.2.1 Standard Newton Newton’s method is a second order optimization technique that makes use of a tangent matrix to determine the update vector. This solver reforms a new tangent matrix at every iteration, and does not rescale the update vector. The convergence behavior of Newton methods is known to be poor “far” from the solution, but the method does display locally quadratic convergence “near” the solution.

5.2.2.2 Initial tangent This solver differs from the standard Newton in that a new tangent matrix is calculated only once per time, or load, increment. The tangent matrix is formed for the initial iteration and is subsequently reused. This approach results in slower rates of convergence, but may result in faster solution times since the number of times the tangent matrix is formed and factorized is reduced. This method assumes that the matrix type selected allows the factorized matrix to be reused at. This method may also provide better performance in cases in which the tangent matrix becomes rank deficient.

5.2.2.3 Modified Newton This solver attempts to determine conditions under which to reform or reuse the tangent matrix based on a series of rules. This solver may be considered experimental.

5.2.2.4 Dynamic relaxation Dynamic relaxation [53] is a first order optimization technique that wraps the problem of finding the solution with a set of damped, second order differential equations. The solution is determined by dissipating the transient behavior resulting from residual forces.

5.2.2.5 Newton with line search This solver augments the standard Newton solver with a line search to determine the length of the update vector. User-specified parameters control the effort expended for the line search. The goal of the line search is to determine the optimal distance along the update direction. User-defined parameters control the maximum number of iterations expended to determine this optimal distance. The optimal distance is defined by the point along the update vector at which the residual vector is orthogonal to the update vector. An *exact* line search finds this point exactly. A second input parameter provides an approximation to the exact search.

5.2.2.6 Preconditioned nonlinear conjugate gradient This solver incorporates the preconditioned, conjugate gradient first order optimization method. User-specified param-

eters control the number of conjugate gradient iterations between “restarts”, as well as specifications for the line search.

5.2.2.7 Interactive Newton This solver allows the user to interactively control the solution process. The underlying solver is a Newton solver with line search. The user can step forward or backward in time, modify solver parameters, and write output data. Additionally, the value of other parameters in the simulation may be viewed and changed by addressing specific *scopes* in **Tahoe**. The console commands also change with scope.

6 Initial conditions and boundary conditions

Initial conditions and boundary conditions may be specified on selected nodes. Additionally, the variational form of governing equations leads to natural boundary conditions that may be specified on selected parts of the domain boundary. **Tahoe** supports a number of geometry database types. These are listed in Table 10.1. The database format determines how nodes or sets of nodes are identified and how parts of the domain boundary are identified. Table 6.1

Table 6.1: Identification of nodes and boundaries.

database	node ID	side ID
TahoeI	node	element, facet
TahoeII	node set ID	side set ID
ExodusII [39]	node set ID	side set ID

lists how nodes and element sides are identified for the different database types. Their use will be explained with examples below. For the TahoeI format, boundary segments are explicitly specified with an element and facet number. Canonical facet numbering for the different integration domain geometries supported by **Tahoe** are listed in Section 7.1. For TahoeII and ExodusII databases, groups of nodes may be collected into node sets and groups of element facets are collected into side sets. Both types of sets are identified with ID numbers.

Many of the boundary conditions may be specified as functions of time. Their values have the form

$$g(t) = g f^{(i)}(t), \quad (6.1)$$

where g is a scalar coefficient and $f^{(i)}(t)$ is a schedule function. Schedule functions are explained in Section 4.2.2. Time varying functions in the input file are specified by providing g and i , as defined in (6.1).

6.1 Nodal initial conditions

Each initial condition is specified as

$$[node\ ID] \quad [degree\ of\ freedom] \quad [order] \quad [value]$$

The definition of the *node ID* depends on the geometry database type as listed in Table 6.1. The *order* refers to the time derivative of the field variable to which the initial conditions applies. A *node ID* of -1 applies the boundary condition to *all* nodes.

6.2 Nodal kinematic boundary conditions

Each nodal kinematic boundary condition is specified as

$$[node\ ID] \quad [degree\ of\ freedom] \quad [code] \quad [schedule] \quad [value]$$

6.3 Kinematic boundary condition controllers

The definition of the *node ID* depends on the geometry database type as listed in Table 6.1. The *code* specifies if the condition is applied to the nodal unknown u , or one of its time derivatives $\dot{u}$, $\ddot{u}$, or higher. The codes are listed in Table 6.2. The *value* and *schedule* in

Table 6.2: Codes for nodal kinematic boundary conditions.

code	type
0	<i>fixed</i>
1	u
2	$\dot{u}$
3	$\ddot{u}$

the nodal kinematic boundary condition specification refer to g and i in (6.1), respectively, for the variation of the boundary condition in time. For fields approximated by meshfree shape functions, which lack the Kronecker delta property, a kinematic boundary condition prescribes the nodal coefficient rather than the physical value. Adding `collocation="1"` to a `kinematic_BC` entry prescribes the physical value instead, see Section 6.6.

6.3 Kinematic boundary condition controllers

Table 6.3: Kinematic boundary condition controllers.

code	XML name	description
0	K-field	K-field
1	bi-material K-field	bimaterial K-field
2	mapped_nodes	Mapped displacements
3	tied_nodes	Tied displacements
5	periodic_nodes	Periodic displacements
7	scaled_velocity	Scaled velocity
9	torsion	Torsion
10	conveyor	Conveyor (moving periodic domain)
11	symmetric_conveyor	Symmetric conveyor
12	collocation_KBC	Direct nodal collocation of essential boundary conditions for meshfree fields, Section 6.6

6.3.1 K-field

6.3.2 Interface K-field

6.3.3 Mapped displacements

6.3.4 Tied displacements

6.3.5 Periodic displacements

6.3.6 Scaled velocity

6.3.7 Torsion

6.3.8 Conveyor

6.3.9 Direct nodal collocation

See Section 6.6.

6.4 Applied nodal forces

Each nodal force boundary condition is specified as

$$[node\ ID] \quad [degree\ of\ freedom] \quad [schedule] \quad [value]$$

The definition of the *node ID* depends on the geometry database type as listed in Table 6.1. The *value* and *schedule* in the nodal kinematic boundary condition specification refer to g and i in (6.1), respectively, for the variation of the boundary condition in time.

6.5 Force controllers

[To do: This boundary condition type actually behaves more like “elements” because they do produce a contribution to the stiffness matrix, i.e., they act as “follower forces” and are not simply defined as functions of time.]

Table 6.4: Force boundary condition controllers.

code	XML name	description
0	wall_penalty	Rigid barrier (penalty)
1	sphere_penalty	Rigid sphere (penalty)
2	sphere_augmented_Lagrangian	Rigid sphere (augmented Lagrangian)
3	sphere_penalty_meshfree	Rigid sphere for meshfree domains (penalty)
4	wall_augmented_Lagrangian	Rigid barrier (augmented Lagrangian)
5	cylinder_penalty	Rigid cylinder (penalty)
6	augmented_Lagrangian_KBC_meshfree	Kinematic constraints on meshfree nodes (augmented Lagrangian)
7	cylinder_augmented_Lagrangian	Rigid cylinder (augmented Lagrangian)
8	field_augmented_Lagrangian_KBC_meshfree	Field-driven kinematic constraints on meshfree nodes (augmented Lagrangian)
9	pressure_bc	Pressure boundary condition
10	angled_bc	Kinematic condition normal to an inclined surface (penalty)
11	penalty_displacement_meshfree	Prescribed physical displacement on meshfree fields (penalty), Section 6.6

6.5.1 Rigid barrier (penalty method)

6.5.2 Rigid sphere (penalty method)

6.5.3 Rigid sphere (augmented Lagrangian method)

6.5.4 Rigid sphere for meshfree domains (penalty method)

6.5.5 Rigid barrier (augmented Lagrangian method)

6.5.6 Rigid cylinder (penalty and augmented Lagrangian methods)

6.5.7 Kinematic constraints on meshfree nodes (augmented Lagrangian method)

6.5.8 Pressure boundary condition

6.5.9 Inclined kinematic condition (penalty method)

6.5.10 Prescribed physical displacement on meshfree fields (penalty method)

See Section 6.6.

6.6 Essential boundary conditions for meshfree fields

Meshfree shape functions do not satisfy $\Phi_J(\mathbf{x}_A) = \delta_{JA}$ (Section 7.2.1), so the physical value at a node,

$$u^h(\mathbf{x}_A) = \sum_J \Phi_J(\mathbf{x}_A) d_J, \quad (6.2)$$

differs from its nodal coefficient d_A , and an ordinary kinematic boundary condition, which prescribes d_A , does not impose the intended displacement. Two treatments are available. Both obtain the values $\Phi_J(\mathbf{x}_A)$ from the element group through a common interface, which is currently implemented by the meshfree Kirchhoff–Love shell (Section 7.3).

6.6.1 Direct nodal collocation

The kinematic boundary condition controller `collocation_KBC`, or equivalently the flag `collocation="1"` on a `kinematic_BC` entry, solves at every step for the coefficients of the constrained nodes that make (6.2) equal to the prescribed value,

$$\Phi_c \mathbf{d}_c = \bar{\mathbf{u}} - \Phi_f \mathbf{d}_f, \quad (6.3)$$

where the subscripts c and f denote constrained and free coefficients, and imposes them as ordinary prescribed values. For example,

```
<kinematic_BC dof="1" node_ID="3" schedule="1" type="u"
value="-1.0" collocation="1"/>
```

Collocation is exact for the prescribed value but is not a variational enforcement: the free coefficients in the support of a constrained node receive no constraint force, and the reaction reported for the constrained coefficient is not the physical load.

6.6.2 Penalty enforcement of physical displacements

The force controller `penalty_displacement_meshfree` enforces the constraint $g_A = u^h(\mathbf{x}_A) - \bar{u}_A(t) = 0$ for one degree of freedom at every node A of the listed node sets by a penalty force distributed over the support,

$$f_J = \Phi_J(\mathbf{x}_A) \lambda_A, \quad \lambda_A = -k g_A - c_A \dot{g}_A, \quad c_A = 2 \zeta \sqrt{k m_A^{\text{eff}}}, \quad \frac{1}{m_A^{\text{eff}}} = \sum_J \frac{\Phi_J(\mathbf{x}_A)^2}{m_J}, \quad (6.4)$$

where k is the penalty stiffness, ζ the damping ratio of an optional dashpot that suppresses oscillation of the penalty spring in explicit dynamics, and m_J the lumped nodal masses. The multiplier λ_A is the physical reaction at node A , work conjugate to $u^h(\mathbf{x}_A)$. The controller contributes the stiffness $k \Phi_J \Phi_K$, so it can also be used in implicit analyses. The parameters are `dof`, `schedule`, `value` (the prescribed physical value, scaled by the schedule), `penalty`, `damping_ratio` (default 0) and `element_group` (default 0, which selects the first group that

provides the shape function values). For example,

```
<penalty_displacement_meshfree dof="1" schedule="1"
  value="-300.0" penalty="1.0e5" damping_ratio="1.0">
  <node_ID_list><String value="3"/></node_ID_list>
</penalty_displacement_meshfree>
```

At each output step the controller writes one line per constrained node to the output file with the physical displacement, the reaction λ_A and their means since the previous output. The penalty stiffness should be large relative to the structural stiffness at the node but, in explicit dynamics, small enough that $2\sqrt{k/m_A^{\text{eff}}}$ does not limit the stable time step.

6.7 Body forces

Certain element formulations include body forces. Applied body forces are specified with the vector analog of (6.1). Their values have the form

$$\mathbf{b}(t) = \mathbf{b} f^{(i)}(t), \quad (6.5)$$

where $\mathbf{b}$ is a vector coefficient of length n_{dof} , the number of degrees of freedom per node, and $f^{(i)}(t)$ is a scalar schedule function. Schedule functions are explained in Section 4.2.2. Body forces are specified as

$$[schedule] \quad [b_1] \quad \dots \quad [b_{n_{dof}}]$$

6.8 Natural boundary conditions

Recasting balance laws in weak form leads to natural boundary conditions involving flux terms across domain surfaces. These fluxes are specified as part of the input of element parameters. Fluxes are associated with the faces of elements. As shown in Table 6.1, the description of element faces differs depending on the geometry database format. Each natural boundary condition requires specification of the flux vector at every node on the selected element face, or set of faces. Canonical number of element faces and node numbering over each face are described in Section 7.1. Natural boundary conditions are specified as

$$\begin{array}{ccc} [side\ ID] & [schedule] & [coordinate\ system] \\ [t_{11}] & \dots & [t_{1n_{dof}}] \\ \vdots & & \\ [t_{n_{fn}1}] & \dots & [t_{n_{fn}n_{dof}}] \end{array}$$

Each flux vector is length n_{dof} , and a vector must be specified for all n_{fn} nodes on the element face. The *coordinate system* is identified by the values listed in Table 6.5. The facet local coordinate frame is constructed such that the last component of the flux vector $t_{n_{dof}}$ corresponds to flux in the outward normal direction. The tangential components of the flux

Table 6.5: Coordinate systems for natural boundary conditions.

code	description
0	Cartesian
1	facet local

vector $\mathbf{a}$ constructed from the mapping of the face to its corresponding parent integration domain.

An example specification of two-dimensional, traction boundary conditions appears below:

```
1 # number of traction boundary conditions
####
1 # side ID
1 # schedule number
0 # coordinate system
# traction vectors
0.0 1.0
0.0 1.0
```

The example shows a single specification. As shown in Table 6.1, the format of the *side ID* specification depends on the geometry database format. In the example, a single integer is used to define the element edges where the boundary conditions are applied. This format would be appropriate for geometry database types that support definition of side sets. Two traction vectors, each with two components, are given indicating that the element edges specified by the *side ID* have two nodes per edge. The traction vectors are specified with respect to the global Cartesian coordinate frame, meaning the values for each vector correspond to $\{t_x, t_y\}$. A traction vector must be provided for each node of the element face. The number of nodes on each face for a given element geometry and the canonical order of the nodes on each face is described in Section 7.1. The traction is interpolated over the face from the nodal values using the standard finite element shape functions.



Figure 7.1: Node, face, and node-on-face numbering for line integration domains.

7 Element types

Within the structure of **Tahoe**, any formulation that relates nodal unknowns to the corresponding conjugate forces falls into the category of *elements*. A list of the element codes are shown in Table 7.1. The category of continuum elements is broken into three parts: Solid, Diffusion, and Mesh Free. Although mesh free methods do not use elements, they do need interpolation schemes, these schemes are discussed in this chapter.

7.1 Integration domains

Tahoe supports a number of Gaussian integration rules for the finite element types listed in Table 7.1. These rules are defined for the geometries listed in Table 7.2. Additionally, the meshfree formulations make use of a background grids composed of “integration cells” with the same geometries and integration rules. Each of these geometries has a corresponding local node numbering. The local node numbering, as well as the local ordering of the facets is shown in Figures 7.1 through 7.6. Although the pattern of local node numbering is not derived from a set of rules, it does display several common characteristics for all domain geometries. The vertex nodes always have lower local numbers that the mid-side nodes. For the three-dimensional domains, the local numbering of the facet nodes is always defined in a sense, or line direction, consistent with the outward normal to the face. Finally, the local node numbering for the follows the convention used by **ABAQUS** [22] and **EnSight** [15], which is not the same as the convention used by the **ExodusII** [39] database format for some domain geometries. However, the numbering of nodes on facets differs from the convention used by **ABAQUS** in that element normals point outward. When reading from or writing to an **ExodusII** database, **Tahoe** adjusts for any differences in the numbering convention.

7.2 Meshfree methods

Recently, there has been intense interest in the development of meshfree numerical simulation methods. The methods used in **Tahoe** have their origins in Smoothed Particle Hydrodynamics (SPH) proposed by Gringold and Monaghan[20]. A number of methods have

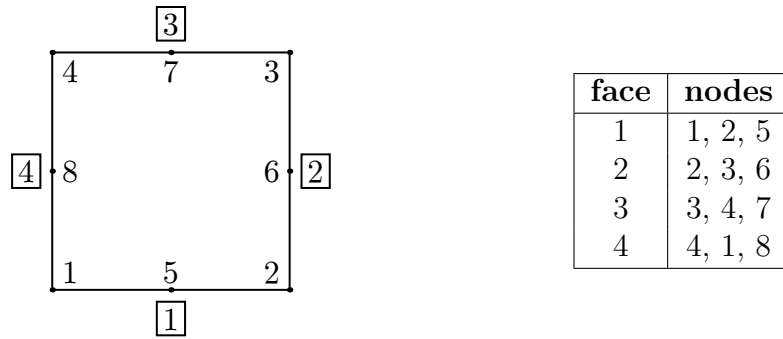


Figure 7.2: Node, face, and node-on-face numbering for quadrilateral integration domains.

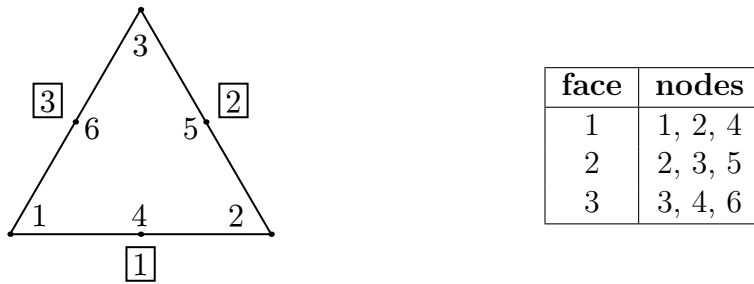


Figure 7.3: Node, face, and node-on-face numbering for triangular integration domains.

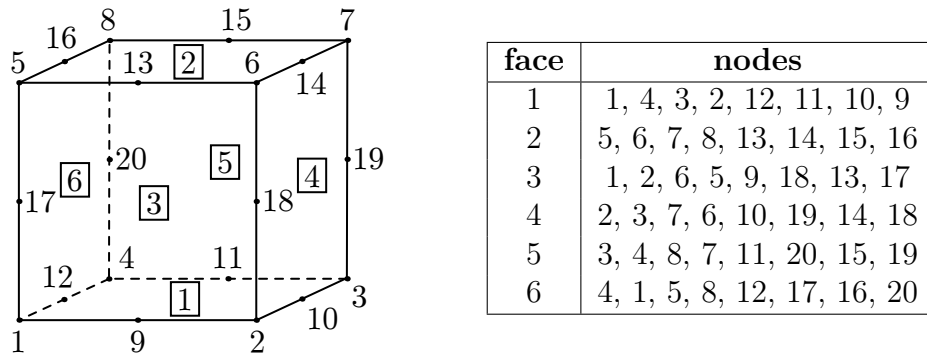


Figure 7.4: Node, face, and node-on-face numbering for hexahedral integration domains.

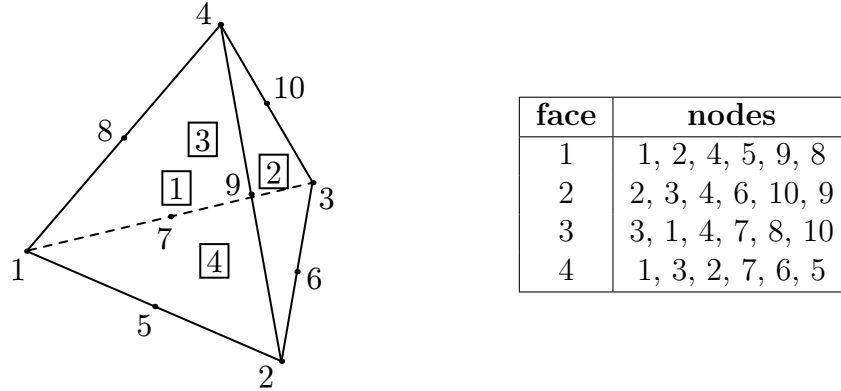


Figure 7.5: Node, face, and node-on-face numbering for tetrahedral integration domains.

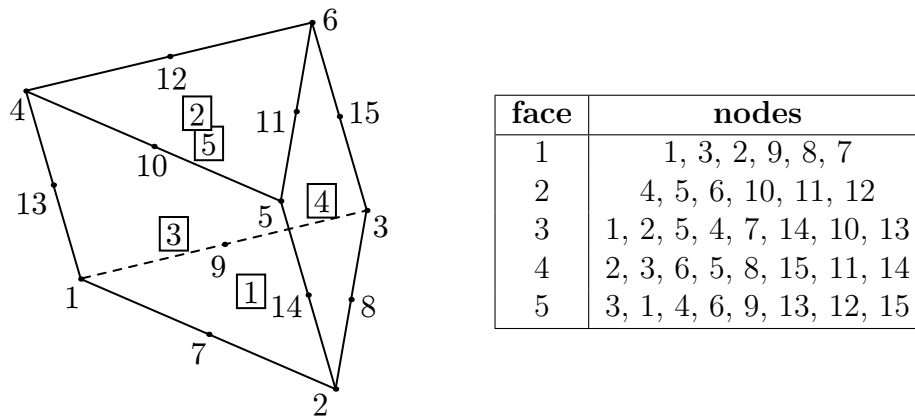


Figure 7.6: Node, face, and node-on-face numbering for pentahedral integration domains.

developed that employ variants of the Moving Least Squares fitting procedure of Lancaster and Salkauskas[28]. These include, but are not limited to, the Diffuse Element Method of Nayroles *et al.* [35], the Element Free Galerkin method of Belytschko *et al.* [11], the Reproducing Kernel Particle Method of Liu and Chen[29], and the Meshless Local Petrov-Galerkin method of Atluri and Zhu[8]. A number of overviews and reviews have appeared regarding these methods[10, 31, 30, 32, 33], though the interested reader will find no shortage of new developments.

7.2.1 Reproducing Kernel Particle Method (RKPM)

RKPM belongs to class of methods for which the approximation, or “image”, of a signal is given by a kernel expression. Without loss of generality, we can consider the expression for the approximation in one dimension

$$u^{R_a}(x) = \int_{-\infty}^{+\infty} \phi_a(x-y) u(y) dy, \quad (7.1)$$

where ϕ_a is alternately called a weight, kernel, or smoothing function. From the analogy to signal processing, ϕ_a may be viewed as a customizable low pass filter between the original signal, or data, $u(y)$ and its reproduced image. This function is positive, even, and has compact support characterized by the dilation parameter a . Liu and Chen[29] improved the accuracy of the method by modifying the window function with a correction to yield a reproducing condition as

$$u^{R_a}(x) = \int_{-\infty}^{+\infty} \bar{\phi}_a(x-y) u(y) dy, \quad (7.2)$$

where the modified window function

$$\bar{\phi}_a(x-y) = C(x; x-y) \phi_a(x-y) \quad (7.3)$$

incorporates the polynomial

$$C(x; x-y) = b_0(x) + b_1(x) (x-y) + b_2(x) (x-y)^2 + \dots + b_m(x) (x-y)^m \quad (7.4)$$

which ensures that the approximation can exactly represent polynomials of order m . In general, we may express the correction function as

$$C(\mathbf{x}; \mathbf{x}-\mathbf{y}) = \mathbf{b}(\mathbf{x}) \cdot \mathbf{P}(\mathbf{x}-\mathbf{y}) \quad (7.5)$$

where $\mathbf{P}$ is a basis of polynomials that possess the desired degree of completeness and $\mathbf{b}(\mathbf{x})$ is an vector of unknown coefficients determined from the reproducing condition (7.2). The requirement that each member of the basis be reproduced follows from (7.2) as

$$\mathbf{P}(\mathbf{0}) = \int_{-\infty}^{+\infty} \mathbf{b}(\mathbf{x}) \cdot \mathbf{P}(\mathbf{x}-\mathbf{y}) \phi_a(\mathbf{x}-\mathbf{y}) \mathbf{P}(\mathbf{x}-\mathbf{y}) dy. \quad (7.6)$$

The vector of coefficients follows as

$$\mathbf{b}(\mathbf{x}) = \mathbf{M}_a^{-1}(\mathbf{x}) \mathbf{P}(\mathbf{0}), \quad (7.7)$$

where

$$\mathbf{M}_a(\mathbf{x}) = \int_{-\infty}^{+\infty} \mathbf{P}(\mathbf{x} - \mathbf{y}) \otimes \mathbf{P}(\mathbf{x} - \mathbf{y}) \phi_a(\mathbf{x} - \mathbf{y}) d\mathbf{y} \quad (7.8)$$

is known as the moment matrix. Using the result from (7.7), the reproducing condition (7.2) can be written as

$$\mathbf{u}^{R_a}(\mathbf{x}) = \int_{-\infty}^{+\infty} \mathbf{b}(\mathbf{x}) \cdot \mathbf{P}(\mathbf{x} - \mathbf{y}) \phi_a(\mathbf{x} - \mathbf{y}) \mathbf{u}(\mathbf{y}) d\mathbf{y}. \quad (7.9)$$

First and higher order gradients of the field representation follow from (7.7)–(7.9) through simple differentiation. For clarity, we present the expressions for derivatives. We note that these derivatives may be evaluated exactly though some early work in the meshfree field advocated approximate expressions for these derivatives[35]. The gradient of the field may be expressed as

$$\begin{aligned} \frac{\partial \mathbf{u}^{R_a}(\mathbf{x})}{\partial \mathbf{x}} = \int_{-\infty}^{+\infty} \mathbf{u}(\mathbf{y}) \otimes \left[\frac{\partial \mathbf{b}(\mathbf{x})}{\partial \mathbf{x}} \mathbf{P}(\mathbf{x} - \mathbf{y}) \phi_a(\mathbf{x} - \mathbf{y}) + \right. \\ \left. \mathbf{b}(\mathbf{x}) \frac{\partial \mathbf{P}(\mathbf{x} - \mathbf{y})}{\partial \mathbf{x}} \phi_a(\mathbf{x} - \mathbf{y}) + \right. \\ \left. \mathbf{b}(\mathbf{x}) \cdot \mathbf{P}(\mathbf{x} - \mathbf{y}) \frac{\partial \phi_a(\mathbf{x} - \mathbf{y})}{\partial \mathbf{x}} \right] d\mathbf{y}, \end{aligned} \quad (7.10)$$

where the gradient of the coefficient vector

$$\frac{\partial \mathbf{b}(\mathbf{x})}{\partial \mathbf{x}} = -\mathbf{M}_a^{-1}(\mathbf{x}) \frac{\partial \mathbf{M}_a(\mathbf{x})}{\partial \mathbf{x}} \mathbf{b}(\mathbf{x}) \quad (7.11)$$

follows from (7.7) making use of the relation

$$\frac{\partial [\mathbf{M}_a^{-1}]_{ij}}{\partial x_k} = -[\mathbf{M}_a^{-1}]_{ir} \frac{\partial [\mathbf{M}_a]_{rs}}{\partial x_k} [\mathbf{M}_a^{-1}]_{sj}. \quad (7.12)$$

Higher-order gradients may be calculated by further differentiation of (7.10)–(7.12). Notably, the expressions for the representation of the unknown field variables are general for an arbitrary number of field and spatial dimensions, including the expressions for the first and higher order gradients of the field.

In evaluating the representations for the field numerically, we discretize the integrals in the previous expressions. The discrete reproducing condition follows from (7.9) as

$$\mathbf{u}^{R_h}_a(\mathbf{x}) = \sum_{I=1}^{N_p} \mathbf{b}(\mathbf{x}) \cdot \mathbf{P}(\mathbf{x} - \mathbf{x}_I) \phi_a(\mathbf{x} - \mathbf{x}_I) \mathbf{u}_I \Delta V_I, \quad (7.13)$$

where N_p is the number of sampling points, or particles, under consideration, and $\mathbf{x}_I$, $\mathbf{u}_I = \mathbf{u}(\mathbf{x}_I)$, and ΔV_I are the coordinates, field value, and integration weight (volume) associated with particle I , respectively. From (7.13), we can identify the RKPM nodal shape functions as

$$\mathbf{u}^{R_h}_a(\mathbf{x}) = \sum_{I=1}^{N_p} \Phi_{aI}(\mathbf{x}) \mathbf{u}_I, \quad (7.14)$$

where

$$\Phi_{aI}(\mathbf{x}) = \mathbf{b}(\mathbf{x}) \cdot \mathbf{P}(\mathbf{x} - \mathbf{x}_I) \phi_a(\mathbf{x} - \mathbf{x}_I) \Delta V_I. \quad (7.15)$$

Unlike the shape functions derived for standard finite element methods, the RKPM shape functions (7.15) do not in general possess the so-called Kronecker delta property, that is

$$\Phi_{aI}(\mathbf{x}_J) \neq \delta_{IJ}. \quad (7.16)$$

The lack of this property complicates the enforcement of essential boundary conditions. Furthermore, this characteristic affects the enforcement of contact constraint and the imposition of cohesive tractions.

[To do: User must specify

(1) the window function

(2) basis functions

(3) order of completeness for polynomial basis functions]

7.3 Meshfree Kirchhoff–Love shell

The element `meshfree_kl_shell` (class `RKShellT`) is a rotation-free Kirchhoff–Love shell discretized with the RKPM of Section 7.2.1. It follows the formulation of Wang and Bazilevs [56]. The shell is described by a cloud of mid-surface nodes; no surface parameterization or connectivity is required for the approximation, although a background mesh of quadrilateral or triangular cells may be supplied for output.

7.3.1 Formulation

A point at through-thickness coordinate $\xi^3 \in [-1, 1]$ is located at

$$\mathbf{x}(\xi^1, \xi^2, \xi^3) = \bar{\mathbf{x}}(\xi^1, \xi^2) + \xi^3 \frac{h}{2} \mathbf{n}, \quad \mathbf{n} = \frac{\bar{\mathbf{x}}_{,1} \times \bar{\mathbf{x}}_{,2}}{\|\bar{\mathbf{x}}_{,1} \times \bar{\mathbf{x}}_{,2}\|}, \quad (7.17)$$

where $\bar{\mathbf{x}}$ is the mid-surface, h the thickness and $\mathbf{n}$ the unit normal. Because the normal follows from the mid-surface, the only unknowns are the three mid-surface displacement components. Membrane strains involve first and curvatures second derivatives of the approximation, so the shape functions must be at least C^1 .

For each node a local chart (ξ^1, ξ^2) is constructed by principal component analysis of its neighborhood, and the RKPM shape functions (7.15) are evaluated on that chart with a basis complete to quadratic order and the cubic B-spline window. The support radius is the product of the normalized support size (`support_factor`) and the nodal spacing, taken as the minimum distance between nodes. Clouds should therefore be quasi-uniform.

The shape functions do not possess the Kronecker delta property, so the nodal unknowns are coefficients, not displacements. Nodal output reports the physical displacement $\mathbf{u}^h(\mathbf{x}_J) = \sum_K \Phi_K(\mathbf{x}_J) \mathbf{d}_K$, and essential boundary conditions must be imposed with the controllers of Section 6.6.

Integrals over the mid-surface are evaluated at the nodes with Voronoi nodal areas A_K . The rank deficiency of pure nodal integration is removed by a first-order Taylor expansion of the membrane virtual work over each nodal cell [56, 24],

$$\delta W_K^{\text{stab}} = \sum_{l=1}^2 \delta \boldsymbol{\varepsilon}_{,\xi^l} : \boldsymbol{\sigma}_{,\xi^l} A_K h \frac{A_K}{12}, \quad (7.18)$$

in which the stress gradient $\boldsymbol{\sigma}_{,\xi^l}$ is carried as a history variable, advanced with the mid-surface algorithmic tangent and co-rotated with the material. An optional bending analogue of (7.18) (`stab_bending`) is available for thin, nearly inextensional, bending-dominated problems; it is not part of the reference formulation.

Stresses are integrated at three Gauss points through the thickness with a plane-stress J_2 model with isotropic hardening

$$Y(\bar{\varepsilon}^p) = Y_0 + H \bar{\varepsilon}^p + (Y_{\text{sat}} - Y_0) (1 - e^{-\delta \bar{\varepsilon}^p}), \quad (7.19)$$

where $\sigma_{33} = 0$ is enforced by iterating on the through-thickness rate of deformation. The stress update is objective, using a co-rotational frame obtained from the polar decomposition of the incremental deformation [18], and the thickness may be updated from the through-thickness rate of deformation. Explicit dynamic analyses use a translational lumped mass $m_K = \rho A_K h$.

7.3.2 Input parameters

The element parameters are listed in Table 7.3. Apart from `completeness`, whose default is cubic, the defaults reproduce the reference formulation; set `completeness="2"` for the quadratic basis of [56].

Table 7.3: Parameters of the `meshfree_kl_shell` element.

parameter	default	description
<code>shell_thickness</code>	–	initial thickness h
<code>Young_modulus</code> , <code>Poisson_ratio</code>	–	isotropic elastic constants
<code>density</code>	1.0	mass density ρ (explicit dynamics)
<code>support_factor</code>	3.0	normalized support size (reference values 2.4–3.0)
<code>completeness</code>	3	order of the polynomial basis; 2 (quadratic) is the reference setting
<code>kernel</code>	1	window function: 1 cubic B-spline, 0 Gaussian
<code>stab_natural</code>	1.0	scale of the membrane stabilization (7.18)
<code>stab_siggrad</code>	1	carry the stress gradient as a history variable (1) or use the tangent operator (0)
<code>stab_bending</code>	0.0	scale of the optional bending stabilization
<code>stab_scn</code> , <code>smoothed_gradient</code>	0	alternative stabilizations based on stabilized conforming nodal integration [14]; not needed with the default
<code>yield_stress</code> , <code>hardening_modulus</code>	0.0	Y_0 and H in (7.19); $Y_0 = 0$ gives an elastic shell
<code>yield_saturation</code> , <code>saturation_rate</code>	0.0	Y_{sat} and δ in (7.19)
<code>finite_strain</code>	0	large-deformation kinematics (required for plasticity)
<code>corotational</code>	1	objective co-rotational stress update
<code>thickness_update</code>	0	update the thickness from the through-thickness rate of deformation
<code>load_x</code> , <code>load_y</code> , <code>load_z</code>	0.0	uniform load per unit mid-surface area
<code>damping</code>	0.0	mass-proportional damping for dynamic relaxation
<code>contact_stiffness</code> , <code>contact_r_in</code> , <code>contact_r_out</code>	0.0	pinball self-contact; radii default to multiples of the nodal spacing
<code>penalty_coupling</code> , <code>couple_node_ID_a</code> , <code>couple_node_ID_b</code>	0.0	rotational-continuity coupling of two shell patches along paired node sets
<code>monitor_node</code> , <code>monitor_stride</code> , <code>monitor_count</code>	0, 1, 1	report the reaction and physical displacement at a node, or summed over a node set

Self-contact and patch coupling are implemented but have not yet been validated on a

complete problem. The internal force computation is parallelized with OpenMP; the number of threads is set by `OMP_NUM_THREADS` and does not change the results.

7.3.3 Output variables

The nodal output values are listed in Table 7.4. In addition, each output step writes diagnostic lines to standard output, among them `[RKShell-uphys]` with the physical displacement at the monitored node and `[RKShell-react]` with its reaction.

7.3.4 Example

The following element block defines the elasto-plastic shell of the pinched-cylinder benchmark; the two pinch points are driven by the controller of Section 6.6.

```
<meshfree_kl_shell field_name="displacement"
  shell_thickness="3.0" Young_modulus="3.0e3"
  Poisson_ratio="0.3" support_factor="2.4"
  completeness="2" kernel="1" density="7800.0"
  yield_stress="24.3" hardening_modulus="300.0"
  finite_strain="1" thickness_update="1"
  monitor_node="281"/>
```

7.3.5 Verification

The element has been verified on the examples of the reference paper [56], as summarized in Table 7.5. The published elasto-plastic pinched-cylinder curves [56, 3, 1] come from symmetry models whose plotted reaction is one quarter of the physical point load; the element reports the physical load. The input decks, scripts and a detailed technical report are in `docs/meshfree_kl_shell` of the source distribution, and regression tests are described in Section 3.3.

Table 7.5: Verification of the meshfree Kirchhoff–Love shell against the reference paper [56].

problem	reference	result
Scordelis–Lo roof [12]	0.3006	0.259, 0.290, 0.298, 0.300 on 315 to 18,193 nodes; paper 0.261, 0.290, 0.298, 0.300
pinched hemisphere [12]	0.0924	0.0207, 0.0787, 0.0909 at 11, 21, 41 nodes per side (full model, <code>stab_bending=1</code>); paper 0.0105, 0.065, 0.090
pinched cylinder [12]	1.8248×10^{-5}	0.593, 1.290, 1.622, 1.745 ($\times 10^{-5}$) on 330 to 19,440 nodes; paper 1.055, 1.627, 1.783, 1.822; converges more slowly than the paper on coarse clouds (membrane locking)
necking of a cylindrical shell	reference Fig. 15	peak 614 MPa (paper 609); within 1–4% of the stabilized curve to $U_{\text{norm}} = 10.7$ mm
elasto-plastic pinched cylinder	reference Fig. 18	within the published band [3, 1] over 0–300 mm at the reference resolution

7.4 Solid deformation

This section describes the elements available for performing analysis of stress and deformation in solids. The material models available for these elements are described in Section 8.

7.4.1 Element 29: finite strain element with enhanced modes

This element formulation is due to Simo, Armero, and Taylor [44]. The element incorporates internal element modes of deformation to improve performance under bending and under nearly incompressible states of deformation.

7.4.2 Element 36: finite strain mixed element

The formulation is due to Simo, Taylor, and Pister [47]. The basic idea behind the formulation is to represent the pressure and dilatation Θ as separate fields from the displacement. For the continuous case, the determinant of the deformation gradient

$$J = \det \mathbf{F} = \mathbf{1} + \frac{\partial \mathbf{u}}{\partial \mathbf{X}} \quad (7.20)$$

is equal to the dilatation. However, when the displacement field $\mathbf{u}$ is restricted to a finite dimensional representation, it may not contain enough degrees of freedom to represent nearly incompressible deformations without making the response overly stiff. Therefore, the dilatation and pressure are represented as separate fields. The modified deformation gradient is

given by

$$\bar{\mathbf{F}} = \left(\frac{\Theta}{J} \right)^{1/3} \mathbf{F}. \quad (7.21)$$

The remainder of the formulation results as a consequence. For Q1P0, the (2D) bi- or (3D) trilinear displacement field (Q1) is combined with piecewise constant (P0) pressure and dilatation fields. Since the space for these fields is restricted to within element domains, these degrees of freedom can be determined analytically at the element level and substituted into the remaining element equations to results in a purely displacement-based formulation.

Note that several errors appear in the derivation of the consistent tangent in the CMAME paper. Therefore, the implementation of the tangent here does not match the published formulation.

7.4.3 Output variables

The values that can be written to output as nodal or element quantities are listed in Tables 7.6 and 7.7, respectively. The element output of integration point values have the labels

$$\text{ip}N. [label] \quad N = 1, \dots, n_{ip},$$

where n_{ip} is the number of element integration points.

7.4.4 Constitutive models

The materials available for analysis of solid deformation are listed in Table 8.1.

7.5 Diffusion

7.5.1 Output variables

7.5.2 Constitutive models

7.6 Cohesive surface elements

At every point along a cohesive surface, we define a local coordinate frame which resolves the opening displacements across the surface into normal and shear components. As shown in Figure 7.7, this coordinate frame is not uniquely defined for finite deformations. We choose to define this coordinate system with respect to the mid-plane between the displaced surfaces. Alternately, this frame may be fixed to the upper or lower facet. The discussion that follows is independent of the particular choice of local coordinate system though we present specific expressions for the mid-plane. Let the transformation of the gap vector from its representation in the global coordinate frame Δ to its local representation $\tilde{\Delta}$ in a frame defined with respect to the cohesive surface be given by

$$\tilde{\Delta} = \mathbf{Q}^T \Delta. \quad (7.22)$$

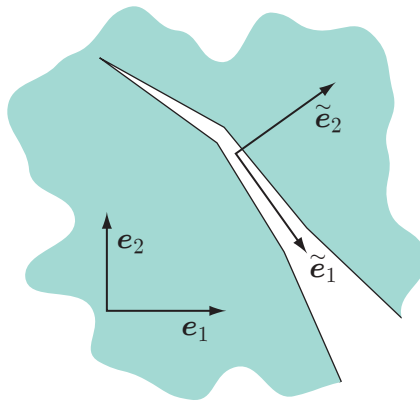


Figure 7.7: Local coordinate frame $\tilde{\mathbf{e}}_i$ constructed along the surface where $\mathbf{e}_i$ is the global cartesian frame.

The transformation resolves the opening displacement into normal and shear components relative to the current orientation of the surface. This distinction allows for the definition of mixed-mode, surface constitutive relations that show a dependence on the character of the opening displacement. In a variational setting, the contribution to the virtual work from cohesive surface elements is

$$\delta w^{\text{CSE}} = \int_{\Gamma_0} \mathbf{T} \cdot \delta \mathbf{\Delta} d\Gamma, \quad (7.23)$$

where the integration domain Γ_0 is defined in the undeformed configuration and $\delta \mathbf{\Delta}$ is the variation in the surface opening displacement. We review the formulation of several surface constitutive models in Section 7.6.2. The discussion that follows applies to all displacement-driven cohesive relations $\tilde{\mathbf{T}}(\tilde{\mathbf{\Delta}})$ defined in the local surface frame. From (7.22), the traction in the global coordinate frame is given by the transformation

$$\mathbf{T} = \mathbf{Q} \tilde{\mathbf{T}}. \quad (7.24)$$

7.6.1 Output variables

Nodal and element output variables are prescribed in the input file by specifying a 1 for variables to be written and a 0 for variables not to be written. The sequence and definition of the nodal and element output variables are described in the next sections.

7.6.1.1 Nodal output variables The output variables written by cohesive surface elements are listed in Tables 7.8 and Table 7.9. Output variables defined at the integration points are projected to the nodes using the algorithm proposed by Zinkiewicz *et al.* [?].

The isotropic and anisotropic element write the same variables; however, some of the variables returned as scalars for the isotropic formulation are returned as vectors for the anisotropic case.

Note that the normal component for the output of the opening displacement and the traction is always the last value. The tangent directions, denoted with the $\mathbf{t1}$ and $\mathbf{t2}$, are determined by the coordinate mapping over surface element and generally vary from element to element.

7.6.1.2 Element output variables The element output variables for both isotropic and anisotropic cohesive surface elements are listed in Table 7.10. Note that some of the variables are element-averaged quantities, while others are integrated over the element.

7.6.2 Constitutive models

The element formulation presented in Section 7.6 makes use of a local coordinate frame defined with respect the deforming facets of the cohesive surface. This construction allows all surface constitutive relations to define the evolution of the local traction $\tilde{\mathbf{T}}$ with respect to the local opening displacement $\tilde{\Delta}$, while the kinematics of large deformation and surface rotations are accounted for at the element level. For the case of an elastic, or reversible, cohesive relation, the traction is derived from a free energy potential φ as

$$\tilde{\mathbf{T}} = \frac{\partial \varphi}{\partial \tilde{\Delta}} \quad (7.25)$$

while the surface stiffness may be expressed as

$$\frac{\partial \tilde{\mathbf{T}}}{\partial \tilde{\Delta}} = \frac{\partial^2 \varphi}{\partial \tilde{\Delta} \partial \tilde{\Delta}}. \quad (7.26)$$

Motivated by the universal binding energy curves of Rose *et al.* [38] from atomistic modeling, Xu and Needleman[59] proposed the mixed-mode cohesive potential

$$\varphi(\tilde{\Delta}) = \phi_n + \phi_n \exp\left(-\frac{\tilde{\Delta}_n}{\delta_n}\right) \left(\left[1 - r + \frac{\tilde{\Delta}_n}{\delta_n} \right] \frac{1-q}{r-1} - \exp\left(-\frac{\tilde{\Delta}_t^2}{\delta_t^2}\right) \left[q + \left(\frac{r-q}{r-1} \right) \frac{\tilde{\Delta}_n}{\delta_n} \right] \right). \quad (7.27)$$

From (7.25), the components of the traction are

$$\tilde{T}_n(\tilde{\Delta}) = \frac{\phi_n}{\delta_n} \exp\left(-\frac{\tilde{\Delta}_n}{\delta_n}\right) \left(\frac{\tilde{\Delta}_n}{\delta_n} \exp\left(-\frac{\tilde{\Delta}_t^2}{\delta_t^2}\right) + \frac{1-q}{r-1} \left[1 - \exp\left(-\frac{\tilde{\Delta}_t^2}{\delta_t^2}\right) \right] \left[r - \frac{\tilde{\Delta}_n}{\delta_n} \right] \right) \quad (7.28)$$

and

$$\tilde{T}_t(\tilde{\Delta}) = 2\phi_n \frac{\tilde{\Delta}_t}{\delta_t^2} \left(q + \left[\frac{r-q}{r-1} \right] \frac{\tilde{\Delta}_n}{\delta_n} \right) \exp\left(-\frac{\tilde{\Delta}_n}{\delta_n}\right) \exp\left(-\frac{\tilde{\Delta}_t^2}{\delta_t^2}\right). \quad (7.29)$$

In (7.27)–(7.29), the ratio $q = \frac{\phi_t}{\phi_n}$ relates the pure normal mode to shear mode fracture energy and $r = \frac{\tilde{\Delta}_n^*}{\delta_n}$ defines $\tilde{\Delta}_n^*$, the value of $\tilde{\Delta}_n$ after complete shear separation with $\tilde{T}_n = 0$.

The effect of r vanishes when $q = 1$, which is the value used for all calculations in this study. The normal and tangential openings are defined as $\tilde{\Delta}_n = \tilde{\Delta} \cdot \tilde{\mathbf{n}}$ and $\tilde{\Delta}_t = \tilde{\Delta} \cdot \tilde{\mathbf{t}}$ with respect to the local normal and tangent. Xu and Needleman developed this cohesive relation in two dimensions. We extend this relation to three dimensions by assuming the response to be isotropic with respect to the tangential opening. That is, the response is expressed only in terms of the magnitude of the tangential opening, which can also be written as

$$\tilde{\Delta}_t = \left| \tilde{\Delta} - \tilde{\Delta}_n \tilde{\mathbf{n}} \right|, \quad (7.30)$$

to emphasize that it is independent of $\tilde{\mathbf{t}}$. For the three-dimensional model, we did not attempt to incorporate any additional effects from the relative rotation of the surfaces about the normal.

7.7 Cohesive surface elements for meshfree domains

7.7.1 Constitutive models

7.7.2 Parameters for adaptive insertion

7.8 Contact

In the input section for contact elements, the user specifies a regularization constant and the entities involved with contact. Contact entities are specified in two sections, contact surfaces and contact, or striker, nodes.

7.8.1 Penalty formulation

With this contact formulation, the penetration of a striker node into a contact surface is penalized with a quadratic potential as

$$U_c = \frac{1}{2} k_c g^2, \quad (7.31)$$

where k_c is the contact stiffness, or regularization parameter, and g is the penetration distance. When used in conjunction with cohesive surface elements, a reasonable choice for the contact stiffness is

$$k_c = k_{\text{CSE}} A, \quad (7.32)$$

where A is the area of the cohesive surface element, and k_{CSE} is the tangent stiffness of the surface constitutive relation. For the potential proposed by Xu-Needleman potential [7.27](#), this stiffness is

$$k_{\text{CSE}} = \frac{\phi}{\delta_n^2}. \quad (7.33)$$

7.8.2 Augmented Lagrangian formulation

The augmented Lagrangian formulation in Tahoe is based on the formulation by Heegaard and Curnier [\[21\]](#).

7.9 Pair-potential

7.10 Two- and three-body potential

7.11 Surface-surface interactions

7.11.1 Element 37: two-point surface interactions

The formulation of surface-surface interactions is similar to what is described by Argento *et al.* [6]. The interaction energy of two surfaces S_1 and S_2 is given by the double-integral

$$U(S_1, S_2, t) = \alpha_{12}(t) \int_{S_1} \int_{S_2} u(|\mathbf{g}|) dS_1 dS_2, \quad (7.34)$$

where u is a potential function and $\mathbf{g}$ is a vector connecting a point on S_1 with a point on S_2 . The function $\alpha_{12}(t)$ is introduced to allow the interaction forces to scale with simulation time. Two of differential elements dS_1 and dS_2 interact if

$$\mathbf{n}_1 \cdot \mathbf{g} > 0 \quad \text{and} \quad \mathbf{n}_2 \cdot \mathbf{g} < 0, \quad (7.35)$$

where $\mathbf{n}_1$ and $\mathbf{n}_2$ are the normals to dS_1 and dS_2 , respectively. The interaction force is then given by

$$\mathbf{f} = -\frac{\partial U}{\partial \mathbf{g}}. \quad (7.36)$$

Table 7.1: Element types.

type	sub-type	code	sub-code
finite element continuum	small deformation	2	N/A
	updated Lagrangian	3	
	total Lagrangian	17	
	updated Lagrangian AES [44]	29	
	updated Lagrangian Q1P0 [47]	36	
	linear diffusion	21	
meshfree continuum	small deformation	18	N/A
	total Lagrangian	19	
	strain gradient total Lagrangian	20	
meshfree shell	Kirchhoff-Love RKPM [56] <code>meshfree_kl_shell</code>	—*	N/A
finite element cohesive surface	isotropic	11	0
	anisotropic with rotating frame		1
	anisotropic with fixed frame		2
meshfree cohesive surface	anisotropic with rotating frame	22	N/A
thermal contact conducting surface	rotating frame	12	N/A
contact	penalty	14	N/A
	augmented Lagrangian (2D)	16	
	ACME [?] library	23	
pair-potential	predefined	1	N/A
	self-connecting	8	
	8 with periodic BC's	9	
two- and three-body potential	Stillinger-Weber [49]	6	N/A
	mixed Stillinger-Weber	7	
	6 with periodic BC's	10	
surface-surface interaction	two-point potential	37	N/A

*Elements added with the XML input format have no numeric code; they are selected by the XML name shown under the sub-type.

Table 7.2: Integration domain geometries and Gauss rules.

geometry	nodes	Gauss rule	code
point	1	1	-1
line	2, 3	1, 2, 3, 4	0
quadrilateral	4–8	1, 4, 9, 16	1
triangle	3, 6	1, 4, 6	2
hexahedron	8, 20	1, 8, 27, 64	3
tetrahedron	4, 10	1, 4	4
pentahedron	6, 15	N/A	5

Table 7.4: Nodal output values for the meshfree Kirchhoff–Love shell.

label	description
D_X, D_Y, D_Z	physical mid-surface displacement $\mathbf{u}^h(\mathbf{x}_J)$
EQ_PLASTIC_STRAIN	maximum equivalent plastic strain over the through-thickness points (when <code>yield_stress</code> > 0)
THICKNESS	current thickness (when <code>yield_stress</code> > 0 and <code>thickness_update</code> =1)

Table 7.6: Nodal output values for solid deformation.

#	var.	description	labels
1	$\mathbf{X}$	reference coordinates	$\mathbf{x}[i] \quad i = 0, \dots, n_{sd}$
2	$\mathbf{d}$	displacements	$\mathbf{D}_-[i] \quad i = \mathbf{X}, \mathbf{Y}, \dots, n_{dof}$
3	$\boldsymbol{\sigma}$	extrapolated stresses	2D: <code>s11</code> , <code>s22</code> , <code>s12</code> 3D: <code>s11</code> , <code>s22</code> , <code>s33</code> , <code>s23</code> , <code>s13</code> , <code>s12</code>
4	$\hat{\boldsymbol{\sigma}}$	extrapolated principal stresses	$\mathbf{s}[i] \quad i = 0, \dots, n_{sd}$
5	ϕ	extrapolated strain energy density	<code>phi</code>
6	c_d, c_s	extrapolated acoustical wave speeds	2D: <code>cd</code> , <code>cs</code> 3D: <code>cd</code> , <code>cs_min</code> , <code>cs_max</code>
7		extrapolated material model output	determined by model

Table 7.7: Element output values for solid deformation.

#	var.	description	labels
1	$\bar{\mathbf{X}}$	reference centroid coordinates	$\mathbf{x}c_ [i] \quad i = 0, \dots, n_{sd}$
2	m	integrated mass	mass
3	ϕ	integrated strain energy	U
4	T	integrated kinetic energy	T
5	$\mathbf{L}$	integrated linear momentum	$\mathbf{L}_ [i] \quad i = X, Y, \dots, n_{dof}$
6	$\boldsymbol{\sigma}$	integration point stresses	2D: s11, s22, s12 3D: s11, s22, s33, s23, s13, s12
7		integration point material output	determined by model

Table 7.8: Nodal output variables for isotropic cohesive surface elements.

#	var.	description	labels
1	$\mathbf{X}$	reference coordinates	$\mathbf{x} [i] \quad i = 0, \dots, n_{sd}$
2	$\mathbf{d}$	displacements	$\mathbf{D}_ [i] \quad i = X, Y, \dots, n_{dof}$
3	$ \Delta $	opening displacements	jump
4	$ \mathbf{T} $	traction	Tmag
5		constitutive output variables	determined by model

Table 7.9: Nodal output variables for anisotropic cohesive surface elements.

#	var.	description	labels
1	$\mathbf{X}$	reference coordinates	$\mathbf{x} [i] \quad i = 0, \dots, n_{sd}$
2	$\mathbf{d}$	displacements	$\mathbf{D}_ [i] \quad i = X, Y, \dots, n_{dof}$
3	$\tilde{\Delta}$	local opening displacements	2D: d_t, d_n 3D: d_t1, d_t2, d_n
4	$\tilde{\mathbf{T}}$	local traction	2D: T_t, T_n 3D: T_t1, T_t2, T_n
5		constitutive output variables	determined by model

Table 7.10: Element output variables for cohesive surface elements.

#	var.	description	labels
1	$\bar{\mathbf{X}}$	centroid in reference coordinates	$\mathbf{xc_}[i] \quad i = 0, \dots, n_{sd}$
2	ϕ	integrated cohesive energy	phi
3	$\bar{t}$	average traction	2D: T_t, T_n 3D: T_t1, T_t2, T_n

8 Constitutive models for solid mechanics

This section of the guide describes the constitutive models available for analysis of displacements and stresses. A separate section is dedicated to each material model. Each section provides some background information about the model followed by a description of the input required for three- and two-dimensional analysis. All materials support implicit as well as explicit analysis. That is, most models implement a consistent tangent matrix that should produce locally quadratic convergence when used with Newton-type, nonlinear solution algorithms (see Section 5.2.2). As will be noted in the sections that follow, some models only provide approximate tangents that will result in slower convergence rates with Newton-type solvers.

Table 8.1: Materials for analysis of displacements and stress.

code	description
1	small deformation Kirchhoff-St. Venant
2	finite deformation Kirchhoff-St. Venant
3	small deformation cubic
4	finite deformation cubic
5	uncoupled volumetric/deviatoric
6	quadratic-logarithmic (Simo)
7	quadratic-logarithmic (Ogden)
8	small deformation J_2 plasticity
9	finite deformation J_2 plasticity
10	finite deformation J_2 plasticity in principal stretches
11	small deformation Drucker-Prager plasticity
12	2D hexagonal Lennard-Jones lattice
13	plane strain FCC Lennard-Jones lattice
14	FCC lattice with EAM potentials
15	Stillinger-Weber diamond cubic
16	Virtual Internal Bond model
17	isotropic Virtual Internal Bond model (Simo)
18	isotropic Virtual Internal Bond model (Ogden)
19	isotropic Virtual Internal Bond model with J_2 plasticity
30	Thermo-viscoplastic model
31	Thermo-viscoplastic model
40	Finite strain viscoplastic model
45	BCJ model
46	BCJ model with damage
47	BCJ model with damage
49	Finite strain crystal elasticity
50	Crystal plasticity model

Table 8.1: Materials for analysis of displacements and stress.

code	description
51	Crystal plasticity model
52	Gradient crystal plasticity model
55	Crystal plasticity model
56	Crystal plasticity model
57	Gradient crystal plasticity model
61	Neo-Hookean viscoelastic model
62	Neo-Hookean viscoelastic model
63	Finite strain viscoelastic Kirchhoff-St.Venant
64	Small strain viscoelastic Kirchhoff-St.Venant
80	ABAQUS/Standard [22] UMAT BCJ
90	ABAQUS/Explicit VUMAT BCJ

8.0.1 Common input parameters

All constitutive models for solid mechanics share some common input parameters. An example of these parameters is shown below:

```
# Rayleigh damping parameters
0.0 # mass proportionate damping
0.0 # stiffness proportionate damping
# mass density
1.0 # mass/reference volume
# thermal expansion
0 # schedule number
0.0 # % expansion
```

The force for Rayleigh damping is defined as

$$\mathbf{F}_R = \mathbf{C}_R \mathbf{v} \quad (8.1)$$

where the damping matrix $\mathbf{C}_R$ is a linear combination of the mass and stiffness matrices

$$\mathbf{C}_R = a \mathbf{M} + b \mathbf{K}, \quad (8.2)$$

where a and b are the parameters specified in the input file.

Thermal expansion may be defined as a function of time with two parameters. The schedule number corresponds to the schedules defined in the time parameters block of the input file 4.2. The second parameter defines the volume change as a percentage. Thermal expansion is disabled by specifying a value of 0 for the schedule number. Thermal strains are imposed differently for small strain and finite strain material models. For small strain materials, an additive decomposition of the strain is assumed

$$\boldsymbol{\epsilon} = \boldsymbol{\epsilon}^\theta + \boldsymbol{\epsilon}^{\text{mechanical}}, \quad (8.3)$$

where the thermal strain is

$$\boldsymbol{\epsilon}^\theta(t) = \varepsilon(t) \mathbf{1} \quad (8.4)$$

For materials formulated at finite strains, the multiplicative split

$$\mathbf{F} = \mathbf{F}^{\text{mechanical}} \mathbf{F}^\theta, \quad (8.5)$$

of the deformation gradient is assumed, where the thermal deformation gradient is

$$\mathbf{F}^\theta(t) = (1 + \varepsilon(t)) \mathbf{1}. \quad (8.6)$$

8.0.2 Common input parameters for 2D analysis

If constructed using the standard **Tahoe** materials model toolbox, all materials for two-dimensional analysis contain two common parameters. An example of these parameters is shown below:

```
##### common 2D parameters
1.0    # thickness
2      # constraint option
```

The thickness t is used to transform the differential volume element as

$$d\Omega = t \, dx_1 \, dx_2. \quad (8.7)$$

The acceptable values for the constraint option are shown in Table 8.2. Some materials

Table 8.2: Constraint options for two-dimensional analysis.

code	description
1	plane stress
2	plane strain

models have been specifically formulated as plane stress or plane strain. These materials will silently override the user-specified value of the constraint option.

8.1 Material 1: small deformation Kirchhoff-St. Venant

The stress response of this isotropic model is governed by the strain energy function

$$\Phi(\boldsymbol{\epsilon}) = \frac{1}{2} \epsilon_{ij} \, \mathbf{C}_{ijkl} \, \epsilon_{kl}, \quad (8.8)$$

where ϵ is the infinitesimal strain tensor and $\mathbf{C}$ is the isotropic tensor of elastic constants which may be written as

$$\mathbf{C} = \begin{bmatrix} \lambda + 2\mu & \lambda & \lambda & 0 & 0 & 0 \\ \lambda & \lambda + 2\mu & \lambda & 0 & 0 & 0 \\ \lambda & \lambda & \lambda + 2\mu & 0 & 0 & 0 \\ 0 & 0 & 0 & \mu & 0 & 0 \\ 0 & 0 & 0 & 0 & \mu & 0 \\ 0 & 0 & 0 & 0 & 0 & \mu \end{bmatrix} \quad (8.9)$$

for the three-dimensional case. The Lamé constants λ and μ are related to Young's modulus E and Poisson's ratio ν by

$$\lambda = \frac{\nu E}{(1 + \nu)(1 - 2\nu)} \quad (8.10)$$

and

$$\mu = \frac{E}{2(1 + \nu)}. \quad (8.11)$$

8.1.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### Kirchhoff-St.Venant
1      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν .

8.1.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### Kirchhoff-St.Venant
1      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### 2D parameters
1.0    # thickness
2      # constraint option
```

After the common parameters 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . The material thickness and constraint option from Table 8.2 follow the parameters for three-dimensional analysis.

8.1.3 Output values

This constitutive model does not have any output values.

8.2 Material 2: finite deformation Kirchhoff-St. Venant

This model is the finite deformation version of the material model described in Section 8.1. The stress response of this isotropic model is governed by the strain energy function

$$\Phi(\mathbf{E}) = \frac{1}{2} E_{ij} \mathbf{C}_{ijkl} E_{kl}, \quad (8.12)$$

where $\mathbf{E}$ is the Green Lagrangian strain tensor

$$\mathbf{E} = \frac{1}{2} (\mathbf{F}^T \mathbf{F} - \mathbf{1}), \quad (8.13)$$

where $\mathbf{F}$ is the deformation gradient and $\mathbf{C}$ is the isotropic tensor of elastic constants given by (8.9).

8.2.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### Kirchhoff-St.Venant
2      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν .

8.2.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### Kirchhoff-St.Venant
2      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### 2D parameters
1.0   # thickness
2     # constraint option
```

After the common parameters 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . The material thickness and constraint option from Table 8.2 follow the parameters for three-dimensional analysis.

8.2.3 Output values

This constitutive model does not have any output values.

8.3 Material 3: small deformation cubic

The stress response of this model is governed by the strain energy function

$$\Phi(\epsilon) = \frac{1}{2} \epsilon_{ij} C_{ijkl} \epsilon_{kl}, \quad (8.14)$$

where ϵ is the infinitesimal strain tensor and $\mathbf{C}$ is the tensor of elastic constants which may be written as

$$\mathbf{C} = \begin{bmatrix} C_{11} & C_{12} & C_{12} & 0 & 0 & 0 \\ C_{12} & C_{11} & C_{12} & 0 & 0 & 0 \\ C_{12} & C_{12} & C_{11} & 0 & 0 & 0 \\ 0 & 0 & 0 & C_{44} & 0 & 0 \\ 0 & 0 & 0 & 0 & C_{44} & 0 \\ 0 & 0 & 0 & 0 & 0 & C_{44} \end{bmatrix} \quad (8.15)$$

for the three-dimensional case. The elasticity tensor in (8.15) has cubic symmetry. The degree of anisotropy is often characterized by the parameter

$$A = \frac{2C_{44}}{C_{11} - C_{12}}, \quad (8.16)$$

where $A = 1$ for isotropy.

8.3.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### cubic model
3      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### cubic moduli
100.0 # C11
10.0  # C12
50.0  # C44
```

After the common parameters described in Section 8.0.1, the model requires specification of the cubic moduli C_{11} , C_{12} , and C_{44} .

8.3.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### cubic model
3      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### cubic moduli
100.0 # C11
10.0  # C12
50.0  # C44
##### orientation
0      # is rotated
0.0    # rotation angle
##### 2D parameters
1.0    # thickness
2      # constraint option
```

After the common parameters described in Section 8.0.1, the model requires specification of the cubic moduli C_{11} , C_{12} , and C_{44} . The cubic moduli are followed by specification of the lattice rotation, consisting of a boolean flag indicating whether the lattice is rotated and the angle rotation. The material thickness and constraint option from Table 8.2 follow the parameters for three-dimensional analysis.

8.3.3 Output values

This constitutive model does not have any output values.

8.4 Material 4: finite deformation cubic

This model is the finite deformation version of the material model described in Section 8.3. The stress response of this isotropic model is governed by the strain energy function

$$\Phi(\mathbf{E}) = \frac{1}{2} E_{ij} C_{ijkl} E_{kl}, \quad (8.17)$$

where $\mathbf{E}$ is the Green Lagrangian strain tensor

$$\mathbf{E} = \frac{1}{2} (\mathbf{F}^T \mathbf{F} - \mathbf{1}), \quad (8.18)$$

where $\mathbf{F}$ is the deformation gradient and $\mathbf{C}$ is the tensor of elastic constants given by (8.15).

8.4.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### cubic model
4      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### cubic moduli
100.0 # C11
10.0  # C12
50.0  # C44
```

After the common parameters described in Section 8.0.1, the model requires specification of the cubic moduli C_{11} , C_{12} , and C_{44} .

8.4.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### cubic model
4      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### cubic moduli
100.0 # C11
10.0  # C12
50.0  # C44
##### orientation
0      # is rotated
0.0    # rotation angle
##### 2D parameters
1.0    # thickness
2      # constraint option
```

After the common parameters described in Section 8.0.1, the model requires specification of the cubic moduli C_{11} , C_{12} , and C_{44} . The cubic moduli are followed by specification of the lattice rotation, consisting of a boolean flag indicating whether the lattice is rotated and the angle rotation. The material thickness and constraint option from Table 8.2 follow the parameters for three-dimensional analysis.

8.4.3 Output values

This constitutive model does not have any output values.

8.5 Material 5: uncoupled volumetric/deviatoric

This model is the finite deformation, isotropic constitutive model proposed by Simo *et al.* [47] for which the response to volumetric and deviatoric deformations is uncoupled. The hyperelastic stress response is derived from the stored energy function

$$\Phi = U(J) + \bar{\Phi}(\bar{\mathbf{b}}) \quad (8.19)$$

$$U(J) = \frac{1}{2} \kappa \left[\frac{1}{2} (J^2 - 1) - \ln J \right] \quad (8.20)$$

$$\bar{\Phi}(\bar{\mathbf{b}}) = \frac{1}{2} \mu (\text{tr } \bar{\mathbf{b}} - 3), \quad (8.21)$$

where κ is the bulk modulus, μ is the shear modulus, $\bar{\mathbf{b}} = J^{-2/3} \mathbf{b}$ is the deviatoric part of the left Cauchy-Green stretch tensor

$$\mathbf{b} = \mathbf{F} \mathbf{F}^T, \quad (8.22)$$

and $J = \det \mathbf{F}$. As a result of the form of the volumetric portion of the stored energy function $U(J)$ (8.20), this model is stable under severe compression.

The bulk and shear moduli are related to Young's modulus E and Poisson's ratio ν by

$$\kappa = \frac{E}{3(1 - 2\nu)} \quad (8.23)$$

and

$$\mu = \frac{E}{2(1 + \nu)}. \quad (8.11)$$

8.5.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### uncoupled volumetric/deviatoric model
5      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν .

8.5.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### uncoupled volumetric/deviatoric model
5      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### 2D parameters
1.0    # thickness
2      # constraint option (override)
```

After the common parameters 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . The material thickness and constraint option from Table 8.2 follow the parameters for three-dimensional analysis. This two-dimensional model is formulated in plane strain, so other values for the constraint option are silently overridden.

8.5.3 Output values

This constitutive model does not have any output values.

8.6 Material 6: quadratic-logarithmic (Simo)

This model is the finite deformation, isotropic constitutive model proposed by Simo *et al.* [43] expressed in principal stretch space. The hyperelastic stress response is derived from the stored energy function expressed in terms of the logarithms of the principal stretches as

$$\Phi(e_1, e_2, e_3) = \frac{1}{2} \lambda (e_1 + e_2 + e_3)^2 + \mu (e_1^2 + e_2^2 + e_3^2), \quad (8.24)$$

where

$$e_A = \log \lambda_A, \quad A = 1, 2, 3. \quad (8.25)$$

The Lamé constants λ and μ are related to Young's modulus E and Poisson's ratio ν by

$$\lambda = \frac{\nu E}{(1 + \nu)(1 - 2\nu)} \quad (8.10)$$

and

$$\mu = \frac{E}{2(1 + \nu)}. \quad (8.11)$$

The implementation of the model follows the spectral formulation developed by Simo and Taylor [46]. Notably, the principal values of the Kirchhoff stress are linearly related to the logarithmic strains

$$\begin{Bmatrix} \tau_1 \\ \tau_2 \\ \tau_3 \end{Bmatrix} = \begin{bmatrix} \kappa + \frac{4}{3}\mu & \kappa - \frac{2}{3}\mu & \kappa - \frac{2}{3}\mu \\ \kappa - \frac{2}{3}\mu & \kappa + \frac{4}{3}\mu & \kappa - \frac{2}{3}\mu \\ \kappa - \frac{2}{3}\mu & \kappa - \frac{2}{3}\mu & \kappa + \frac{4}{3}\mu \end{bmatrix} \begin{Bmatrix} e_1 \\ e_2 \\ e_3 \end{Bmatrix} \quad (8.26)$$

for elastic deformations. As a result of (8.26), the return mapping algorithm is readily transferred to stress space, leading to a highly efficient numerical implementation.

The model described in Section 8.7 uses the stored energy function (8.24) with the spectral formulation developed by Ogden [36]. The Ogden formulation allows more straightforward handling of repeated principal stretches.

8.6.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### quad-log (Simo)
6      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν .

8.6.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### quad-log (Simo)
6      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### 2D parameters
1.0    # thickness
2      # constraint option (override)
```

After the common parameters 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . The material thickness and constraint option from Table 8.2 follow the parameters for three-dimensional analysis. This two-dimensional model is formulated in plane strain, so other values for the constraint option are silently overridden.

8.6.3 Output values

This constitutive model does not have any output values.

8.7 Material 7: quadratic-logarithmic (Ogden)

This model is the finite deformation, isotropic constitutive model proposed by Simo *et al.* [43] expressed in principal stretch space. The hyperelastic stress response is derived from the stored energy function given by (8.24), expressed in terms of the logarithms of the principal stretches. This model differs from the model described in Section 8.7 only in the underlying spectral formulation. Instead of the formulation proposed by Simo and Taylor [46], this model use the spectral formulation developed by Ogden [36]. This formulation allows more straightforward handling of repeated principal stretches.

8.7.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### quad-log (Ogden)
7      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν .

8.7.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### quad-log (Ogden)
7      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### 2D parameters
1.0    # thickness
2      # constraint option (override)
##### moduli
100.0  # Young's modulus
0.25   # Poisson's ratio
```

The material thickness and constraint option from Table 8.2 follow the the common parameters 8.0.1. Finally, the model requires specification of Young's modulus E and Poisson's ratio ν . This two-dimensional model is formulated in plane strain, so other values for the constraint option are silently overridden.

8.7.3 Output values

This constitutive model does not have any output values.

8.8 Material 8: small deformation J_2 plasticity

This model combines the isotropic stored energy function from Section 8.1 with a J_2 yield condition incorporating linear kinematic and isotropic hardening for infinitesimal strain analysis. This model is due to Simo and Hughes [45]. The yield condition is formulated as

$$f(\boldsymbol{\sigma}, \mathbf{q}) := \|\boldsymbol{\eta}\| - \sqrt{\frac{2}{3}}K(\alpha) \leq 0, \quad (8.27)$$

where the internal variables $\mathbf{q} = \{\alpha, \bar{\boldsymbol{\beta}}\}$ are the equivalent plastic strain α and the center of the yield surface $\bar{\boldsymbol{\beta}}$ in deviatoric stress space. The relative stress $\boldsymbol{\eta}$ is

$$\boldsymbol{\eta} = \text{dev}[\boldsymbol{\sigma}] - \bar{\boldsymbol{\beta}}, \quad (8.28)$$

where $\boldsymbol{\sigma}$ is the Cauchy stress. The associative flow rule and hardening laws are

$$\dot{\boldsymbol{\epsilon}}^p = \gamma \frac{\boldsymbol{\eta}}{\|\boldsymbol{\eta}\|}, \quad (8.29)$$

$$\dot{\bar{\boldsymbol{\beta}}} = \gamma \frac{2}{3}H'(\alpha) \frac{\boldsymbol{\eta}}{\|\boldsymbol{\eta}\|}, \quad (8.30)$$

$$\dot{\alpha} = \gamma \sqrt{\frac{2}{3}}, \quad (8.31)$$

where γ is the consistency parameter from the Kuhn-Tucker complementary conditions

$$\gamma \geq 0, \quad f(\boldsymbol{\sigma}, \mathbf{q}) \leq 0, \quad (8.32)$$

and

$$\gamma f(\boldsymbol{\sigma}, \mathbf{q}) = 0. \quad (8.33)$$

The combined linear kinematic and isotropic hardening functions are defined as

$$H(\alpha) = (1 - \theta) \bar{H} \alpha \quad (8.34)$$

and

$$K(\alpha) = \sigma_y + \theta \bar{H} \alpha, \quad (8.35)$$

where σ_y is the initial yield stress and $\bar{H}$ is the single hardening modulus. The parameter $0 \leq \theta \leq 1$ determines the mixity of isotropic to kinematic hardening, with $\theta = 1$ indicating purely isotropic hardening.

8.8.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### small strain J2 plasticity
8      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### plasticity
0      # isotropic hardening function (0:linear)
0.25  # yield stress
0.01  # hardening parameter
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . Finally, the user defines the specification for the isotropic hardening function $K(\alpha)$, the initial yield stress σ_y , and the hardening parameter $\bar{H}$. All available hardening functions are listed in Table 8.5. An example of implementing the general cubic spline is detailed in Section 8.9.

8.8.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### small strain J2 plasticity
8      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### plasticity
0      # isotropic hardening function
0.25  # yield stress
0.01  # hardening parameter
##### 2D parameters
1.0    # thickness
2      # constraint option (override)
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . Following these parameters, the user defines the specification for the isotropic hardening function $K(\alpha)$, the initial yield stress σ_y , and the hardening parameter $\bar{H}$. All available hardening functions are listed in Table 8.5. An example of implementing the general cubic spline is detailed in Section 8.9. The material thickness and constraint option from Table 8.2 follow the parameters for three-dimensional analysis. This two-dimensional model is formulated in plane strain, so other values for the constraint option are silently overridden.

8.8.3 Output values

This constitutive model has three scalar output values listed in Table 8.3.

Table 8.3: Output values for a small strain J_2 plasticity model.

var.	description	labels
α	equivalent plastic strain	alpha
$ \text{dev}[\boldsymbol{\sigma}] $	second invariant of the deviatoric stress	VM
p	pressure	press

8.9 Material 9: finite deformation J_2 plasticity

This model is based on a formulation due to Simo [41, 42] for finite strain J_2 plasticity. The formulation includes both linear isotropic and kinematic hardening; however, the implementation allows specification of an arbitrary isotropic hardening function, but does not include kinematic hardening. The isotropic stored energy function for this model is taken from the model described in Section 8.2. The yield condition is formulated in terms of the Kirchhoff stress $\boldsymbol{\tau}$ as

$$f(\boldsymbol{\tau}, \mathbf{q}) := \|\bar{\boldsymbol{\eta}}\| - \sqrt{\frac{2}{3}} K(\alpha) \leq 0, \quad (8.36)$$

where the internal variables $\mathbf{q} = \{\alpha, \bar{\boldsymbol{\beta}}\}$ are the equivalent plastic strain α and the center of the yield surface $\bar{\boldsymbol{\beta}}$ in deviatoric Kirchhoff stress space. The deviatoric relative stress $\bar{\boldsymbol{\eta}}$ is

$$\bar{\boldsymbol{\eta}} = \text{dev}[\boldsymbol{\tau}] - \bar{\boldsymbol{\beta}}. \quad (8.37)$$

As in the small strain case, the hardening variable is assumed to evolve as

$$\dot{\alpha} = \sqrt{\frac{2}{3}} \gamma, \quad (8.38)$$

where γ is the consistency parameter in (8.32) and (8.33). The flow rule and kinematic hardening law are derived assuming a multiplicative split of the deformation gradient as $\mathbf{F} = \mathbf{F}^e \mathbf{F}^p$. The isotropic hardening functions available for this model are described in Section 8.9.4.

NOTE: The implementation of the consistent tangent moduli for this model has not been verified. The moduli are not used in the stress-update equations, but do affect the rate of convergence for Newton-type solvers. Finite strain, J_2 plasticity is also implemented in the model described in Section 8.10.

8.9.1 3D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### finite strain J2 plasticity
  9      # material code
##### common material parameters
0.0     0.0     1.0
0       0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### plasticity
  2     # isotropic hardening function
##### spline hardening function
  4     # number of points
# [alpha] [K(alpha)]
0.00  0.25
0.01  0.255
0.05  0.26
0.10  0.30
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . The moduli are followed by the specification of the isotropic hardening function $K(\alpha)$. In the example, the function specification 2 indicates the general cubic spline. All available hardening functions are listed in Table 8.5. The points defining the spline follow.

8.9.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### finite strain J2 plasticity
9      # material code
##### common material parameters
0.0    0.0    1.0
0      0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### 2D parameters
1.0    # thickness
2      # constraint option (override)
##### plasticity
2      # isotropic hardening function
##### spline hardening function
4      # number of points
# [alpha] [K(alpha)]
0.00  0.25
0.01  0.255
0.05  0.26
0.10  0.30
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . The material thickness and constraint option from Table 8.2 follow the parameters for three-dimensional analysis. This two-dimensional model is formulated in plane strain, so other values for the constraint option are silently overridden. The parameters for 2D analysis are followed by the specification of the isotropic hardening function $K(\alpha)$. In the example, the function specification 2 indicates the general cubic spline. All available hardening functions are listed in Table 8.5. The points defining the spline follow.

8.9.3 Output values

This constitutive model has three scalar output values listed in Table 8.4.

8.9.4 Hardening functions

The isotropic hardening function for this model is specified by the user. The functions available are listed in Table 8.5. The input parameters for each function are described in the sections that follow. For the nonlinear hardening functions, it is possible to specify the function parameters to produce hardening behavior that is not strictly convex. The

Table 8.4: Output values for a small strain J_2 plasticity model.

var.	description	labels
α	equivalent plastic strain	alpha
$ \bar{\beta} $	norm of kinematic hardening	norm_beta
$\sqrt{\frac{3}{2}} \text{dev}[\boldsymbol{\tau}] $	second invariant of the deviatoric stress	VM_Kirch
p	pressure	press

Table 8.5: Isotropic hardening functions.

code	description
0	linear
1	linear-exponential saturation
2	general cubic spline

local Newton iteration used to compute the return mapping during plastic loading is not guaranteed to converge under these conditions.

8.9.4.1 linear The linear hardening function has the form

$$K(\alpha) = \sigma_y + K \alpha, \quad (8.39)$$

where σ_y is the initial yield stress and K is the hardening modulus. An example of input parameters is shown below:

```
##### linear hardening function
0.25 # initial yield stress
0.01 # hardening modulus
```

8.9.4.2 linear-exponential saturation The linear-exponential saturation hardening function has the form

$$K(\alpha) = \sigma_y + K \alpha + \Delta K_\infty \left(1 - \exp\left[-\frac{\alpha}{\delta}\right]\right), \quad (8.40)$$

where σ_y is the initial yield stress and K is the linear hardening modulus. The saturation hardening has a magnitude of ΔK_∞ and occurs at a characteristic equivalent plastic strain of δ . An example of input parameters is shown below:

```
##### linear-exponential saturation
0.25 # initial yield stress
0.10 # linear hardening modulus
0.05 # saturation hardening
0.05 # saturation strain
```

8.9.4.3 general cubic spline The cubic spline hardening function has the general form

$$K(\alpha) = \begin{cases} K^{(1)}(\alpha_1) + (\alpha - \alpha_1) K^{(1)'}(\alpha_1) & \text{if } \alpha < \alpha_1, \\ K^{(i)}(\alpha) & \text{if } \alpha_i < \alpha < \alpha_{i+1} \text{ for } i = 1, \dots, n-1, \\ K^{(n)}(\alpha_n) + (\alpha - \alpha_n) K^{(n)'}(\alpha_n) & \text{if } \alpha > \alpha_n, \end{cases} \quad (8.41)$$

where the n is the number of points used to define the spline. Over each sub-interval, the spline takes the form

$$K^{(i)}(\alpha) = a_0^{(i)} + a_1^{(i)} \Delta\alpha + a_2^{(i)} \Delta\alpha^2 + a_3^{(i)} \Delta\alpha^3, \quad (8.42)$$

where $\Delta\alpha = \alpha - \alpha_i$. The coefficients $\mathbf{a}^{(i)}$ are computed from the spline points under the conditions that spline pass through the data and that the first derivative of the spline is continuous between sub-intervals. Additionally, the end conditions for the spline are specified as

$$K^{(1)''}(\alpha_1) = K^{(n)''}(\alpha_n) = 0 \quad (8.43)$$

to match the linear extensions assumed beyond the range of the spline data. A specific example of input to define a hardening function is shown below:

```
##### spline hardening function
4      # number of spline points
# [alpha] [K(alpha)]
0.00  0.25
0.01  0.255
0.05  0.26
0.10  0.30
```

Generally, the input has the form

```
##### spline hardening function
[n_pts = number of spline points]
[alpha]      [K(alpha)]
:
[alpha_npts] [K(alpha_npts)]
```

Although not required or enforced, one typically selects $\alpha_1 = 0$ in order to define the initial yield stress specifically with $K(\alpha_1)$.

8.10 Material 10: finite deformation J_2 plasticity in principal stretches

This model combines the isotropic stored energy function from Section 8.6 with a J_2 yield condition incorporating linear kinematic and isotropic hardening for finite strain analysis. This model is due to Simo [43]. The yield condition is formulated in terms of the Kirchhoff stress $\boldsymbol{\tau}$ as

$$f(\boldsymbol{\tau}, \mathbf{q}) := \|\bar{\boldsymbol{\eta}}\| - \sqrt{\frac{2}{3}}K(\alpha) \leq 0, \quad (8.44)$$

where the internal variables $\mathbf{q} = \{\alpha, \bar{\boldsymbol{\beta}}\}$ are the equivalent plastic strain α and the center of the yield surface $\bar{\boldsymbol{\beta}}$ in deviatoric Kirchhoff stress space. The relative stress $\bar{\boldsymbol{\eta}}$ is

$$\bar{\boldsymbol{\eta}} = \text{dev}[\boldsymbol{\tau}] - \bar{\boldsymbol{\beta}}. \quad (8.45)$$

As in the small strain case, the hardening variable is assumed to evolve as

$$\dot{\alpha} = \sqrt{\frac{2}{3}}\gamma, \quad (8.46)$$

where γ is the consistency parameter in (8.32) and (8.33). The flow rule and kinematic hardening law are derived assuming a multiplicative split of the deformation gradient. These will not be outlined here; however, the linear kinematic and isotropic hardening functions

$$H(\alpha) = (1 - \theta)\bar{H}\alpha \quad (8.47)$$

and

$$K(\alpha) = \tau_y + \theta\bar{H}\alpha \quad (8.48)$$

functional analogously to the small strain model described in Section 8.8. τ_y is the initial Kirchhoff yield stress and $\bar{H}$ is the hardening modulus. The parameter $0 \leq \theta \leq 1$ determines the mixity of isotropic to kinematic hardening, with $\theta = 1$ indicating purely isotropic hardening. This model is formulated in principal stretches which preserves the classical return mapping schemes from the infinitesimal theory. An important results of the spectral treatment is that the consistent tangent is symmetric even during plastic loading.

8.10.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### finite strain J2 plasticity
10      # material code
##### common material parameters
0.0     0.0     1.0
0       0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### plasticity
0.25  # yield stress
0.01  # hardening parameter
1.0   # kinematic/isotropic mixity
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . Finally, the user defines the initial yield stress σ_y , the hardening parameter $\bar{H}$, and the kinematic/isotropic hardening mixity parameter θ , for which $\theta = 1$ produces purely isotropic hardening.

8.10.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### finite strain J2 plasticity
10      # material code
##### common material parameters
0.0     0.0     1.0
0       0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### 2D parameters
1.0   # thickness
2     # constraint option (override)
##### plasticity
0.25  # yield stress
0.01  # hardening parameter
1.0   # kinematic/isotropic mixity
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . The material thickness and constraint option

from Table 8.2 follow the parameters for three-dimensional analysis. This two-dimensional model is formulated in plane strain, so other values for the constraint option are silently overridden. Following these parameters, the user defines the initial yield stress σ_y , the hardening parameter $\bar{H}$, and the kinematic/isotropic hardening mixity parameter θ , for which $\theta = 1$ produces purely isotropic hardening.

8.10.3 Output values

This constitutive model has five scalar output values listed in Table 8.6.

Table 8.6: Output values for a finite strain J_2 plasticity model.

var.	description	labels
α	equivalent plastic strain	alpha
$\ \boldsymbol{\tau}\ $	second invariant of the Kirchhoff stress	VM_Kirch
p	pressure	press
$\sigma_{\max}$	maximum principal Cauchy stress	s_max
$\sigma_{\min}$	minimum principal Cauchy stress	s_min

8.11 Material 11: small deformation Drucker-Prager plasticity

This model combines the isotropic stored energy function from Section 8.1 with a pressure-dependent yield condition incorporating linear kinematic hardening for infinitesimal strain analysis. The parameters for this model follow the presentation of Regueiro and Borja [37]. The pressure-dependent yield condition is formulated as

$$f(\boldsymbol{\sigma}, \alpha_1, \alpha_2) := \sqrt{\frac{3}{2}} \|\text{dev}[\boldsymbol{\sigma}]\| + \sqrt{3} \left(\zeta + \beta \frac{\text{tr}[\boldsymbol{\sigma}]}{3} \right) \leq 0, \quad (8.49)$$

where $\boldsymbol{\sigma}$ is the Cauchy stress and

$$\zeta = -\bar{\alpha} + b \alpha_1 + \frac{\alpha_2}{\sqrt{3}}. \quad (8.50)$$

The internal variables

$$\begin{Bmatrix} \alpha_1 \\ \alpha_2 \end{Bmatrix} = - \begin{bmatrix} K' & 0 \\ 0 & H' \end{bmatrix} \begin{Bmatrix} v^p \\ e^p \end{Bmatrix} \quad (8.51)$$

are related to the total volumetric and deviatoric plastic strains v^p and e^p , respectively, through the hardening moduli K' and H' . The material constants $\bar{\alpha}$ and β may be defined in terms of the cohesive $\bar{c}$ and friction angle $\bar{\phi}$ as

$$\bar{\alpha} = \frac{6 \bar{c} \cos \bar{\phi}}{\sqrt{3} (3 + A \sin \bar{\phi})} \quad (8.52)$$

and

$$\beta = \frac{6 \sin \bar{\phi}}{\sqrt{3} (3 + A \sin \bar{\phi})}, \quad (8.53)$$

where $-1 \leq A \leq 1$. The plastic potential function is

$$\varphi(\boldsymbol{\sigma}, \alpha_1, \alpha_2) = \sqrt{\frac{3}{2}} \|\text{dev}[\boldsymbol{\sigma}]\| + \sqrt{3} \left(\zeta + b \frac{\text{tr}[\boldsymbol{\sigma}]}{3} \right). \quad (8.54)$$

Associative plasticity is recovered if $b = \beta$, where b is the material dilation constant, and J_2 plasticity is recovered if $b = \beta = 0$.

8.11.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### small strain Drucker-Prager plasticity
11      # material code
##### common material parameters
0.0     0.0     1.0
0       0.0
##### moduli
100.0 # Young's modulus
0.25  # Poisson's ratio
##### plasticity
0.15  # cohesive-like strength parameter
0.04  # friction-like parameter
0.0   # dilation parameter
0.0   # deviatoric hardening parameter
0.0   # volumetric hardening parameter
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . The cohesion-like strength parameter $\bar{\alpha}$, the friction-like parameter β , the dilatation parameter b , and the hardening moduli follow.

8.11.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### small strain Drucker-Prager plasticity
11      # material code
##### common material parameters
0.0     0.0     1.0
0       0.0
##### moduli
100.0   # Young's modulus
0.25    # Poisson's ratio
##### plasticity
##### plasticity
0.15    # cohesive-like strength parameter
0.04    # friction-like parameter
0.0     # dilation parameter
0.0     # deviatoric hardening parameter
0.0     # volumetric hardening parameter
##### 2D parameters
1.0     # thickness
2       # constraint option (override)
```

After the common parameters described in Section 8.0.1, the model requires specification of Young's modulus E and Poisson's ratio ν . The cohesion-like strength parameter $\bar{\alpha}$, the friction-like parameter β , the dilatation parameter b , and the hardening moduli follow. The material thickness and constraint option from Table 8.2 follow the parameters for three-dimensional analysis. This two-dimensional model is formulated in plane strain, so other values for the constraint option are silently overridden.

8.11.3 Output values

This constitutive model has four scalar output values listed in Table 8.7.

8.12 Material 12: 2D hexagonal Lennard-Jones lattice

[To do: this is a pure plane stress material] The stress response of this material is calculated from an underlying lattice structure and bonding potential by making use of the Cauchy-Born rule. The method was originally described by Born [13] as a means for estimating the theoretical strength of crystals and for assessing the stability of cubic crystal configurations subject simple deformations. The Cauchy-Born rule is a multiscale assumption about how the motion of many atoms can be related to continuum deformation measures. Under the assumption, the atoms in an crystal subject to a homogeneous deformation move according

Table 8.7: Output values for the small strain Drucker-Prager plasticity model.

var.	description	labels
α_2	deviatoric part of stress-like plastic variable	alpha_dev
α_1	volumetric part of stress-like plastic variable	alpha_vol
$ \text{dev}[\boldsymbol{\sigma}] $	second invariant of the deviatoric stress	VM
p	pressure	press

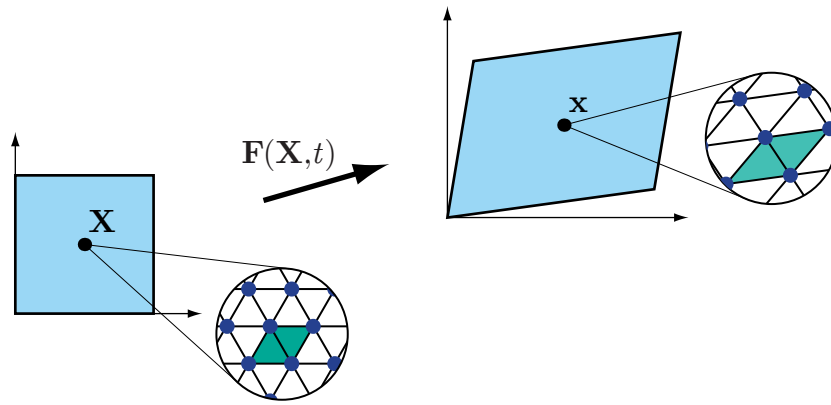


Figure 8.1: Deformation of the underlying microstructure as prescribed by the Cauchy-Born rule.

to a single mapping from the undeformed to the deformed configuration. From the continuum level, this mapping is taken to be the deformation gradient $\mathbf{F}(\mathbf{X}, t)$. As shown in Figure 8.1, the continuum region surrounding a point $\mathbf{X}$ in the undeformed configuration distorts as described by the deformation gradient. The coordinate of the continuum point in the deformed configuration is denoted by $\mathbf{x}$. At a microstructural length scale below the continuum level, we assume the deformation of an underlying crystal lattice undergoes the same homogeneous transformation. The deformation gradient at a point at the continuum length scale is assumed to be constant over a boundless crystal at the microstructural length scale. The microstructural description of a crystalline material and the corresponding continuum constitutive properties are linked using an equivalence in strain energy density. The continuum-level strain energy density is equated to the energy in a representative volume of the microstructure. Since the deformation is homogeneous, the behavior of an arbitrarily large crystal can be studied by considering only the representative volume subject to periodic boundary conditions. For crystalline materials of a single species with a primitive unit cell, this representative volume is the atomic volume. The change in energy per unit volume for a given deformation predicted by this procedure corresponds to the bulk behavior of the

crystal at zero Kelvin. There are no surface or temperature-dependent effects. The atoms in the crystal interact as determined by assumed bonding potential functions. Taken from atomistic calculations, these semi-empirical relationships may be in the form of pairwise potentials, multi-body potentials, or embedded atom method potentials. The number of bonds contributing to the energy in the representative volume depends on the range of influence of the bonding potentials.

For a lattice bound by pair potentials, the strain energy density is

$$\Phi_2(\mathbf{C}) = \frac{1}{\Omega_0} \sum_{i=1}^{n_b} U^{(i)}(r^{(i)}(\mathbf{C})), \quad (8.55)$$

where $\mathbf{C} = \mathbf{F}^T \mathbf{F}$ is the right Cauchy-Green stretch tensor, n_b is the number of bonds, Ω_0 is the representative volume, $r^{(i)}$ is the deformed bond length, and $U^{(i)}(r)$ is a pairwise bond potential. Typically, all the bonds are assumed to be governed by the same potential function $U(r)$. Since the deformation is assumed homogeneous over the crystal, the transformation defined by $\mathbf{F}$ can be applied to vectors of finite length. For each bond in the undeformed unit cell, we define a bond vector

$$\mathbf{R} = R \mathbf{\Xi}, \quad (8.56)$$

where R and $\mathbf{\Xi}$ are the undeformed bond length and direction, respectively. Assuming the deformation gradient $\mathbf{F}$ is homogeneous over the representative volume, we can express the deformed bond length as

$$r(\mathbf{C}) = R \sqrt{\mathbf{\Xi} \cdot \mathbf{C} \mathbf{\Xi}}. \quad (8.57)$$

Given the strain energy density function (8.55), we can derive a complete description of the stress response and elastic tangent moduli using Green elastic theory [36].

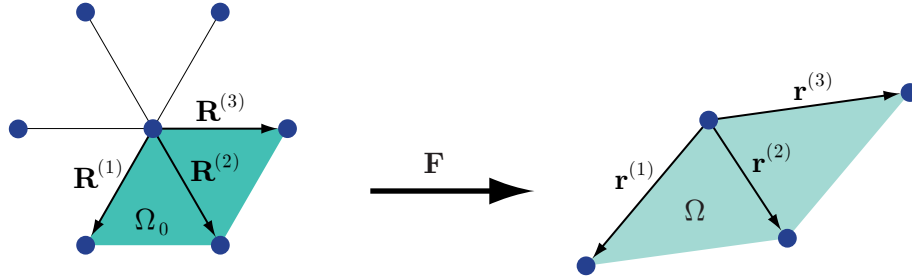


Figure 8.2: Deformation of the unit cell of a two-dimensional hexagonal lattice subject the Cauchy-Born rule.

Figure 8.2 shows the primitive unit cell of a two-dimensional, hexagonal lattice with nearest neighbor bonding for which $n_b = 3$. In the undeformed configuration, the bond vectors are given by

$$\mathbf{R}^{(1)} = a_0 \begin{Bmatrix} -1/2 \\ -\sqrt{3}/2 \end{Bmatrix} \quad \mathbf{R}^{(2)} = a_0 \begin{Bmatrix} 1/2 \\ -\sqrt{3}/2 \end{Bmatrix} \quad \mathbf{R}^{(3)} = a_0 \begin{Bmatrix} 1 \\ 0 \end{Bmatrix}, \quad (8.58)$$

where a_0 is the undeformed lattice parameter.

Following Green elastic theory, the Cauchy stress and spatial tangent modulus are given by

$$\boldsymbol{\sigma} = \sigma \mathbf{1}, \quad (8.59)$$

where the magnitude of the equibiaxial stress is

$$\sigma = \frac{3}{2} \frac{a_0}{\lambda \Omega_0} U'(\lambda a_0), \quad (8.60)$$

and

$$\mathbf{c} = \mu [\mathbf{1} \otimes \mathbf{1} + 2 \mathbf{I}], \quad (8.61)$$

where the instantaneous shear modulus is

$$\mu = \frac{3}{8 \Omega_0} \left[a_0^2 U''(\lambda a_0) - \frac{a_0}{\lambda} U'(\lambda a_0) \right]. \quad (8.62)$$

With instantaneous isotropic response under the restriction of Cauchy symmetry, the complete description of the modulus involves only the single parameter μ from (8.62). Cauchy symmetry for this two-dimensional model implies a fixed Poisson's ratio $\nu = \frac{1}{3}$.

The earliest potentials used in lattice modeling were constructed to display the essential features of cohesive interactions between atoms with parameters fitted to available experimental data, such as lattice parameters and elastic properties in the undeformed state. Born [13] employed an “inverse power” potential of the form

$$U(r) = D \frac{nm}{n-m} \left[\frac{1}{n} \left(\frac{a_0}{r} \right)^n - \frac{1}{m} \left(\frac{a_0}{r} \right)^m \right], \quad (8.63)$$

where $n > m$, D , and a_0 were chosen to match experimental data. In particular, the well-known Lennard-Jones potential results from (8.63) with $m = 6$ and $n = 12$.

8.12.1 3D input parameters

This model is formulated in two-dimensions. Other models use the Cauchy-Born rule to calculate the stress response of single crystals in three dimensions.

8.12.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

8.12.3 Output values

This constitutive model does not have any output values.

8.13 Material 13: plane strain FCC Lennard-Jones lattice

see 8.12 With instantaneous isotropic response under the restriction of Cauchy symmetry, the complete description of the modulus involves only the single parameter μ from (8.62). Cauchy symmetry for a three-dimensional lattice implies a fixed Poisson's ratio $\nu = \frac{1}{4}$.

Table 8.8: FCC lattice orientation options.

code	orientation
0	$\mathbf{e}_1: [1\ 0\ 0]$ $\mathbf{e}_3: [0\ 0\ 1]$
1	$\mathbf{e}_1: [1\ 0\ \bar{1}]$ $\mathbf{e}_3: [1\ 0\ 1]$
2	$\mathbf{e}_1: [1\ 0\ \bar{1}]$ $\mathbf{e}_3: [1\ 1\ 1]$

8.13.1 3D input parameters

Although the underlying lattice for this material is three-dimensional, it has only be implemented for plane strain deformation. Other models use the Cauchy-Born rule to calculate the stress response of single crystals in three dimensions.

8.13.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

8.13.3 Output values

This constitutive model does not have any output values.

8.14 Material 14: FCC lattice with EAM potentials

The stress response of this model is calculated by applying the Cauchy-Born, described in Section 8.12, rule together with embedded atom method potentials. Application of the Cauchy-Born rule dates back to the work of Born [13]. More recently, Tadmor *et al.* [50], used the method with the embedded atom method potentials as a means to derive nonlinear, hyperelastic constitutive models for use in numerical simulations to predict stress and deformation. The embedded atom method [16] (EAM) was developed in an attempt to incorporate more physical understanding into the bond potential functions. Originally motivated by the view of metals as nuclei in a sea of electrons, the total energy of any atom in the crystal is comprised of a contribution due to the nuclear repulsion and an embedding energy

representing the attraction of the nucleus to the background electron density. In fitting a set of EAM potentials to a particular material, one must define three functions: a pair potential describing nuclear interactions, the electron density distribution around each nucleus, and a function describing the interaction between a nucleus and the background electron density. Although quite different from simple pair interactions, EAM potentials may be incorporated into the procedures for deriving constitutive properties without any modifications to the assumptions of the Cauchy-Born rule.

Using EAM potentials, the strain energy density has two contributions,

$$\Phi_{\text{EAM}} = \Phi_2 + \Phi_e, \quad (8.64)$$

where Φ_2 is the contribution from the nuclear interactions given by the pair potential expression (8.55) and Φ_e is the embedding energy. This additive split in the total energy results in additive contributions to the stress response and tangent moduli. Therefore, all expressions derived from (8.55) still apply, and we only need to discuss the additional contribution due to the embedding energy. The contribution to the total energy (8.64) due to the embedding energy may be written as

$$\Phi_e = \frac{1}{\Omega_0} U_e(\bar{\rho}_e) \quad (8.65)$$

where $\bar{\rho}_e$ is the background electron density, that is, the electron density at the nucleus of a particular atom resulting from all neighbors of the atom, but not the atom itself. As a first approximation, the electron density is assumed to be centrosymmetric

$$\bar{\rho}_e = \sum_{i=1}^{n_b} \rho_e(r^{(i)}) . \quad (8.66)$$

Table 8.9: Embedded atom method potentials.

code	potential
0	Al: Ercolessi and Adams [17]
1	Al: Voter and Chen [55]
2	Cu: Voter and Chen [54]
3†	Au: Foiles, Baskes, and Daw [?]

† This potential requires the file `auu3` with the potential data in ParaDyn [?] format.

[To do: print tables of modulus values and units for each potential.]

8.14.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

8.14.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

8.14.3 Output values

This constitutive model does not have any output values.

8.15 Material 15: Stillinger-Weber diamond cubic

The cubic unit cell for the diamond cubic structure is shown in Figure 8.3. The unit cell

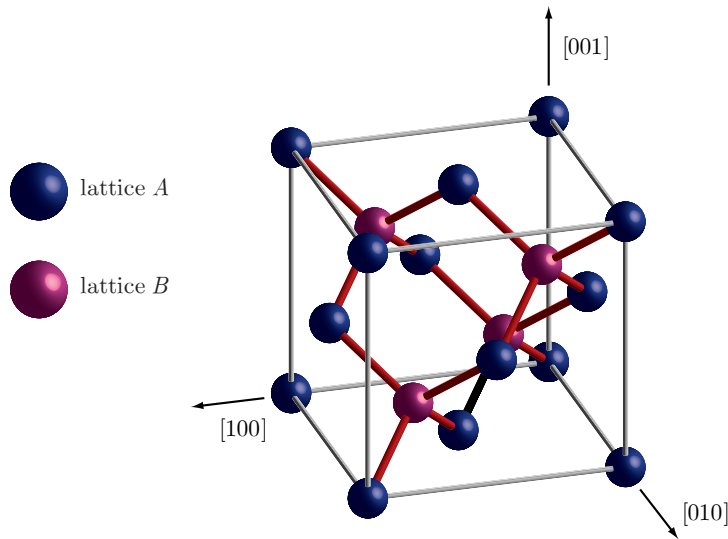


Figure 8.3: Cubic unit cell of the diamond cubic lattice.

can be constructed from two interlaced face-centered cubic lattices. The lattice B in the figure is positioned relative to lattice A by a displacement of $\{1/4, 1/4, 1/4\}$ of the cube edge dimensions. Stakgold[48] discussed how such structure, which Wiener[57] called a composite lattice, should be treated, though he limits his discussion to pair potentials and his approach does not strictly guarantee objectivity of the resulting strain energy density function. In general terms, symmetry constraints require that each constituent sub-lattice deforms homogeneously, but the sub-lattices can move relative to each other by a rigid body translation which depends on the interatomic potentials. The rigid body motion is determined by minimizing the energy of the crystal as a function of the translational degrees of freedom. By enforcing this condition, static equilibrium is satisfied for all atoms in the lattice.

Keeping in mind that the degrees of freedom will be introduced in a manner which maintains objectivity, we can express a modified strain energy density function as

$$\Phi(\mathbf{C}) = \check{\Phi}(\mathbf{C}, \Xi(\mathbf{C})), \quad (8.67)$$

where Ξ represents a vector of all the degrees of freedom of $\check{\Phi}$. The degrees of freedom are implicit functions of the stretch $\mathbf{C}$. Given a strain energy density function of the form (8.67), we can apply the standard relations from Green elastic theory to derive expressions for the stress response and tangent moduli. The internal degrees of freedom are added as

$$\mathbf{x} = \mathbf{F}(\mathbf{X} + \Xi) \quad (8.68)$$

Though perhaps not the most accurate for all types of simulations, the original potentials by Stillinger and Weber [49] are still highly attractive due to their simplicity. With these potentials, the strain energy density is composed of two contributions,

$$\Phi_{\text{SW}} = \Phi_2 + \Phi_3. \quad (8.69)$$

Here, Φ_2 represents the contribution from standard inverse-power pair potentials, and Φ_3 represents the contribution from three-body potentials which are needed to stabilize the tetrahedral structures that compose the lattice. For a range of deformations around the undeformed state, these potentials involve only nearest neighbor bonds. In the original formulation[49], the two-body potential is given by

$$U_2(r) = \begin{cases} A(Br^{-p} - r^{-q}) \exp\left[\frac{\delta}{r-r_{\text{cut}}}\right] & r < r_{\text{cut}} \\ 0 & r \geq r_{\text{cut}}, \end{cases} \quad (8.70)$$

and the three-body potential is

$$U_3(r^{(ji)}, r^{(jk)}, \theta^{(ijk)}) = \begin{cases} \lambda \left(\frac{1}{3} + \cos \theta^{(ijk)}\right)^2 \exp\left[\frac{\gamma}{r^{(ji)}-r_{\text{cut}}}\right] \exp\left[\frac{\gamma}{r^{(jk)}-r_{\text{cut}}}\right] & r^{(ji)}, r^{(jk)} < r_{\text{cut}} \\ 0 & r^{(ji)}, r^{(jk)} \geq r_{\text{cut}}. \end{cases} \quad (8.71)$$

Both potentials make use of an exponential cut-off term which causes the potentials, and all derivatives of the potentials, to vanish smoothly when the atomic spacing r reaches r_{cut} . This cut-off term is advantageous because it clearly defines the distance over which atoms interact and provides a smooth termination for all the potentials through to the second derivatives required to compute the moduli. U_3 (8.71) is clearly designed to favor the “ideal” tetrahedral angle of $\theta_t = \cos^{-1}(-1/3)$, though the functional form is not motivated by any detailed attributes of the sp^3 orbitals which produce this configuration. In terms of (8.70) and (8.71), we can express the strain energy density (8.69) as

$$\Phi_{\text{SW}} = \frac{1}{\Omega_0} \left[\sum_{(ij)=1}^8 U_2(r^{(ij)}) + \sum_{(ijk)=1}^{12} U_3(\tilde{\mathbf{r}}^{(ijk)}) \right], \quad (8.72)$$

where we represent the arguments to the three-body term as a vector composed of the length of two bond vectors and the cosine of the angle they subtend,

$$\tilde{\mathbf{r}}^{(ijk)} = \begin{Bmatrix} r^{(ji)} \\ r^{(jk)} \\ \cos \theta^{(ijk)} \end{Bmatrix}. \quad (8.73)$$

Table 8.10: Diamond cubic lattice orientation options.

code	orientation
0	$\mathbf{e}_1: [1\ 0\ 0]$ $\mathbf{e}_3: [0\ 0\ 1]$
1	$\mathbf{e}_1: [1\ 0\ \bar{1}]$ $\mathbf{e}_3: [1\ 0\ 1]$
2	$\mathbf{e}_1: [1\ 0\ \bar{1}]$ $\mathbf{e}_3: [1\ 1\ 1]$

[To do: with and without internal equilibration.]

8.15.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

8.15.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

8.15.3 Output values

This constitutive model does not have any output values.

8.16 Material 16: Virtual Internal Bond model

The general features of the VIB model, as well as more detailed study of its localization behavior under conditions of dynamic crack propagation, are described in greater detail elsewhere[19, 25, 26, 27]. The VIB model is developed within the framework of hyperelasticity. The current, or deformed, configuration of a body is described as $\mathbf{x} = \boldsymbol{\varphi}(\mathbf{X})$, by a mapping $\boldsymbol{\varphi}$ of the undeformed configuration $\mathbf{X}$. The arrangement of cohesive interactions among material particles is described by a spatial bond density function. The strain energy density is computed by integrating the bond density in space in a continuous analog to the sum over discrete lattice neighbors for the case of crystalline materials. The VIB form of the strain energy density function is

$$\Phi = \frac{1}{\Omega_0} \int_{\Omega_0^*} U(l) D_\Omega d\Omega, \quad (8.74)$$

where Ω_0 is the undeformed representative volume, l is the deformed virtual bond length, $U(l)$ is the bonding potential, D_Ω is the volumetric bond density function, and Ω_0^* is the integration volume defined by the range of influence of U .

[To do: show results for the fracture energy, cohesive strength, and modulus in terms of potential, and then show specific results for the Smith-Ferrante potential. Is the potential selectable?]

Unlike constitutive models based on the Cauchy-Born rule for crystalline materials described in Section 8.14, the VIB model may suffer significant numerical errors when evaluating integrals of the bond distribution to determine the strain energy density (8.74) and the resulting stress response and moduli. Many different schemes could be developed for evaluating the VIB model integrals. As with any numerical procedure, the optimal scheme represents a compromise between accuracy and speed of execution. The primary concern with the implementation of the isotropic VIB model is that the response in numerical calculations is indeed lacking in any orientational dependence resulting from the integration procedure. For the plane stress, isotropic VIB model, the integral of the volumetric bond density function reduces to an integral over the interval $-\pi \leq \phi \leq \pi$. A simple scheme for evaluating these integrals is shown in Figure 8.4. The uniform bond distribution is sampled

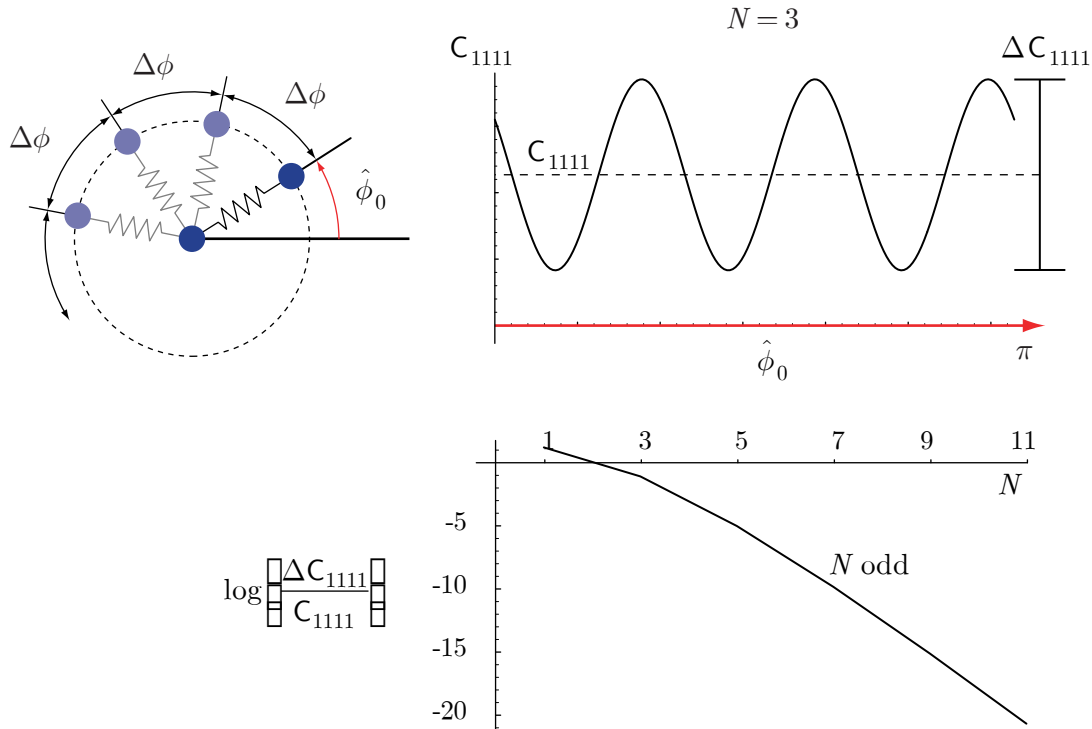


Figure 8.4: Convergence to isotropic response in the numerical implementation of the plane stress, isotropic VIB model.

at fixed intervals $\Delta\phi$. The resulting values of the stress and modulus should be independent of the orientation of the “microstructure”, represented by the offset angle $\hat{\phi}_0$ in positioning the integration points. Clearly, the value of the VIB integrals for the stress and modulus will not be independent of $\hat{\phi}_0$ if an insufficient number of sampling points is used. The variation of the C_{1111} component of the spatial tangent modulus for an arbitrarily chosen state of

deformation is shown in Figure 8.4 for the case of $N = 3$ sampling points. As is evident from the figure, the value displays a six-fold symmetric variation. The $N = 3$ configuration corresponds to the bonding arrangement in a two-dimensional, close-packed lattice. This is the lattice arrangement on the close-packed planes of the face-centered cubic (FCC) lattice and on the basal plane of the hexagonal close-packed (HCP) lattice. For calculations at infinitesimal deformations, these planes are commonly assumed to be isotropic. However, only the instantaneous response of the undeformed lattice lacks orientational dependence. For deformations at finite strains, this crystal plane displays six-fold symmetric anisotropy. Figure 8.4 also shows how the numerical response converges toward isotropy as the number of integration points is increased. The magnitude of the variation in the modulus ΔC_{1111} relative to the “exact” value for the applied deformation C_{1111}^∞ drops to below machine precision with only 11 integration points. Therefore, the computational effort to produce essentially isotropic response in the numerical implementation does not present significant difficulties. The evenly-spaced integration point arrangement seems to be a highly efficient scheme for evaluating the VIB integrals. For a given number of points, the results using integration schemes based on evenly-spaced points display much smaller dependencies on orientation than do the results using integration schemes based on Gaussian quadrature.

For the three-dimensional isotropic VIB model, the task of generating isotropic response is significantly more difficult. This case requires evaluating the VIB integrals over a sphere. Unlike the two-dimensional case, there are no schemes for arranging an arbitrary number of points on a sphere in an evenly-spaced manner. The icosahedron with its 20 equilateral triangular faces presents a starting point from which more dense arrangements of points can be constructed. Integration rules with 80 and 320 points in arrangements which are symmetric about the “equator” are shown in Figure 8.5. These arrangements are created by

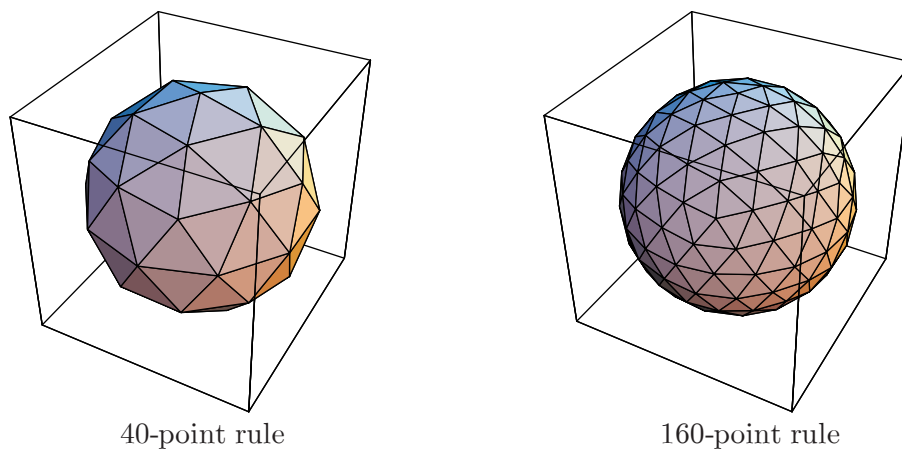


Figure 8.5: Integration point arrangements in three dimensions based on the icosahedron.
[To do: get picture of lat-long points.]

successively bisecting the edges of the triangular facets and projecting the vertex points onto the sphere. The center of each facet is used as an integration point with a weight selected

to produce the correct result when integrating a constant over the surface of the sphere. For a rotation about an arbitrarily selected axis, the variation in the C_{1111} component of the spatial tangent modulus is shown in Figure 8.6 for the integration schemes depicted

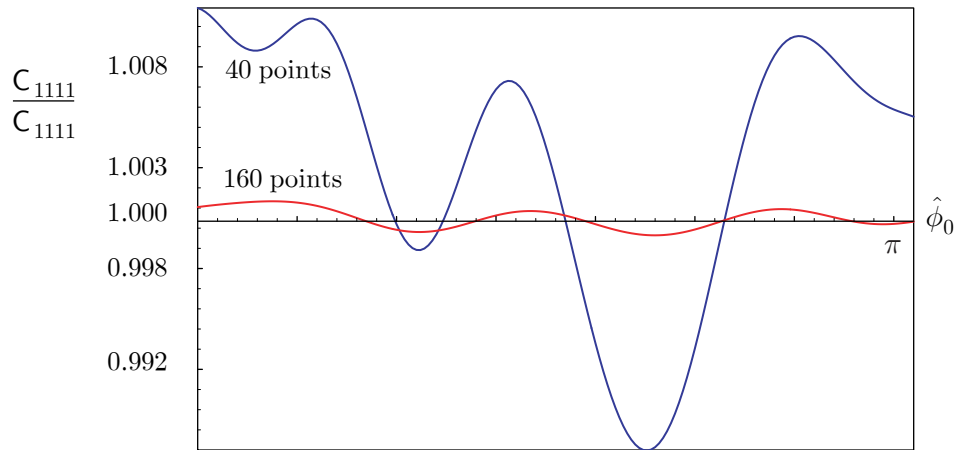


Figure 8.6: Orientational dependence of C_{1111} for the three-dimensional isotropic VIB model in a deformed state.

in Figure 8.5. The figure shows that the modulus displays noticeable anisotropic behavior although a much larger number of points is used than would be required in two dimensions to produce essentially no error. Bažant[9] has developed more sophisticated schemes for numerical integration over the surface over of a sphere, but even these schemes produce a significant increase in computational effort with the extension from two to three dimensions.

8.16.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### VIB
16      # material code
##### common material parameters
0.0     0.0     1.0
0       0.0
##### potential parameters
1       # potential code
4.775 # A
0.0643# B
##### integration parameters
1       # integration rule
40      # number of integration points
```

8.16.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

```
##### VIB
16      # material code
##### common material parameters
0.0     0.0     1.0
0       0.0
##### 2D parameters
1.0     # thickness
1       # constraint option (override)
##### potential parameters
1       # potential code
4.775 # A
0.0643# B
##### integration parameters
0       # integration rule
11      # number of integration points
```

8.16.3 Output values

This constitutive model does not have any output values.

8.17 Material 17: isotropic Virtual Internal Bond model (Simo)

This model combines the VIB strain energy density from Section 8.16 with the spectral formulation developed by Simo and Taylor [46]. Expressed in principal stretches, the model displays exactly isotropic response regardless of the accuracy of the scheme used to integrate the bond distribution. *[To do: However, the model will not display the modulus, fracture energy, or cohesive strength developed from analysis of the model.]*

8.17.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

8.17.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

8.17.3 Output values

This constitutive model does not have any output values.

8.18 Material 18: isotropic Virtual Internal Bond model (Ogden)

This model combines the VIB strain energy density from Section 8.16 with the spectral formulation developed by Ogden [36]. Expressed in principal stretches, the model displays exactly isotropic response regardless of the accuracy of the scheme used to integrate the bond distribution. *[To do: However, the model will not display the modulus, fracture energy, or cohesive strength developed from analysis of the model.]*

8.18.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

8.18.2 2D input parameters

An example of input parameters for two-dimensional analysis is shown below:

8.18.3 Output values

This constitutive model does not have any output values.

8.19 Material 19: isotropic Virtual Internal Bond model with J_2 plasticity

This model combines the VIB strain energy density from Section 8.16 with the spectral formulation developed by Simo and Taylor [46] and the finite strain framework of J_2 plasticity from Simo [43] described in Section 8.10. Expressed in principal stretches, the model displays exactly isotropic response regardless of the accuracy of the scheme used to integrate the bond distribution. *[To do: However, the model will not display the modulus, fracture energy, or cohesive strength developed from analysis of the model.]*

The yield condition is formulated in terms of the Kirchhoff stress $\boldsymbol{\tau}$ as

$$f(\boldsymbol{\tau}, \mathbf{q}) := \|\boldsymbol{\eta}\| - \sqrt{\frac{2}{3}} K(\alpha) \leq 0, \quad (8.75)$$

where the internal variables $\mathbf{q} = \{\alpha, \bar{\boldsymbol{\beta}}\}$ are the equivalent plastic strain α and the center of the yield surface $\bar{\boldsymbol{\beta}}$ in deviatoric Kirchhoff stress space. The relative stress $\boldsymbol{\eta}$ is

$$\boldsymbol{\eta} = \text{dev}[\boldsymbol{\tau}] - \bar{\boldsymbol{\beta}}. \quad (8.76)$$

As in the small strain case, the hardening variable is assumed to evolve as

$$\dot{\alpha} = \sqrt{\frac{2}{3}} \gamma, \quad (8.77)$$

where γ is the consistency parameter in (8.32) and (8.33). The flow rule and kinematic hardening law are derived assuming a multiplicative split of the deformation gradient. These will not be outlined here; however, the linear kinematic and isotropic hardening functions

$$H(\alpha) = (1 - \theta) \bar{H} \alpha \quad (8.78)$$

and

$$K(\alpha) = \tau_y + \theta \bar{H} \alpha \quad (8.79)$$

functional analogously to the small strain model described in Section 8.8. τ_y is the initial Kirchhoff yield stress and $\bar{H}$ is the hardening modulus. The parameter $0 \leq \theta \leq 1$ determines the mixity of isotropic to kinematic hardening, with $\theta = 1$ indicating purely isotropic hardening.

8.19.1 Output values

[To do: output values]

8.80 Material 80: ABAQUS/Standard UMAT BCJ

This model makes of the the ABAQUS/Standard [22] UMAT interface in Tahoe. This model is due to Bammann, Chiesa, and Johnson [?]. The ABAQUS/Standard UMAT interface in Tahoe parses ABAQUS-format input. The specifications for UMAT's in Tahoe may be taken directly from an ABAQUS Standard input deck.

Like all material models, the materials using the ABAQUS/Standard UMAT interface may be used for analysis types not supported by ABAQUS/Standard. Namely, the materials may be applied to dynamic analysis using explicit time integration. Using this time integration scheme, the UMAT's will calculate both the stress and modulus, unless specifically rewritten to detect whether both or just one is actually needed for the analysis. Unmodified UMAT's will perform unnecessary calculations, leading to longer simulations times. This model requires an installation of Tahoe that include the f2c [?] module.

8.80.1 3D input parameters

An example of input parameters for three-dimensional analysis is shown below:

```
##### ABAQUS/Standard BCJ
80      # material code
##### common material parameters
0.0      0.0      1.0
0      0.0
##### ABAQUS input deck
*****
*MATERIAL, NAME=A356
**
** user subroutine is dmg1.f using metric units
** material name = alum 356
** creation date = 08/08/96
** data fit by   = Jim Lathrop for uscar A356 temp-strain rate
*USER MATERIAL, CONSTANTS=52
**
**G, a,Bulk,b,melttemp,C1, C2, C3
** C4, C5, C6, C7, C8, C9, C10, C11
**C12,C13,C14,C15,C16,C17, C18, C19
**C20,Ca,Cb,init.temp,heat gen.coeff,void growth exp,initial rad,tors constant a
**nuc const b,nuc const c,nuc coeff,fract.tough,part.size,part.vol.fract,cd1,cd2
**
2.592E+04 1.000E+00 6.763E+04 0.000E+00 5.556E+03 5.309E+01 9.453E+02 1.559E+02
1.105E+02 1.000E-05 0.0000000 1.128E-03 -1796.200 4.820E+03 1.094E+01 2.385E-03
1.441E+03 1.674E-03 0.000E+00 2.818E+03 4.622E+00 0.000E+00 0.000E+00 0.000E+00
0.000E+00 -5.000000 -0.389490 297.00000 0.0000000 0.3000000 0.0002000 615460.00
```

```
58640.000 30011.000 86.600000 17.300000 4.000E-06 0.0700000 4.5000000 40.000000
20.00E-00 20.00E-00 0.0509000 0.0010000 0.0000000 0.0000000 0.0000000 0.0000000
0.0000000 0.0000000 0.0035000 0.0090000
```

```
*DEPVAR
```

```
25
```

[To do: This is for A356]

8.80.2 2D input parameters

The parameters for using the *[To do: same as for three-dimensional analysis. Since this model is formulated in three-dimensions, the thing will be plane strain when used for two dimensional analysis. Mention this above in the general description of the UMAT interface.]*

8.80.3 Output values

[To do: output values determined by the implementation of the UMAT. Show output parameters for A356.]

9 Constitutive models for cohesive surface elements

This section of the guide describes the constitutive models that may be used as material models specifying the traction vector as a function of the displacement discontinuity for the cohesive surface elements described in Section 7.6.2. Each of the following subsections provides some background information about its respective material model followed by a description and example of the input required for analysis. All cohesive surface models have been implemented in two dimensions, and most also have three-dimensional implementations. All surface materials support implicit and explicit analysis; in other words, all models implement consistent tangent matrices that produce quadratic convergence near local minima when used with the Newton-type, nonlinear solution algorithms (see Section 5.2.2).

Table 9.1: Constitutive models for cohesive surface elements

code	description
0	Xu-Needleman (elastic)
1	Tvergaard-Hutchinson (elastic)
2	Linear Damage
3	Tvergaard-Hutchinson (elastic + viscous dissipation)
4	Tijssens (elastic/viscoplastic)
5	trilinear rate-dependent
6	“Tied” potential (elastic)
7	Yoon, Allen, and Searcy (linear viscoelastic)
8	thermomechanical model (not implemented)
9	rate-based ductile fracture model
10	elastoplastic model for geomaterials
11	rigid-plastic model for geomaterials

9.0.1 Background

The Tahoe input for the cohesive surface elements described in Section 7.6.2 specify the traction-separation relation(s) for a given cohesive surface element block by specifying a list of material codes ranging from 0 to 7 along with any required model parameters. In the following sections, “Surface Material x ” refers to the constitutive relation specified by a material code of “ x ” for a given element block. In these constitutive models, the displacement discontinuity between two opposite crack faces is quantified by the gap vector Δ whose components Δ_t and Δ_n in two dimensions are specified for directions along and normal to the midplane between the two crack faces. In three dimensions, the components are Δ_{t1} ,

Δ_{t2} , and Δ_n . Each constitutive model described below returns the traction vector $\mathbf{T}$ as a function of Δ and any internal state variables $\{\mathbf{q}_i\}$ the model has. Also, many of these models specify length scales $\delta_i, i = n, t$ in the normal and tangential directions directions ($i = n, t1, t2$ in 3D), and one defines a dimensionless vector λ related to the gap vector Δ via

$$\lambda \equiv \frac{\Delta_i}{\delta_i}. \quad (9.1)$$

With this notation many models define the traction vector as a function of the λ_i or indeed of the scalar opening parameter $\lambda \equiv \sqrt{\lambda \cdot \lambda}$.

In addition, many of these cohesive models have a penalty stiffness multiplier that stiffens the material under compression in order to improve the poor behavior of cohesive surface elements in this regime. The penalty stiffness multiplier is a single number multiplying the natural stiffness that would be calculated by the cohesive surface model.

9.1 Surface Material 0: Xu-Needleman (elastic)

This model due to Xu and Needleman [59] is based on the universal binding energy curve and exhibits complete reversibility; consequently, the traction vector can be derived from a potential $\Phi(\Delta)$ via

$$\mathbf{T} = \frac{\partial \Phi}{\partial \Delta}. \quad (9.2)$$

For the Xu-Needleman model, Φ is a function of the dimensionless gap vector λ described above and is given by

$$\Phi(\Delta) = \phi_n(1 + e^{-\lambda_n})(1 - r + \lambda_n)\frac{1 - q}{r - 1} - e^{-\lambda_t^2}(q + \frac{r - q}{r - 1}\lambda_n). \quad (9.3)$$

Here ϕ_n is an input parameter specifying the work to fracture and one has the identity

$$\phi_n = \int \frac{\partial \phi}{\partial \Delta} \cdot \delta \Delta. \quad (9.4)$$

Two additional dimensionless parameters appearing in the expression for Φ are $q = \frac{\phi_t}{\phi_n}$ relating the normal to shear fracture energies and $r = \frac{\Delta_n^*}{\delta_n}$ which defines the normal opening Δ_n^* after pure shear separation at zero normal traction. The exponential decay of this potential implies an infinite range of interaction between two crack faces, but the Tahoe implementation of this model truncates the potential at a distance $r_{fail} \delta_n$. A plot of the normal traction as a function of Δ_n for pure mode I loading is shown in Figure ?? .

9.1.1 Input parameters

The Xu-Needleman model is implemented for both two- and three-dimensional simulations, and the input parameters are independent of the dimensionality of the problem. An example

of these input parameters is shown below:

```
##### Xu Needleman
0      # surface material code
##### model parameters
1      # q
0      # r
0.1    # dn
0.1    # dt
1.0    # phi_n
100.0  # r_fail
10.0   # penetration stiffness multiplier
```

9.1.2 Output values

This constitutive model does not have any output values.

9.2 Surface Material 1: Tvergaard-Hutchinson (elastic)

As with the Xu-Needleman model, the model of this section due to Tvergaard and Hutchinson [52] is completely reversible and uses the dimensionless gap vector $\boldsymbol{\lambda}$ to characterize the opening displacement. Unlike the Xu-Needleman model, the Tvergaard-Hutchinson model first specifies a potential density ϕ shown in Figure ?? and given by

$$\phi(\boldsymbol{\Delta}) = \begin{cases} \sigma_{max} \frac{\lambda}{\Lambda_1} & : \lambda < \Lambda_1 \\ \sigma_{max} & : \Lambda_1 < \lambda < \Lambda_2. \\ \sigma_{max} \frac{1-\lambda}{1-\Lambda_2} & : \Lambda_2 < \lambda < \Lambda_{fail} \end{cases} \quad (9.5)$$

The potential Φ can be obtained from the potential density via

$$\Phi = \delta_n \int_0^\lambda \phi(\tilde{\lambda}) \, d\tilde{\lambda}, \quad (9.6)$$

and the traction may be concisely expressed as

$$T_i = \frac{\partial \Phi}{\partial \Delta_i} = \frac{\delta_n}{\delta_i} \Lambda_i \phi(\lambda). \quad (9.7)$$

9.2.1 Input parameters

This cohesive model is implemented in two- and three-dimensions, and it should be noted that the zero-stiffness plateau in the traction-separation relation when $\Lambda_1 \neq \Lambda_2$ can lead to poorer-than-quadratic or convergence or no convergence at all in certain problems. An

example of the input parameters for the Tvergaard-Hutchinson model follows:

```
##### Tvergaard-Hutchinson
1      # material code
##### model parameters
100.0 # sigma
0.01  # dn
0.01  # dt
0.1    # Lambda_1
0.9    # Lambda_2
1.0    # Lambda_fail (set to 1 if lambda_fail < 1)
0.     # penetration stiffness multiplier
```

9.2.2 Output values

This constitutive model has a single nodal output value; it prints the value of λ .

9.3 Surface Material 2: Linear Damage Model

This model is not currently implemented in three-dimensions.

9.3.1 Input parameters

An example of input parameters for the linear damage model is shown below:

```
##### linear damage code
2      # surface material code
##### model parameters
0.01  # dn
0.01  # dt
0.0    # penetration stiffness multiplier
```

9.3.2 Output values

This constitutive model does not have any output values.

9.4 Surface Material 3: Tvergaard-Huthcinson with viscous dissipation

This model generalizes the Tvergaard-Hutchinson model described in Section 9.2 to include the effects of viscous dissipation. Consequently, this model does not exhibit reversability upon unloading, and may not be completely described by a potential. The traction $\mathbf{T}$ is now

a function of $\dot{\Delta}$, the rate of change of the gap vector, as well as a function of Δ itself. It may be expressed as

$$\mathbf{T} = \mathbf{T}^{tv} + \eta_0(1 - \lambda)\dot{\Delta} \quad (9.8)$$

where $\mathbf{T}^{tv}$ denotes the traction as a function of Δ specified by the Tvergaard-Hutchinson model in Equation (9.7).

9.4.1 Input parameters

Sample input parameters are shown below:

```
##### Tvergaard-Hutchinson with viscous dissipations
3      # surface material code
##### model parameters
100.0 # sigma
0.01  # dn
0.01  # dt
0.10  # lambda_1
0.90  # lambda_2
1.0   # lambda_fail (set to 1 if lambda_fail < 1)
0.01  # eta_0
0.0   # penalty stiffness multiplier
```

9.4.2 Output values

This constitutive model outputs nodal values of λ and the energy dissipated by the viscous term in the traction-separation relation.

9.5 Surface Material 4: Tijssens' crazing model (elastic-plastic)

This model is due to Tijssens *et al.* [51], and models crazing in amorphous polymers. This cohesive law has three distinct regimes corresponding to elastic deformation until craze initiation, plastic deformation as the craze fibrils lengthen, and finally failure as the crazes lose their load-carrying capacity. The elastic regime is linear elastic with traction-separation relation

$$T_i = k_i \Delta_i \quad (9.9)$$

where the normal and tangential stiffnesses $k_i; i = n, t$ are parameters of the model and summation over repeated indices is not implied. Elastic deformation continues until the stress state in the bulk material surrounding the cohesive layer satisfies a yield condition. For mode I loading, the yield condition reduces to specifying a critical normal traction for craze initiation. For more general loading, the stress tensor may be sampled in the

surrounding bulk, and the initiation criterium is given by a function of T_n and the bulk stress tensor σ

$$f(\sigma, T_n) = -\frac{1}{2}\sigma_{ii} - \frac{1}{2}A_\theta - \frac{B_\theta}{2\sigma_{ii}} - T_n = 0 \quad (9.10)$$

The parameters A_θ and B_θ are temperature dependent according to

$$X_\theta = X_0 e^{-\frac{Q_X}{k_B\theta}}, X = A, B. \quad (9.11)$$

After craze initiation, the traction-separation relation is specified in rate form as

$$\dot{T}_i = k_i(\dot{\Delta}_i - \dot{\Delta}_i^{cr}) \quad (9.12)$$

The components Δ_i^{cr} represent the span of craze fibrils between the surfaces of the cohesive layer and evolve according to

$$\dot{\Delta}_i = \dot{\Delta}_i^0 (e^{-\frac{K^*}{k_B\theta}(\tau_i - T_i)} - \delta_{it} e^{-\frac{K^*}{k_B\theta}(\tau_t - T_t)}). \quad (9.13)$$

Here, $k_B\theta$ is Boltzmann's constant multiplied by the absolute temperature (assumed constant throughout the simulation), K^* and the τ_i are input parameters governing the kinetics of craze growth, and the Kronecker delta, δ_{it} , is included to ensure that T_t is an even function of Δ_t . It is assumed that $\tau_n = \sqrt{3}\tau_t$ and $\dot{\Delta}_t^0 = \sqrt{3}\dot{\Delta}_n^0$ in analogy to von Mises' stresses. Also, the normal elastic stiffness k_n is constant while the tangential elastic stiffness k_t softens according to

$$k_t = k_{t0} e^{-c_t \Delta_n^{cr} / \Delta^{crit}}. \quad (9.14)$$

Once the craze width Δ_n^{cr} reaches a critical opening Δ^{crit} , the craze is assumed to have failed and lost its load-carrying capacity. In the Tahoe implementation of this model, the traction is reduced to zero over n_{fail} timesteps with tangential and normal stiffnesses given by k_t and k_n , respectively. The result for pure mode I loading at constant rate yields the traction-separation curves shown in Figure ???. The hardening behavior is seen when $\dot{\Delta}_n < \dot{\Delta}_n^0$; otherwise softening behavior is observed.

9.5.1 Input Parameters

The input parameters for this model are as shown below:

```
##### Tijssens
4      # surface material code
##### model parameters
100.0 # k_t^0
100.0 # k_n
50.0  # decay constant for k_t^0
0.005 # critical craze opening width
1.0   # A_0 - mixed-mode initiation parameter
1.0   # B_0 - mixed-mode initiation parameter
1.0   # Q_A - mixed-mode initiation parameter
1.0   # Q_B - mixed-mode initiation parameter
0.001 # deltaxdot_n^0 - should be of order the opening rate, Delta_dot
10.0  # tau_n
1.0   # K^star
273.0 # absolute temperature
1     # integer > 0 giving bulk element group to obtain stress from
10    # integer number of timesteps n_fail
```

9.5.2 Output Values

This cohesive model outputs four nodal values. The first is the local normal opening rate $\dot{\Delta}_n$. The second is the tangential craze width, Δ_t^{cr} , and the third is the normal component of the same, Δ_n^{cr} . The final output parameter is the initiation function $f(\sigma, T_n)$.

9.6 Surface Material 5: trilinear rate dependent

This model is a generalization of the Tvergaard-Hutchinson model of Section 9.2; its primary distinction is that the critical normal length scale δ_n varies as a function of the normal opening rate $\dot{\delta}_n$. The material is assumed to yield when it first reaches the plateau traction (for pure mode I loading) σ^m , and at this yield point δ_n is set as a function of rate given by

$$\delta_n = \delta_n^b + \delta_n^m \log \dot{\Delta}_n \quad (9.15)$$

where log denotes the natural logarithm. Before the critical normal traction is reached, the normal length scale is rate-independent and given as an input parameter. Once the normal length scale has been set for a particular integration point, the parameter is finalized there and that point is no longer sensitive to variations in the local opening rate.

A second distinction between this trilinear model and Tvergaard and Hutchinson's is that the traction-separation relation is no longer constrained to have a plateau (region of

zero stiffness); there is an additional parameter m_p allowing nonzero stiffness and better numerical performance. The traction on the plateau is determined by m_p via

$$\mathbf{T}(\lambda) = \mathbf{T}(\Lambda_1)(1 + m_p(\lambda - \Lambda_1)), \quad (9.16)$$

where Λ_1 has the same meaning as in the Tvergaard-Hutchinson model of Section ?? . Likewise, $\mathbf{T}(\Lambda_1)$ is identical to that determined by Surface Material 1 since the loading rate effects the plastic failure portion of the model and not the elastic loading portion.

9.6.1 Input parameters

An example of the input parameters for the trilinear, rate-dependent model is shown below:

```
##### rate-dependent Tvergaard-Hutchinson
5      # surface material code
##### model parameters
100.0 # sigma
0.25  # rate-independent normal length scale, dn
0.25  # dt
0.1   # Lambda_1
0.9   # Lambda_2
1.0   # Lambda_fail (set to 1 if lambda_fail < 1)
0.0   # penalty stiffness multiplier
1.0   # delta_n^b (intercept of rate function)
1.0   # delta_n^m (slope of rate functions)
0.0   # m_p
```

Setting “delta_n.b” equal to “d_n” and “delta_n.m” and “m_p” to zero in the input file reduces this model to the rate-independent Tvergaard-Hutchinson one of Section 9.2.

9.6.2 Output values

This model outputs nodal values for λ , each component of the local opening rate $\dot{\Delta}$, and the rate-dependent normal length scale δ_n .

9.7 Surface Material 6: Tied Potential

This material surface model works in conjunction with the TiedNodes KBC Controller 3, and its behavior is unpredictable unless the TiedNodes controller is activated. This co-dependency is used in an attempt to remedy some shortcomings of cohesive surface models. To begin with, cohesive surface models are in general designed to model the loss of a material’s load-carrying capacity across a crack face, yet by necessity the traction-separation relation must also specify the response of this region of material to loading before this capacity is assumed to have been lost. Consequently, there is *a priori* no relation between the

bulk material and the initial (presumably) elastic loading portion of the traction separation relation. The tied nodes KBC Controller constrains the two faces separated bridged by a cohesive surface to admit no displacement discontinuity across them until an initiation criterion is reached, and consequently, the cohesive surface element is inactive until its nodes are “untied” at initiation. To this end, the tied potential provides a function `TiedPotentialT::InitiationQ` that determines if a critical traction has been reached for yield to occur and for the material to exhibit cohesive failure. Although a minimal initiation function is provided and will be described below, this function should in general be re-implemented by the end user. Since the material response is that of the bulk until the nodes are untied, this model should provide greater numerical stability in situations where the cohesive layer may reasonably be expected to undergo severe compression or other deformations for which a cohesive surface element may be ill-equipped.

Currently, two traction-separation laws may be used after the nodes are untied. The first is the Xu-Needleman model of Section 9.1 and the second is the Tvergaard-Hutchinson model of Section 9.2. The Xu-Needleman implementation begins the traction separation law at the peak stress of $\phi_n/(e\delta_n)$ in the notation of Section 9.1, so that the materials yield criterium is taken to be one of maximum normal traction.

The Tvergaard and Hutchinson tied-potential implementation allows the user to specify the initiation point in terms of the scalar opening parameter λ and the model parameters given in Section 9.2. More concretely, a given value of λ may be inverted to determine a specific traction that acts as the initiation traction. Since the initiation traction is not constrained to be the maximum traction, this choice of tied potential is possibly more robust than the Xu-Needleman choice, but it is plagued by poorer numerical performance.

9.7.1 Input parameters

The tied potential input parameters contain a flag that is set to zero to choose the Tvergaard-Huthcinson model and to one to choose the Xu-Needleman model. Examples of both sets of input will be shown.

Below is shown sample input for a tied Xu-Needleman model:

```

##### Tied potential
6      # surface material code
0. 1. # normal direction to sample stress to obtain traction
##### surface material code for Xu-Needleman
0
##### model parameters
1.0    # q
0.0    # r
0.001  # dn
0.001  # dt
10.0   # phi
10.0   # r_fail

```

For the tied Tvergaard-Hutchinson model:

```

##### Tied potential
6      # surface material code
0. 1. # normal direction to sample stress to obtain traction
##### surface material code for Tvergaard-Hutchinson
1
##### model parameters
100.0  # sigma
0.001  # dn
0.001  # dt
0.25   # lambda_1
0.75   # lambda_2
1.0    # lambda_3
0.25   # lambda_0, initiate at T = T(lambda_0)

```

Although the input parameters have been ordered to correspond exactly with the ones for their respective untied cohesive laws, it should be noted that there are no penalty stiffness multipliers for interpenetration, and that the mode I formalism currently used overrides any effects in the Xu-Needleman specification other than $q = 1$, $r = 0$.

9.7.2 Output values

The tied potentials output a single nodal value that reflects a given nodes status as tied ($\neq 0$) or untied (0.) and may be useful in conjunction with visualization tools.

9.8 Material 7: Yoon, Allen, and Searcy (linear viscoelastic)

This model presented initially by Yoon and Allen and later by Searcy and Allen models a cohesive layer as a linear viscoelastic material that undergoes damage as it loses its load-

carrying capacity. An undamaged material will respond exactly as a linear viscoelastic material, while a material undergoing damage will require the specification of a damage evolution law. The details of the physical interpretation of the damage parameter α are given in Reference [60], and the traction-separation law is given as

$$\mathbf{T} = \boldsymbol{\lambda}(1 - \alpha(t)) \left[T_0 + \int_0^t \left(E_\infty + \sum_i^{n_e} (1 - E_i e^{-\frac{t}{\tau_i}}) \right) \frac{\partial \lambda(\tau')}{\partial \tau'} d\tau' \right] \quad (9.17)$$

where the E_i and τ_i are elastic moduli and relaxation times corresponding to relaxation process i , and E_∞ is the equilibrium elastic modulus as for a Prony series in a standard linear solid. The model as published contains an initiation traction T_0 , but calculations using the model set T_0 to zero, the same convention currently used in Tahoe. There is a choice of damage evolution laws where $\dot{\alpha}$ is specified as a function of λ , $\dot{\lambda}$, and α itself. In all the damage evolution laws, it is assumed that α does not evolve for closing cohesive surfaces, i.e when $\dot{\lambda}$ is non-negative.

The first damage law is given in Reference [2] as

$$\dot{\alpha} = \alpha_0 \lambda^m, \quad (9.18)$$

and the response of this evolution law to cyclic loading is shown in Figure . A second evolution law is given in Reference [60] as

$$\dot{\alpha} = \alpha_0 (1 - \alpha^n) (1 - b \lambda^m), \quad (9.19)$$

and the response of this law to identical cyclic loading as before is shown in Figures . A third and for the moment final damage evolution law is given by

$$\dot{\alpha} = \alpha_0 \dot{\lambda}^m, \quad (9.20)$$

and its response to cyclic loading is shown in Figure .

9.8.1 Input parameters

An example of input parameters for two- or three-dimensional analysis is shown below:

```
##### Yoon-Allen-Searcy
7      # potential code
0.     # fsigma_0
1.     # d_c^t - critical tangential length scale
1.     # d_c^n - critical normal length scale
10.    # E_infinity - equilibrium modulus
2      # number of elements in prony series (nprony)
10.    # first relaxation modulus
10.    # first relaxation time
100.   # second relaxation modulus
2.     # second relaxation time
.
.
.
100.   # nprony_th relaxation modulus
100.   # nprony_th relaxation time
1      # damage evolution law code
      # 1: alpha_dot = alpha_0*l^alpha_exp
      # 2: alpha_dot =
      #      ((alpha_0(1-alpha)^alpha_exp)(1-lambda_0)^lambda_exp)
      # 3: alpha_dot = alpha_0*lambda_dot^alpha_exp
2.     # alpha_exp
10.    # alpha_0
#-3.   # lambda_exp # only for damage code 2
#10.   # lambda_0  # only for damage code 2
10.    # penetration stiffness multiplier
```

9.8.2 Output values

This constitutive model has three nodal output values. It outputs values of λ , $\dot{\lambda}$, and α .

10 Geometry and results files

The user input consists of analysis parameters and a geometry specification. Analysis parameters are specified in a plain text file. Several options exist for the specification of the geometry. Table 10.1 lists the file types available to specify the geometry. These will be described in greater detail in Section 10.1. Table 10.2 lists the file types available for the output of results. Note that some of the file types may be used for both input and results, while others are limited to one or the other.

Table 10.1: Geometry input file formats.

code	description
0	TahoeI
1	TahoeII
3	EnSight [15] Gold ASCII version 6
4	EnSight Gold Binary version 6
5	ExodusII [39]
6	ABAQUS ASCII [?]
7	ABAQUS Binary
10	PATRAN [34]

Table 10.2: Result output file formats.

code	description
0	TahoeI
2	TecPlot [5] version 7
3	EnSight [15] Gold ASCII version 6
4	EnSight Gold Binary version 6
5	ExodusII [39]
6	ABAQUS ASCII [?]
7	ABAQUS Binary
10	PATRAN [34]

10.1 Geometry file formats

Input data is allowed to be one of three database formats listed in Table 10.1:

- (1) **TahoeI**: With this format, geometry data may be supplied embedded in the parameters file or in files separate from, or external to, the parameters files. When specified in external files, the block of geometry data in the parameters file is replaced by the name of the external file where the data resides. The format of the data itself is the same in both cases.
- (2) **TahoeII**: This format allows random access to the geometry data which is stored in one or more plain text files.
- (3) **ExodusII**: This format allows random access to the geometry data which is stored in platform independent binary file.

All **Tahoe** plain text input files are white space delineated, that is, values are separated by white space, but the arrangement of white space is ignored. The **#** symbol denotes that the remainder of a given line of text should be ignored, or treated as a comment.

10.1.1 TahoeI format

The **TahoeI** format includes specifications for model geometry as well as kinematic and force boundary conditions. Each of these input types is described in greater detail in the sections that follow. All of the input types precede the actual input data with some indication of the dimensions of the data. When beginning to read a block of input data, **Tahoe** will determine whether the block begins with an integer or with a character string. A leading integer denotes the dimensions of an embedded block of input data. If **Tahoe** finds a character string, it assumes the string is the name of an external file in which the input data resides. **Tahoe** cannot distinguish file names beginning with an integer from the dimensions of an embedded data block.

10.1.1.1 Coordinate list The format of coordinate data is shown below:

```
# dimensions
[nnd = number of points]
[nsd = number of spatial dimensions]
# coordinates
1      [x11]   ...   [x1nsd]
⋮
[nnd] [xnnd1] ... [xnndnsd]
```

The coordinate of each point is preceded by its index. Coordinates may appear in any order; however, **Tahoe** does not verify that all n_{nd} points have been assigned coordinates exactly once.

10.1.1.2 Element block The format of element block data is shown below:

```
# dimensions
[nel = number of elements]
[nen = number of nodes per element]
# connectivities
1    [n11]    ...  [n1nen]
⋮
[nel] [nnel1] ... [nnelnen]
```

The connectivity of each element is preceded by its index. Connectivities may appear in any order; however, **Tahoe** does not verify that all n_{el} elements have been assigned connectivities exactly once.

10.1.1.3 Node set The format of node set data is shown below:

```
# dimension
[nnd = number of nodes]
# list
[n1] ... [nnnd]
```

10.1.1.4 Side set The format of side set data is shown below:

```
# dimension
[ns = number of facets]
# facet list
[e1] [s1]
⋮
[ens] [sns]
```

Facets are specified by two numbers, the element number e and the side s . The side numbering convention for the various integration domain geometries are shown in Figures 7.1 through 7.6.

10.1.2 TahoeII format

The TahoeII format allows random access to geometry data in a plain text file format. The file format uses the keywords listed in Table 10.3. The keywords must be preceded by an asterisk (*). Aside from the keyword **set**, each keyword denotes a separate type of data. These keywords may appear in any order in a TahoeII file. However, since the file is scanned whenever data is requested, it is more efficient to place longer blocks of data toward the end

Table 10.3: Keywords for the TahoeII file format.

<code>version</code>	<code>title</code>
<code>dimensions</code>	<code>nodes</code>
<code>elements</code>	<code>nodesets</code>
<code>sidesets</code>	<code>set</code>

of the file. Alternately, blocks of data may be specified in files separate from the primary TahoeII file. When specified in an externally, the block of geometry data in the file is replaced by the path of the external file where the data resides. The format of the data itself is the same in both cases.

A TahoeII file contains a single set of coordinates and an arbitrary number of element sets, node sets, and side sets. The format of the sets is similar to the TahoeI format described in Section 10.1.1. The sets are identified by an ID. ID's may be any positive integer and do not need to be consecutive. ID's for each type of set are stored separately. As mentioned above, TahoeII files are plain text and white space delineated, that is, values are separated by white space, but the arrangement of white space is ignored. The `#` symbol denotes that the remainder of a given line of text should be ignored, or treated as a comment. The current TahoeII version is 1.0. The format of each type of data in a TahoeII file is described in the sections that follow.

10.1.2.1 Version The `version` keyword is used to check the compatibility of the database with the current version of the TahoeII code. The current version is 1.0. An example of this section is shown below:

```
# TahoeII file version
*version 1.0
```

10.1.2.2 Title This section begins with the `version` keyword. A one line title may be assigned to the database:

```
# database title
*title
[one line title of 254 characters or less] (10.1)
```

10.1.2.3 Dimensions This section begins with the `dimensions` keyword. It contains the dimensions of the nodal coordinate data, and declares the ID and dimension of each element, node, and side set. This sequence of data in this section must follow the order

shown in the example below:

```
# dimensions and IDs
*dimensions
# dimension of coordinate data
[ $n_{nd}$  = number of nodes]
[ $n_{sd}$  = number of spatial dimensions]
# element set declarations
[ $n_{es}$  = number of element sets]
[ID1] [ $n_{el1}$  = number of elements] [ $n_{en1}$  = number of element nodes]
⋮
[ID $n_{es}$ ] [ $n_{el\ n_{es}}$ ] [ $n_{en\ n_{es}}$ ]
# node set declarations
[ $n_{ns}$  = number of node sets]
[ID1] [ $n_{nd1}$  = number of set nodes]
⋮
[ID $n_{ns}$ ] [ $n_{nd\ n_{ns}}$ ]
# side set declarations
[ $n_{ss}$  = number of side sets]
[ID1] [ $n_{s1}$  = number of facets]
⋮
[ID $n_{ss}$ ] [ $n_{s\ n_{ss}}$ ]
```

If the number of element sets n_{es} , node sets n_{ns} , or side sets n_{ss} is zero, the subsequent list of ID's and dimensions is empty.

10.1.2.4 Node sets This section begins with the **nodesets** keyword. The node sets must appear in the order they have been specified in the dimensions section [10.1.2.3](#). The beginning of the data for each node set is marked by the **set** keyword. The data for each set is comprised of the number of nodes in the set and list of nodes themselves. This data for each set may be defined in line or in an external file. If the data is defined in an external file, the path to the file replaces the data itself in the TahoeII file. In either case, the format of the data itself is the same. The format of each node set specification is the same as the

TahoeI format for node sets [10.1.1.3](#). An example appears below:

```
# node set specifications
*nodesets
*set
  [ $n_{nd1}$  = number of nodes in set 1]
  [ $n_1$ ] ... [ $n_{n_{nd1}}$ ]
  :
*set
  [ $n_{ndn_s}$  = number of nodes in set  $n_s$ ]
  [ $n_1$ ] ... [ $n_{n_{ndn_s}}$ ]
```

The set subscripts for the nodes in each set have been omitted for clarity. Note that the dimensions of the set are from the dimensions section are repeated but that the ID of each set is not repeated. When specifying the data for a node set in an external file, the primary TahoeII contains the path to the external file, as shown below:

```
*set [path to external file]
```

10.1.2.5 Side sets This section begins with the `sidesets` keyword. The side sets must appear in the order they have been specified in the dimensions section [10.1.2.3](#). The beginning of the data for each side set is marked by the `set` keyword. The data for each set is comprised of the number of each in the set and list of facet specifications themselves. Facets are specified by two numbers, the element number e and the side s . The side numbering convention for the various integration domain geometries are shown in Figures [7.1](#) through [7.6](#). This data for each set may be defined in line or in an external file. If the data is defined in an external file, the path to the file replaces the data itself in the TahoeII file. In either case, the format of the data itself is the same. The format of each side set specification is

the same as the TahoeI format for side sets 10.1.1.4. An example appears below:

```
# side set specifications
*sidesets
*set
[ $n_{s1}$  = number of facets in set 1]
[ $e_1$ ] ... [ $s_1$ ]
⋮
[ $e_{n_{s1}}$ ] ... [ $s_{n_{s1}}$ ]
⋮
*set
[ $n_{s n_{ss}}$  = number of facets in set  $n_{ss}$ ]
[ $e_1$ ] ... [ $s_1$ ]
⋮
[ $e_{n_{s n_{ss}}}$ ] ... [ $s_{n_{s n_{ss}}}$ ]
```

The set subscripts for the element and sides in each set have been omitted for clarity. Note that the dimensions of the set are from the dimensions section are repeated but that the ID of each set is not repeated. When specifying the data for a side set in an external file, the primary TahoeII contains the path to the external file, as shown below:

```
*set [path to external file] (10.2)
```

10.1.2.6 Element sets This section begins with the **elements** keyword. The element sets must appear in the order they have been specified in the dimensions section 10.1.2.3. The beginning of the data for each element set is marked by the **set** keyword. The data for each set is comprised of the number of each in the set and list of facet specifications themselves. This data for each set may be defined in line or in an external file. If the data is defined in an external file, the path to the file replaces the data itself in the TahoeII file. In either case, the format of the data itself is the same. The format of each element set specification is the same as the TahoeI format for element sets 10.1.1.2. An example appears

below:

```
# element set specifications
*elements
*set
[nel1 = number of elements in set 1]
  1      [n11]    ...   [n1nen1]
  ⋮
[nel1] [nnel11] ... [nnel1nen1]
⋮
*set
[nel nes = number of elements in set nes]
  1      [n11]    ...   [n1nen nes]
  ⋮
[nel nes] [nnel nes1] ... [nnel nesnen nes]
```

The set subscript on the nodes in each element has been omitted for clarity. The connectivity of each element is preceded by its index. Connectivities may appear in any order; however, Tahoe does not verify that all n_{el} elements have been assigned connectivities exactly once. Note that the dimensions of the set are from the from the dimensions section are repeated but that the ID of each set is not repeated. When specifying the data for a side set in an external file, the primary TahoeII contains the path to the external file, as shown below:

```
*set [path to external file]
```

10.1.2.7 Nodal coordinates The coordinate specification is marked by the **nodes** keyword. The format of coordinate data is the TahoeI format described in Section 10.1.1.1. An example is shown below:

```
# nodal coordinates
*nodes
# dimensions
[nnd = number of points]
[nsd = number of spatial dimensions]
# coordinates
  1      [x11]    ...   [x1nsd]
  ⋮
[nnd] [xnnd1] ... [xnndnsd]
```

The coordinate of each point is preceded by its index. Coordinates may appear in any order; however, Tahoe does not verify that all n_{nd} points have been assigned coordinates exactly once. Coordinate data may be specified in the primary TahoeII file or in an external file. When specifying the data for a side set in an external file, the primary TahoeII contains the path to the external file, as shown below:

```
# nodal coordinates
*nodes [path to external file]
```

(10.3)

10.1.3 EnSight Gold format

EnSight Gold [15] files can be read in version 6 Gold plain text or binary format. The EnSight binary files are portable between some platforms. When an EnSight database is read, the user should enter the case file name, including the extension, when prompted for a database filename.

10.1.4 ExodusII/Genesis format

The ExodusII [39] finite element geometry and results file format was developed at Sandia National Laboratories. This file format is export-controlled, and therefore is not available to all users. A given database should contain a coordinate list, element blocks, and any node set or sides sets references from the parameters file. Tahoe does not read material data from the ExodusII database. Node and element maps are currently ignored and items are assumed to be consecutively numbered. To refer to a specific set or block in the database use the ID value. The blocks and sets do not need to have consecutive ID values.

10.1.5 ABAQUS Results Files format

ABAQUS [?] results files in ASCII or binary (*.fil/*.fin) are provided as an input option. Typically as an input option for translating database information. The Tahoe interface supports 5.8 and 6.2 versions and maybe others, depending upon version variations. A given database should contain a coordinate list, element list, node sets, and element sets. Nodes and elements do not have to be consecutively numbered. ABAQUS results files have sets for both elements and nodes. The toolbox interface only reads the first eight characters of the set name. No spaces are allowed. ABAQUS does not have database records pertaining to side sets.

10.1.6 MSC PATRAN Neutral format

MSC PATRAN [34] neutral files are created using the export command within PATRAN. A given database should contain a coordinate list, element list, named component groups that contain element blocks and/or node sets. Nodes and elements do not have to be consecutively numbered. To refer to an element block or node set, use the group name. PATRAN group

names are created from the Group pull down menu and are typically used for posting different layers. They are exported to a Neutral file as Named Components. When writing the Neutral file, PATRAN will truncate the group name to 12 characters and write it in upper case. The group names must not contain spaces. The Neutral files do not have anything explicitly pertaining to side sets, but if desired, a pseudo side set could be established using contact surface boundaries.

10.1.7 Tahoe Results format

Tahoe results files are the output files written in Tahoe format. This format differs from Tahoe I or Tahoe II input file format and contains variable results. When prompted for the database filename, the user should supply the geometry (*.geo) or results (*.run) file including the extension.

10.2 Results file formats

Output data is associated with an output group. An output group is not necessarily the same as the element group. Element groups can have more than one connectivity set for post-processing purposes. Some element groups have no output. These connectivity sets are not the element blocks listed in the parameters file. For instance, to visualize an internal 3D crack, those elements that represent the crack (localized elements or cohesive surfaces) are separated into an additional connectivity within the element group. This allows post-processors to easily show the crack without having to create isosurfaces or other body representations.

Variable output data is either associated with a node or an element. Nodal data are variable values defined at the nodes within the output group. Element data are variable values associated with the elements. Each element variable has one value per element. All element quadrature data is projected to the nodes using the smoothing algorithm proposed by Zienkiewicz [?].

For the purposes of writing output files, the output print increment is a value starting at one and is consecutively incremented. Thus the print increment is not the same as the computational increments used by the program. This accounts for changes in the frequency with which computational increments are printed and some I/O databases demand consecutive numbering.

Depending upon the type of output database chosen, different output files are created. File extensions are listed in Table 10.4. The log/output file is always created and contains an echo of the input data. If multiple loading sequences are used, the extensions for geometry and variable data will be lengthened to include sequence numbers. **EnSight** uses the variable labels as file extensions on its variable files. For **ExodusII** and **TecPlot**, the variable labels are used as variable names. The nodes are broken up into those used by the connectivities. Node numbers coincide with the internal consecutive node numbering. As described in Section 2.4, **Tahoe** has two output modes for parallel execution. **Tahoe** can join the results data from all the processors at run time, or the distributed results data may be joined during a separate

Table 10.4: File extensions for results files.

file type	extension	description
plain text	<code>.out</code>	input echo/runtime log
	<code>.io[group].fracture</code>	fracture surface area
	<code>.nd[node].hst</code>	nodal history
	<code>.rs[step]</code>	restart data
TahoeI	<code>.geo</code>	output geometry
	<code>.run</code>	output results
TecPlot	<code>.io[group].dat</code>	output results
EnSight	<code>.io[group].case</code>	case file
	<code>.io[group].geo</code>	output geometry
	<code>.io[group].ps[step].geo</code>	changing output geometry
	<code>.io[group].ps[step].[label]</code>	output variable
ExodusII	<code>.io[group].exo</code>	output results
	<code>.io[group].ps[step].exo</code>	changing geometry output

post-processing step. With distributed output files, the extension `.p[processor]` is prepended to each of the extensions listed in Table 10.4.

10.2.1 TecPlot

The TecPlot [5] files are currently written in version 7.5 plain text format. Element data is ignored and not written to an output file. Element data must be averaged or projected to the nodes. TecPlot plain text files can be converted to binary files, which are generally portable, using the PrePlot executable supplied by TecPlot.

10.2.2 EnSight

EnSight [15] files are written in version 6 Gold plain text or binary format. The EnSight binary files are generally portable between some platforms. Users should conduct a preliminary test to verify this portability. Version 6 Gold files are readable by EnSight version 6 Gold and by EnSight version 7 or later. The EnSight case file is written at each print increment to account for unexpected machine crashes. Each output group is treated as a part and is written to a separate geometry and case file. Separate files are produced so only the part that changes will be printed at each print increment. Thus reducing the amount of file space needed for large meshes. Element and node tags are not given. Any vector variable internally named with the $\{_X, _Y, _Z\}$ convention will be written to a vector variable file instead of a scalar variable file.

10.2.3 ExodusII

ExodusII files are portable to all platforms that have the library installed on them. Each block within an output group is treated as a block. Separate files are written for each output group. Separate files are produced so only the block that changes will be printed at each print increment. Separate files are also produced so that nodal data from different output groups does not need to be merged. Element numbering and ordering maps are not given. The nodes are broken up into those used by the connectivities. ExodusII files, from multiple restarts, can be joined together using the `conex` executable.

10.2.4 ABAQUS Results

ABAQUS results files are created for version 5.8. These files could be used in Post. Each block within an output group is treated as an element set. Separate files are written for each output group.

10.2.5 PATRAN Results

PATRAN results files are created following the format for version 2001. These files could be used in Post. Each block within an output group is treated as a Named Component. For each output group, a geometry file, displacement file, force file, node variable file and element variable file is written (if variable data exists).

10.2.6 Fracture surface area

The fracture surface area file contains columnar data, in the order listed as follows:

- (1) time
- (2) fracture surface area

10.2.7 Node history output

The history node file contains columnar data, in the order listed as follows.

- (1) time
- (2) displacement vector
- (3) velocity vector (if dynamic or transient)
- (4) acceleration vector (if dynamic)
- (5) force vector (if prescribed DOF exists)

11 History of updates to the Tahoe User Guide

11.1 History

*[To do: brief history:
originated by Stephanie Wimmer
updated to Version 3.4 by Patrick Klein.]*

11.2 Changes since Version 3.4

This section summarizes changes to the Tahoe input format since Version 3.4.

11.2.1 update 3.4.1

Two additional output variables have been added to the element output for solid deformation elements listed in Table 7.7. This brings the number of element output flags from 5 to 7. This change is version aware. Only 5 flags are read if the input file version is older than 3.4.1.

11.3 Modernisation track (post-3.4.1, GitHub)

After the move to GitHub (see Section 2.1), the following capability changes have landed on top of the v3.4.1 input format. The format itself remains backwards compatible; the items below describe new *features*, new *elements*, new *benchmarks*, and infrastructure cleanups that are not yet folded into the per-section reference material elsewhere in this guide. Pull-request numbers refer to the GitHub tracker.

11.3.0.1 Build system. The legacy `tahoe-manager` Perl driver and CVS/SourceForge workflow have been removed. The project now builds with CMake (`root`, `toolbox/`, `tahoe/`, plus per-library `CMakeLists.txt` files); all bundled third-party C libraries (`expat`, `SP00LES`, `f2c`) are compiled from source under `third_party/` (PR #44). See Section 2.1 for build invocations and Section 1.3 for supported platforms.

11.3.0.2 Activated Drucker–Prager plasticity (Material 11). The small-deformation Drucker–Prager constitutive model is now wired into the active element-type registry and verified on the level.1 material suite (PR #37). The mathematical formulation in Section “Material 11” of this guide already documented the model; it is now available in stock builds without re-enabling.

11.3.0.3 Implicit Coulomb friction in `PenaltyContact3DT`. The 3D penalty-contact element now contributes a consistent tangent contribution for Coulomb friction in implicit analyses (PR #40 / #45). Previously, only the residual was assembled, which forced the

nonlinear solver to fall back on a linear convergence rate. With the LHS contribution in place, frictional contact problems now converge quadratically in Newton.

11.3.0.4 BonetTet element with lagged-J-bar and FD tangent. A new finite-deformation Tet4 element `<bonet_tet>` has been added, following the F-bar / ANP averaging of Bonet *et al.* (PR #29). The implementation caches J-bar at step start and builds the local 12×12 tangent through a forward finite-difference of the F-bar residual. On moderately near-incompressible cases ($\kappa/\mu \sim 2.5$), Newton converges quadratically in ~ 1 iter/step — a fifteenfold iteration reduction relative to the linear-rate baseline. A fully consistent analytical tangent (which would handle the cross-element $\partial \bar{J} / \partial \mathbf{u}$ coupling required at severe near-incompressibility) is tracked as future work.

11.3.0.5 New benchmarks. The following benchmarks have been added to `benchmark_XML/level.5/` (PRs #47, #48, #49 and the broader explicit-solver track):

- `brinell/` — Brinell indentation of a rigid sphere on a J_2 -plastic Hex8 block, exercising contact + plasticity together.
- `tensile/` — Quarter-symmetry uniaxial tension on a slim Hex8 bar; isolates the J_2 radial-return from contact and verifies that the implementation tracks an input `cubic_spline` $\sigma_y(\varepsilon^p)$ hardening curve within 2% under monotonic loading.
- `implicit_friction/` — Companion regression case for the new implicit Coulomb friction tangent (PR #40 / #45).
- `hertz/` — Hertz contact validation case, including a mixed Tet4 indenter on Hex8 base variant for the Tet element track.
- `tet_classic/` — Classic-Tahoe implicit Tet4 reference for the BonetTet element (PR #29).

All per-level benchmark directories now carry a `README.md` that catalogues the input files and references the originating issue (PR #42).

11.3.0.6 Documentation infrastructure. The User Guide source previously depended on `hangcaption.sty`, which was dropped from modern T_EX Live. The `\hangcaption` macro is now aliased to `\caption` in `tahoe.user.tex`; modern `\caption` already hangs the label. No figure layouts change as a result.

11.3.0.7 Meshfree Kirchhoff–Love shell. The rotation-free meshfree shell element `meshfree_kl_shell` following Wang and Bazilevs [56] (Section 7.3), with its parameters (Table 7.3), output values (Table 7.4) and verification (Table 7.5); added to Table 7.1 (issues #59–#73).

11.3.0.8 Essential boundary conditions for meshfree fields. The kinematic boundary condition controller `collocation_KBC` and the `collocation` flag on `kinematic_BC` (#70), and the force controller `penalty_displacement_meshfree`, which reports the physical reaction (Section 6.6). The controller tables (Tables 6.3 and 6.4) now list every controller in the code with its XML name.

11.3.0.9 Meshfree shell regression tests. Level.0 and level.2 decks for the shell and its boundary conditions (Section 3.3, #73).

References

- [1] M.D. Alaydin, D.J. Benson, and Y. Bazilevs, *An updated Lagrangian framework for isogeometric Kirchhoff–Love thin-shell analysis*, Computer Methods in Applied Mechanics and Engineering **384** (2021), 113977.
- [2] D.H. Allen and C.R. Searcy, *A micromechanical model for a viscoelastic cohesive zone*, International Journal of Fracture **107** (2001), 159–176.
- [3] M. Ambati, J. Kiendl, and L. De Lorenzis, *Isogeometric Kirchhoff–Love shell formulation for elasto-plasticity*, Computer Methods in Applied Mechanics and Engineering **340** (2018), 320–339.
- [4] P.R. Amestoy, I.S. Duff, J.-Y. L’Excellent, and J. Koster, *A fully asynchronous multi-frontal solver using distributed dynamic scheduling*, SIAM Journal on Matrix Analysis and Applications **23** (2001), no. 1, 15–41.
- [5] Amtec Engineering, Inc., *TecPlot User Manual*, 8.0 ed., 1999.
- [6] C. Argento, A. Jagota, and W.C. Carter, *Surface formulation for molecular interactions of macroscopic bodies*, Journal of the Mechanics and Physics of Solids **45** (1997), 1161–1183.
- [7] C. Ashcraft, D. Pierce, D.K. Wah, and J. Wu, *The reference manual for **SPOOLES***, Boeing Shared Services Group, 2.2 ed., 1999.
- [8] S. Atluri and T. Zhu, *A new meshless local Petrov-Galerkin approach in computational mechanics*, Computational Mechanics **22** (1998), 117–127.
- [9] Z.P. Bažant, *Efficient numerical integration on the surface of a sphere*, Zeitschrift für Angewandte Mathematik und Mechanik **66** (1986), 37–49.
- [10] T. Belytschko, Y. Krongauz, D. Organ, M. Fleming, and P. Krysl, *Meshless methods: an overview and recent developments*, Computer Methods in Applied Mechanics and Engineering **139** (1996), 3–47.
- [11] T. Belytschko, Y.Y. Lu, and L. Gu, *Element-free Galerkin methods*, International Journal for Numerical Methods in Engineering **37** (1994), 229–256.
- [12] T. Belytschko, H. Stolarski, W.K. Liu, N. Carpenter, and J.S.-J. Ong, *Stress projection for membrane and shear locking in shell finite elements*, Computer Methods in Applied Mechanics and Engineering **51** (1985), 221–258.
- [13] M. Born, *On the stability of crystals*, I., Proceedings of the Cambridge Philosophical Society **36** (1940), 160–172.

- [14] J.S. Chen, C.T. Wu, S. Yoon, and Y. You, *A stabilized conforming nodal integration for Galerkin mesh-free methods*, International Journal for Numerical Methods in Engineering **50** (2001), 435–466.
- [15] Computational Engineering International, Inc., *EnSight User Manual*, 6.2 ed., 1998.
- [16] M.S. Daw and M. Baskes, *Embedded atom method: derivation and application to impurities, surfaces, and other defects in metals*, Physical Review B **29** (1984), 6443–6453.
- [17] F. Ercolessi and J.B. Adams, *Interatomic potentials from first-principles calculations: the force-matching method*, Europhysics Letters **28** (1994), 583–588.
- [18] D.P. Flanagan and L.M. Taylor, *An accurate numerical algorithm for stress integration with finite rotations*, Computer Methods in Applied Mechanics and Engineering **62** (1987), 305–320.
- [19] H. Gao and P. Klein, *Numerical simulation of crack growth in an isotropic solid with randomized internal cohesive bonds*, Journal of the Mechanics and Physics of Solids **46** (1998), 187–218.
- [20] R.A. Gingold and J.J. Monaghan, *Smoothed particle hydrodynamics: theory and application to non-spherical stars*, Monthly Notices of the Royal Astronomical Society **181** (1977), 375–389.
- [21] J.-H. Heegaard and A. Curnier, *An augmented Lagrangian method for discrete large-slip contact problems*, International Journal for Numerical Methods in Engineering **36** (1993), 569–593.
- [22] Hibbit, Karlsson & Sorenson, Inc., *ABAQUS/Standard*, 5.6 ed., 1996.
- [23] H.M. Hilber, T.J.R. Hughes, and R.L. Taylor, *Improved numerical dissipation for time integration algorithms in structural dynamics*, Earthquake Engineering and Structural Dynamics **5** (1977), 283–292.
- [24] M. Hillman and J.S. Chen, *An accelerated, convergent, and stable nodal integration in Galerkin meshfree methods for linear and nonlinear mechanics*, International Journal for Numerical Methods in Engineering **107** (2016), 603–630.
- [25] P. Klein and H. Gao, *Crack nucleation and growth as strain localization in a virtual-bond continuum*, Engineering Fracture Mechanics **61** (1998), 21–48.
- [26] P.A. Klein, *A virtual internal bond approach to modeling crack nucleation and growth*, Ph.D. thesis, Stanford University, 1999.
- [27] P.A. Klein and H. Gao, *Study of crack dynamics using the Virtual Internal Bond model*, James R. Rice 60th Anniversary Volume of Solid Mechanics and its Applications (G.M.L. Gladwell, ed.), Kluwer Academic Publishers, in press.

- [28] P. Lancaster and K. Salkauskas, *Surfaces generated by moving least squares methods*, Mathematics of Computation **37** (1981), 141–158.
- [29] W.K. Liu and Y. Chen, *Reproducing kernel particle methods*, International Journal for Numerical Methods in Fluids **20** (1995), 1081–1106.
- [30] W.K. Liu, Y. Chen, C.T. Chang, and T. Belytschko, *Advances in multiple scale kernel particle methods*, Computational Mechanics **18** (1996), 73–111.
- [31] W.K. Liu, Y. Chen, R.A. Uras, and C.T. Chang, *Generalized multiple scale reproducing kernel particle methods*, Computer Methods in Applied Mechanics and Engineering **139** (1996), 91–158.
- [32] W.K. Liu, S. Hao, T. Belytschko, S. Li, and C.T. Chang, *Multiple scale meshfree methods for damage fracture and localization*, Computational Materials Science **16** (1999), 197–205.
- [33] ———, *Multi-scale methods*, International Journal for Numerical Methods in Engineering **47** (2000), 1343–1361.
- [34] MSC Software Corporation, *MSC PATRAN: User Manual*, 2001 ed., 2001.
- [35] B. Nayroles, G. Touzot, and P. Villon, *Generalizing the finite element method: Diffuse approximation and diffuse elements*, Computational Mechanics **10** (1992), 307–318.
- [36] R.W. Ogden, *Non-linear elastic deformations*, John Wiley and Sons, Inc., New York, 1984.
- [37] R.A. Regueiro and R.I. Borja, *A finite element model of localized deformation in frictional materials taking a strong discontinuity approach*, Finite Elements in Analysis and Design **33** (1999), 283–315.
- [38] J.H. Rose, J. Ferrante, and J.R. Smith, *Universal binding energy curves for metals and bimetallic interfaces*, Physical Review Letters **47** (1981), 675–678.
- [39] L.A. Schoof and V.R. Yarberrry, *ExodusII: A finite element data model*, Tech. Report SAND92-2137, Sandia National Laboratory, 1992.
- [40] J. Schulze, *Towards a tighter coupling of bottom-up and top-down sparse matrix ordering methods*, BIT Numerical Mathematics **41** (2001), no. 4, 800–841.
- [41] J.C. Simo, *A framework for finite strain elastoplasticity based on maximum plastic dissipation and the multiplicative decomposition: part I. continuum formulation*, Computer Methods in Applied Mechanics and Engineering **66** (1988), 199–219.
- [42] ———, *A framework for finite strain elastoplasticity based on maximum plastic dissipation and the multiplicative decomposition: part II. computational aspects*, Computer Methods in Applied Mechanics and Engineering **68** (1988), 1–31.

- [43] ———, *Algorithms for static and dynamic multiplicative plasticity that preserve the classical return mapping schemes of the infinitesimal theory*, Computer Methods in Applied Mechanics and Engineering **99** (1992), 61–112.
- [44] J.C. Simo, F. Armero, and R.L. Taylor, *Improved versions of assumed enhanced strain tri-linear elements for 3D finite deformation problems*, Computer Methods in Applied Mechanics and Engineering **110** (1993), 359–386.
- [45] J.C. Simo and T.J.R. Hughes, *Computational inelasticity*, Springer, New York, 1998.
- [46] J.C. Simo and R.L. Taylor, *Quasi-incompressible finite elasticity in principal stretches. continuum basis and numerical algorithms*, Computer Methods in Applied Mechanics and Engineering **85** (1991), 273–310.
- [47] J.C. Simo, R.L. Taylor, and K.S. Pister, *Variational and projection methods for the volume constraint in finite deformation elastoplasticity*, Computer Methods in Applied Mechanics and Engineering **51** (1995), 177–208.
- [48] I. Stakgold, *The Cauchy relations in a molecular theory of elasticity*, Quarterly of Applied Mechanics **8** (1950), 169–186.
- [49] F.H. Stillinger and T.A. Weber, *Computer simulation of local order in condensed phases of silicon*, Physical Review B **31** (1985), 5262–5271.
- [50] E.B. Tadmor, M. Ortiz, and R. Phillips, *Quasicontinuum analysis of defects in solids*, Philosophical Magazine A **73** (1996), 1529–1563.
- [51] M.G.A. Tijssens, E. van der Giessen, and L.J. Sluys, *Modeling of crazing using a cohesive surface methodology*, Mechanics of Materials **32** (2000), 19–35.
- [52] V. Tvergaard and J.W. Hutchinson, *The relation between crack growth resistance and fracture process parameters in elastic-plastic solids*, Journal of the Mechanics and Physics of Solids **40** (1992), 1377–1397.
- [53] P. Underwood, *Dynamic relaxation*, Computational methods for transient analysis (T. Belytschko and T.J.R. Hughes, eds.), Elsevier Science, 1983, pp. 245–265.
- [54] A.F. Voter, *The embedded atom method*, Intermetallic Compounds: Principles and Practice (J.H. Westbrook and R.L. Fleischer, eds.), John Wiley and Sons, Ltd., 1994, p. 77.
- [55] A.F. Voter and S.P. Chen, *Accurate interatomic potentials for Ni, Al, and Ni₃Al*, Proceedings of the 1987 MRS Fall Symposium, Characterization of Defects in Materials, vol. 82, Materials Research Society, 1987, pp. 175–180.
- [56] J. Wang and Y. Bazilevs, *A general-purpose meshfree Kirchhoff–Love shell formulation*, Engineering with Computers **41** (2025), 1379–1410.

REFERENCES

- [57] J.H. Weiner, *Statistical mechanics of elasticity*, John Wiley and Sons, Inc., New York, 1983.
- [58] Z. Xianyi, W. Qian, and Z. Yunquan, *OpenBLAS: an optimized BLAS library*, <https://www.openblas.net>, 2012.
- [59] X.-P. Xu and A. Needleman, *Numerical simulations of fast crack growth in brittle solids*, *Journal of the Mechanics and Physics of Solids* **42** (1994), 1397–1434.
- [60] C. Yoon and D.H. Allen, *Damage dependent constitutive behavior and energy release rate for a cohesive zone in a thermoviscoelastic solid*, *International Journal of Fracture* **96** (1999), 55–74.